

References — Git and Source Code Version Control

Contents

1 External sources

- 1.1 S1
- 1.2 S2
- 1.3 S3
- 1.4 S4
- 1.5 S5
- 1.6 S6
- 1.7 S7
- 1.8 S8
- 1.9 S9
- 1.10 S10
- 1.11 S11
- 1.12 S12
- 1.13 S13
- 1.14 S14
- 1.15 S15
- 1.16 S16
- 1.17 S17
- 1.18 S18
- 1.19 S19
- 1.20 S20
- 1.21 S21
- 1.22 S22
- 1.23 S23
- 1.24 S24
- 1.25 S25
- 1.26 S26
- 1.27 S27
- 1.28 S28
- 1.29 S29
- 1.30 S30
- 1.31 S31
- 1.32 S32
- 1.33 S33

[1.34 S34](#)

[1.35 S35](#)

[1.36 S36](#)

[1.37 S37](#)

[1.38 S38](#)

[1.39 S39](#)

[1.40 S40](#)

[1.41 S41](#)

[1.42 S42](#)

[1.43 S43](#)

[1.44 S44](#)

[1.45 S45](#)

[1.46 S46](#)

[1.47 S47](#)

2 Vault sources

[2.1 V1 — Git's Object Model and Design Trade-offs](#)

[2.2 V2 — Git's origin, object model, and design trade-offs](#)

[2.3 V3 — Git's Origin, Object Model, and Design Trade-offs](#)

[2.4 V4 — Git's origin, object model, and design trade-offs](#)

[2.5 V5 — Git's Content-Addressable Object Model and Design Trade-offs](#)

[2.6 V6 — Git's Origin, Object Model, and Design Trade-offs](#)

[2.7 V7 — Git's Origin, Object Model, and Design Trade-offs](#)

[2.8 V8 — Git's origin, object model, and design trade-offs](#)

[2.9 V9 — Git's Origin, Object Model, and Design Trade-offs](#)

[2.10 V10 — Git's origin, object model, and design trade-offs](#)

[2.11 V11 — Three-Way Merge Computation, Rebasing, and First-Class Conflict Representation](#)

[2.12 V12 — Three-Way Merge Algorithms and Conflict Representation in Git and Jujutsu](#)

[2.13 V13 — Three-way merge algorithms and conflict representation](#)

[2.14 V14 — Three-way merge computation, criss-cross bases, and Jujutsu's first-class conflicts](#)

[2.15 V15 — Three-Way Merge Computation and Conflict Representation in Git and Jujutsu](#)

[2.16 V16 — Git's Remote-Tracking Refs and the Fetch/Push/Pull Boundary](#)

[2.17 V17 — Remotes, Remote-Tracking Refs, and the Fetch/Push/Pull Boundary](#)

[2.18 V18 — Remotes, Remote-Tracking Refs, and Refspecs in Git](#)

[2.19 V19 — Remotes, remote-tracking refs, and refspecs](#)

[2.20 V20 — Git's Remote, Fetch, Push, and Pull Mechanics](#)

[2.21 V21 — Git remotes, remote-tracking refs, and what fetch, push, and pull do to them](#)

[2.22 V22 — The relocation of version-control centralisation from the data model into policy](#)

[2.23 V23 – The relocation of centralisation from data model to policy](#)

[2.24 V24 – Centralisation relocated from the data model into policy](#)

[2.25 V25 – Centralisation relocated from the data model into policy](#)

[2.26 V26 – Jujutsu as a redesign of Git's user model](#)

[2.27 V27 – Jujutsu's redesign of Git's user model while keeping Git's storage](#)

[2.28 V28 – Patch-based version control's commutation model](#)

[2.29 V29 – The algebra of commuting changes in Darcs and Pijul](#)

[2.30 V30 – Patch theory, as implemented by Darcs and Pijul](#)

[2.31 V31 – the Gitless critique of Git's conceptual design](#)

[2.32 V32 – Git's two families of monorepo scaling fixes](#)

[2.33 V33 – Trunk-based versus branch-based Git workflows](#)

[2.34 V34 – Branch lifetime as the variable behind branching-strategy trade-offs](#)

[2.35 V35 – Trunk-Based versus Branch-Based Development](#)

[2.36 V36 – Branching Strategies, Trunk-Based versus Branch-Based Development](#)

[2.37 V37 – Trunk-based versus branch-based Git development strategies](#)

[2.38 V38 – Code review and the pull request as a review mechanism](#)

[2.39 V39 – Code review and the pull request as a review mechanism](#)

[2.40 V40 – Code Review and the Pull Request as a Review Mechanism](#)

[2.41 V41 – Code Review as a Bounded Comprehension Activity, and the Pull Request's Place in Its History](#)

[2.42 V42 – Stacked diffs as a code-review workflow](#)

[2.43 V43 – Stacked diffs as a code-review workflow](#)

[2.44 V44 – Continuous Integration and Continuous Delivery](#)

[2.45 V45 – Continuous Integration and Continuous Delivery](#)

[2.46 V46 – DORA's five software delivery performance metrics](#)

[2.47 V47 – Deployment Frequency as a DORA metric](#)

[2.48 V48 – The limitations of DORA metrics for measuring AI-assisted software development](#)

[2.49 V49 – The small batches principle in software delivery](#)

[2.50 V50 – Copy-on-write as a cross-scale sharing mechanism](#)

[2.51 V51 – Code review as a social, comprehension-bound process built on top of Git](#)

[2.52 V52 – Code review's measured function, and the pull request's place in its history](#)

[2.53 V60 – Code review and the pull request as a review mechanism](#)

[2.54 V61 – Code Review and the Pull Request as a Review Mechanism](#)

[2.55 V62 – Git's content-addressable object model and design trade-offs](#)

[2.56 V63 – Git's Content-Addressable Object Model and Design Trade-offs](#)

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

v1 · 4e9f6b455 · built 2026-10-08

This document lists every source the talk cites. [S<n>] entries are external sources a reader can open. [V<n>] entries are notes from the author's private research vault. The vault is not public, so each [V] entry carries a summary of the note and every external source the note draws on.

1 External sources

1.1 S1

- **Type:** Reference
- **Credibility:** High — the official Git book, published on the project's own site under a Creative Commons licence.
- **Relevance:** Part I. The four object types, the plumbing sequence the demonstration follows, and the `.git/objects` layout and file modes.
- **URL:** <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>

The spans relied on: content addressing writes a file into `.git/objects/` named by the hash of a header plus the content; a blob holds file content and nothing else; a tree holds a mode, a type, a filename and a hash per entry; `100644`, `100755` and `120000` are the only valid blob modes.

1.2 S2

- **Type:** Reference
- **Credibility:** High — official project documentation, written by the maintainers of the tool it describes.
- **Relevance:** Part I. The founding sentence of the whole topic, and the plumbing-versus-porcelain split that makes the demonstration possible.
- **URL:** <https://git-scm.com/book/en/v2/Git-Internals-Plumbing-and-Porcelain>

The span relied on: Git is "a content-addressable filesystem with a VCS user interface written on top of it", and the low-level commands are called plumbing while the user-facing ones are called porcelain.

1.3 S3

- **Type:** Reference
- **Credibility:** High — official project documentation for the reference layer it defines.
- **Relevance:** Parts I and II. A branch as a file under `.git/refs/heads/`, `HEAD` as a symbolic reference, detached `HEAD`, and `update-ref`.
- **URL:** <https://git-scm.com/book/en/v2/Git-Internals-Git-References>

The spans relied on: a branch file contains one hash; `HEAD` normally contains the text `ref: refs/heads/master`; committing rewrites the branch file that `HEAD` points at.

1.4 S4

- **Type:** Reference
- **Credibility:** High — official project documentation, and the source of the snapshot framing itself.
- **Relevance:** Part I. Snapshots rather than differences, the reuse of an unchanged blob, and the locality of nearly every operation.
- **URL:** <https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>

The spans relied on: Git thinks of its data as a series of snapshots; a file that has not changed is stored as a link to the identical previous file; everything is checksummed before it is stored.

1.5 S5

- **Type:** Reference
- **Credibility:** High — official project documentation for the storage format it specifies.
- **Relevance:** Part I. Loose objects are not delta-compressed and packfiles are, which is the honest second half of the snapshot claim.
- **URL:** <https://git-scm.com/book/en/v2/Git-Internals-Packfiles>

The span relied on: Git periodically packs loose objects into a packfile and only there stores similar objects as deltas against each other.

1.6 S6

- **Type:** Primary source
- **Credibility:** High — Git's own glossary, the project's normative definitions of its own vocabulary.
- **Relevance:** Parts II and VI. Bare repository, working tree, fast-forward, remote-tracking branch, and the statement of the push rule.
- **URL:** <https://git-scm.com/docs/gitglossary>

The spans relied on: a bare repository "does not have a locally checked-out copy of any of the files under revision control"; the working tree is "The tree of actual checked out files"; a fast-forward is "a special type of merge where you have a revision and you are 'merging' another branch's changes that happen to be a descendant of what you have"; a remote-tracking branch is "A ref that is used to follow changes from another repository"; "If the remote head is not an ancestor to the local head, the push fails".

1.7 S7

- **Type:** Primary source
- **Credibility:** High — the manual page for the command that implements the definition, maintained with the code.
- **Relevance:** Part II. The merge base is defined by ancestry ordering, not by age, and it can be plural.
- **URL:** <https://git-scm.com/docs/git-merge-base>

The spans relied on: "A common ancestor that does not have any better common ancestor is a best common ancestor, i.e. a merge base"; "there can be more than one best common ancestor for two commits"; "When the `--all` option is not given, it is unspecified which best one is output".

1.8 S8

- **Type:** Primary source
- **Credibility:** High — the manual page for the command whose behaviour the slides ask the room to predict.
- **Relevance:** Part II. The parent list of a merge commit, what a fast-forward does not create, and the definition of `--squash`.
- **URL:** <https://git-scm.com/docs/git-merge>

The spans relied on: outside a fast-forward "the branches to be merged must be tied together by a merge commit that has both of them as its parents"; in a fast-forward `HEAD` "is updated to point at the named commit, without creating an extra merge commit"; `--squash` produces the merge result but does not "record `$GIT_DIR/MERGE_HEAD`"; the index records up to three stages during a conflict; on the endpoints-only property, "only the heads and the merge base are considered when performing a merge, not the individual commits"; and, for the default `ort` strategy, "When there is more than one common ancestor that can be used for 3-way merge, it creates a merged tree of the common ancestors and uses that as the reference tree for the 3-way merge".

1.9 S9

- **Type:** Primary source
- **Credibility:** High — the manual page for the replay loop the slides describe.
- **Relevance:** Part II. Rebase as a sequence of replays, each equivalent to a cherry-pick, ending in a branch-pointer move.
- **URL:** <https://git-scm.com/docs/git-rebase>

The spans relied on: "Replay the commits, one by one, in order. This is similar to running `git cherry-pick` for each commit"; the documentation's own diagram renames the results `A' --B' --C'`.

1.10 S10

- **Type:** Primary source
- **Credibility:** High — the manual page for the command whose refusal the room is asked to read.
- **Relevance:** Part II. The fast-forward rule, its stated reason, and `--force-with-lease` as a compare-and-swap.
- **URL:** <https://git-scm.com/docs/git-push>

The spans relied on: "only fast-forward updates are allowed, which means the destination must be an ancestor of the source commit"; "a non-fast-forward update will lose history"; `--force-with-lease` protects a ref "by requiring their current value to be the same as the remote-tracking branch we have for them"; the manual's own metaphor of taking a lease on the ref without explicitly locking it.

1.11 S11

- **Type:** Primary source
- **Credibility:** High — the manual page for the command, including the default refspec it ships.
- **Relevance:** Part II. What a fetch touches, and the default refspec that maps the remote's branches into `refs/remotes/origin/*`.
- **URL:** <https://git-scm.com/docs/git-fetch>

The spans relied on: fetch gets refs "along with the objects necessary to complete their histories", and remote-tracking branches are updated; the configured `fetch = +refs/heads/*:refs/remotes/origin/*` "will fetch all branches that exist in the `origin`".

1.12 S12

- **Type:** Primary source
- **Credibility:** High — the manual page, cited here for a statement the vault reproduced and found inexact.
- **Relevance:** Part II. The manual names fast-forward-only as the default, and the reproduction shows the unset default refusing instead.
- **URL:** <https://git-scm.com/docs/git-pull>

The span relied on: "`git pull --ff-only` will only do "fast-forward" updates: it fails if your local branch has diverged from the remote branch. This is the default."

1.13 S13

- **Type:** Primary source
- **Credibility:** High — Git's own documentation for the attribute that causes the behaviour.
- **Relevance:** Part I. Why the same two lines of text can hash to two values on two laptops.
- **URL:** <https://git-scm.com/docs/gitattributes>

The spans relied on: "Line endings are normalized to LF in the index every time the file is checked in"; the default for `eol` is `crlf` on Windows.

1.14 S14

- **Type:** Primary source
- **Credibility:** High — Git's own documentation for an interface that ships with the tool rather than with a forge.
- **Relevance:** Part III. The honest limit on the deck's thesis, because Git does ship enforcement mechanism.
- **URL:** <https://git-scm.com/docs/githooks>

The spans relied on: the update hook can "enforce a 'fast-forward only' policy"; "this hook can be used to implement access control without relying on filesystem ownership and group membership".

1.15 S15

- **Type:** Primary source
- **Credibility:** High — the project's own governing document for planned breaking changes.
- **Relevance:** Part I. The four named SHA-1 attacks, and the SHA-256 default that has not arrived yet.
- **URL:** <https://git-scm.com/docs/BreakingChanges>

The spans relied on: the SHAppening (2015), SHattered (2017), Birthday-Near-Collision (2019) and Shambles (2020) are named as the reason for moving the default hash to SHA-256; SHA-1 is not being deprecated as a format.

1.16 S16

- **Type:** News
- **Credibility:** Medium — established technical journalism reporting a maintainer summit, more current than the project page it contradicts.
- **Relevance:** Part I. The scheduled date for the SHA-256 default, against a project document that still says there is none.
- **URL:** <https://lwn.net/SubscriberLink/1094575/2385e98583715c2b/>

The span relied on: "The 2.99 release will follow in April 2027; that will be a long-term-support release. Git 3.0 will also be released at the same time".

1.17 S17

- **Type:** Reference
- **Credibility:** High — MIT course material, widely adopted as a teaching reference for this exact topic.
- **Relevance:** Parts I and II. The bottom-up framing, and references as the only mutable layer.
- **URL:** <https://missing.csail.mit.edu/2020/version-control/>

The spans relied on: an ugly interface has to be memorised while a beautiful design can be understood; "References are pointers to commits. Unlike objects, which are immutable, references are mutable".

1.18 S18

- **Type:** Primary source
- **Credibility:** High — first-person design rationale from Git's author in the project's own archived mailing list.
- **Relevance:** Part IV. Rename tracking was refused by design, not omitted by accident.
- **URL:** <https://public-inbox.org/git/Pine.LNX.4.58.0504141102430.7211@ppc970.osdl.org/>

The span relied on: "git is about tracking data, not about tracking what happened to create that data".

1.19 S19

- **Type:** Primary source
- **Credibility:** High — a transcript of the author speaking about his own design decisions, archived on the kernel wiki.
- **Relevance:** Part IV. Content addressing was not Git's invention, and the system that had it first lost on speed.
- **URL:** <https://archive.kernel.org/oldwiki/git.wiki.kernel.org/index.php/LinusTalk200705Transcript.html>

The span relied on: Monotone "had a lot of interesting ideas, but the performance was so horrendously bad, that, I tried it for a day and realized that I cannot use it".

1.20 S20

- **Type:** Academic paper
- **Credibility:** Medium — peer-reviewed, but its base is one project-based course at one university, answered by 21 of 26 students, plus the authors' own teaching experience.
- **Relevance:** Parts I and II. The documented misconceptions this room is likely to hold, and the evidence for teaching the model rather than the commands.
- **URL:** <https://www.cochez.nl/papers/dvcs-misconceptions-final.pdf>

The spans relied on: the branch-as-folder misconception "causes learners to regard branches as folders or files in the file system... learners might try to move into a branch with the `cd` command"; `git add` is "associated with creating a new file, while it stages the file for the commit"; "learners do not think about the Git commands, but "just try them out", which makes it very difficult to interpret the responses and error messages"; "the further one goes from basic features and use cases, the more important it is to communicate to student what really takes place in Git internally".

1.21 S21

- **Type:** Academic paper
- **Credibility:** High — peer-reviewed and published at Onward!, from MIT's Software Design Group.
- **Relevance:** Part IV. The analysis that turns "the interface is hard" into checkable claims.
- **URL:** <https://groups.csail.mit.edu/sdg/pubs/2013/git-conceptual-onward13.pdf>

The spans relied on: staged and working versions are not orthogonal; `commit` carries two unrelated purposes; the Gitless prototype removes staging.

1.22 S22

- **Type:** Reference
- **Credibility:** High — official project documentation, advocacy by construction but precise about the concepts it drops.
- **Relevance:** Part IV. The index's two fused roles, and the working copy as a commit.
- **URL:** <https://docs.jj-vcs.dev/latest/git-comparison/>

The spans relied on: "Git's 'index' has multiple roles. One role is as a cache of file system information"; "There's no index (staging area). Because the working copy is automatically committed, an index-like concept doesn't make sense".

1.23 S23

- **Type:** Academic paper
- **Credibility:** High — peer-reviewed at ICSE, with a stated sample and a published classification method.
- **Relevance:** Part VI. What review comments are actually about, and the comprehension bottleneck underneath.
- **URL:** <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/ICSE202013-codereview.pdf>

The spans relied on: "Review comments about defects are few, comprising one-eighth of the total in our sample, and mostly address 'micro' level and superficial concerns"; "code and change understanding is the key aspect of code reviewing"; finding defects is the top-ranked motivation the participants gave, while in the 570 hand-classified comments defect is the fourth most frequent of nine categories at 14% and code improvements is the most frequent at 29%.

1.24 S24

- **Type:** Academic paper
- **Credibility:** High — peer-reviewed at MSR, and cited here because it cuts against the deck's other review source.
- **Relevance:** Part VI. The outcome evidence that review coverage and participation track post-release defects.
- **URL:** https://rebels.cs.uwaterloo.ca/papers/msr2014_mcintosh.pdf

The span relied on: "Low code review coverage and participation are estimated to produce components with up to two and five additional post-release defects respectively".

1.25 S25

- **Type:** Industry report
- **Credibility:** Medium — a vendor-published case study, but the largest measured sample of lightweight review and explicit about its method.

- **Relevance:** Part VI. Both numbers, the recommendation and the ceiling, from one source.
- **URL:** <https://static1.smartbear.co/support/media/resources/cc/book/best-kept-secrets-of-peer-code-review.pdf>

The spans relied on: "the single best piece of advice we can give is to review between 100 and 300 lines of code at a time and spend 30-60 minutes to review it"; "LOC under review should be under 200, not to exceed 400. Anything larger overwhelms reviewers and defects are not uncovered"; "in no case is a defect discovered after 90 minutes"; the study ran for ten months and covered 2,500 reviews of 3.2 million lines of code, and defects were found at relatively constant rates through the first 60 minutes, after which the rate levelled off.

1.26 S26

- **Type:** Reference
- **Credibility:** Medium — the practice's own advocacy site, useful for its definitions and openly partisan about their merits.
- **Relevance:** Part V. The definition of the model, and the word trunk itself.
- **URL:** <https://trunkbaseddevelopment.com/>

The spans relied on: the model is one "where developers collaborate on code in a single branch called 'trunk' and resist any pressure to create other long-lived development branches"; trunk is "also known as 'main' or 'master'".

1.27 S27

- **Type:** Reference
- **Credibility:** Medium — the same advocacy site, carrying the two preconditions that summaries usually drop.
- **Relevance:** Part V. Branch life of at most two days, and one author per branch.
- **URL:** <https://trunkbaseddevelopment.com/short-lived-feature-branches/>

The spans relied on: "the branch should only last a couple of days"; "the developer count should stay at one (or two if pair-programming)"; "With 16 or more, the team is more productive with short-lived feature branches"; "With the Pull Request (and code review) advance, the cut-off point for team sizes that graduated from "direct to the trunk" to short-lived feature branches moved lower"; "While it was up to 100 before Git's lightweight branching, it is now up to 15 people".

1.28 S28

- **Type:** Industry report
- **Credibility:** Medium — a large survey programme, self-reported and correlational by its own account.
- **Relevance:** Parts V and VII. The definition of trunk-based development in terms of merge cadence, and the delivery association.
- **URL:** <https://dora.dev/capabilities/trunk-based-development/>

The spans relied on: "each developer divides their own work into small batches and merges that work into trunk at least once (and potentially several times) a day"; the association with three or fewer active branches and daily merges.

1.29 S29

- **Type:** Blog / editorial
- **Credibility:** High — the primary source for the model, carrying its author's own later scoping note.
- **Relevance:** Part V. The model, and the sentence that decides when it is still the right answer.
- **URL:** <https://nvie.com/posts/a-successful-git-branching-model/>

The spans relied on: "I would suggest to adopt a much simpler workflow (like GitHub flow) instead of trying to shoehorn git-flow into your team"; "If, however, you are building software that is explicitly versioned, or if you need to support multiple versions of your software in the wild, then git-flow may still be as good of a fit to your team".

1.30 S30

- **Type:** Blog / editorial
- **Credibility:** High — a widely cited practitioner reference that names its patterns precisely and states preferences as preferences.
- **Relevance:** Part V. Integration frequency as the variable, and the healthy-branch precondition.
- **URL:** <https://martinfowler.com/articles/branching-patterns.html>

The spans relied on: feature branching is "Put all work for a feature on its own branch, integrate into mainline when the feature is complete"; a healthy branch means "On each commit, perform automated checks, usually building and running tests, to ensure there are no defects on the branch".

1.31 S31

- **Type:** Blog / editorial
- **Credibility:** High — the canonical definition of the practice, revised by its author in 2023 rather than defended unchanged.
- **Relevance:** Part VII. Continuous integration as a frequency claim, and its identity with trunk-based development.
- **URL:** <https://martinfowler.com/articles/continuousIntegration.html>

The spans relied on: each member "merges their changes into a codebase together with their colleagues changes at least daily"; "Each of these integrations is verified by an automated build (including test)"; "This practice is called 'Continuous Integration' (or in some circles it's called 'Trunk-Based Development')".

1.32 S32

- **Type:** Blog / editorial
- **Credibility:** High — the definition practitioners cite, from a co-author of the discipline's canonical book.
- **Relevance:** Part VII. Continuous delivery as a property of the artefact, and continuous deployment as a policy.
- **URL:** <https://martinfowler.com/bliki/ContinuousDelivery.html>

The spans relied on: continuous delivery means "you build software in such a way that the software can be released to production at any time"; "Continuous Deployment means that every change goes through the pipeline and automatically gets put into production"; "In order to do Continuous Deployment you must be doing Continuous Delivery".

1.33 S33

- **Type:** Reference
- **Credibility:** Medium — an author's own companion site rather than a peer-reviewed study, candid about the limits of its advice.

- **Relevance:** Part VII. Decoupling deployment from release.
- **URL:** <https://continuousdelivery.com/implementing/patterns/>

The span relied on: the goal is "to separate the technical decision to deploy from the business decision to launch a feature", and the two named patterns are dark launching and feature toggles.

1.34 S34

- **Type:** Reference
- **Credibility:** Medium — the same author's site, carrying a self-reported hit rate rather than a measured one.
- **Relevance:** Part VII. The three-question test the room is asked to apply, and its stated pass rate.
- **URL:** <https://continuousdelivery.com/foundations/continuous-integration/>

The spans relied on: the three questions about daily trunk pushes, unit tests on every commit and a ten-minute fix; "In my experience, less than 20% of teams that think they are doing CI can actually pass the test"; "feature toggles are only really necessary when practicing continuous deployment".

1.35 S35

- **Type:** Industry report
- **Credibility:** Medium — a vendor's account of its own practice, detailed and self-reported.
- **Relevance:** Part V. Release Flow as the documented case that is trunk-based and release-branched at once.
- **URL:** <https://learn.microsoft.com/en-us/devops/develop/how-microsoft-develops-devops>

The spans relied on: a release branch is started from main when a team is ready to ship, and "Release branches never merge back to the main branch"; "Fixing a bug in the release branch without bringing the change back to `main` would mean the bug would recur during the next deployment".

1.36 S36

- **Type:** Reference

- **Credibility:** High — vendor documentation for the buttons the room actually presses.
- **Relevance:** Parts V and VI. What each merge method leaves on the mainline, and the documented squash hazard.
- **URL:** <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/incorporating-changes-from-a-pull-request/about-pull-request-merges>

The spans relied on: squash merging means "intermediate commits from the pull request are not preserved as separate commits on the base branch"; with squash merging "later pull requests can include commits that were already squashed into the base branch"; rebase merging "always updates the committer information and creates new commit SHAs".

1.37 S37

- **Type:** News
- **Credibility:** High — the vendor's own changelog entry, which is the primary record of the feature's status.
- **Relevance:** Part VI. Stacked pull requests are in public preview rather than general availability.
- **URL:** <https://github.blog/changelog/2026-07-30-stacked-pull-requests-are-now-in-public-preview/>

The spans relied on: the feature is "rolling out in public preview to all repositories over the coming days"; "To land part of a stack, merge one or more lower layers—the pull requests above it stay open and automatically rebase and retarget".

1.38 S38

- **Type:** Academic paper
- **Credibility:** Medium — a peer-reviewed mining study with a large sample, reporting association rules rather than causal effects.
- **Relevance:** Part V. Conflict probability against branch duration, with the effect size that qualifies the headline.
- **URL:** <https://doi.org/10.5753/jserd.2021.1911>

The spans relied on: 182,273 merge scenarios across 80 projects; 40% fewer conflicts under a day and 161% more beyond 30 days; a small effect size of minus 0.3 for branching duration.

1.39 S39

- **Type:** Academic paper
- **Credibility:** Medium — a peer-reviewed empirical study on open-source repositories, with results specific to its corpus.
- **Relevance:** Part V. Conflict resolutions are measurably defect-prone, which is the cost short-lived branches avoid.
- **URL:** https://stairs.ics.uci.edu/papers/2020/emperical_MC.pdf

The spans relied on: 36,122 merge commits across 143 projects; 19.32% produced conflicts requiring human intervention; conflicted lines about twice as likely to be buggy, and semantic ones about 26 times more likely to need a bug-fixing commit.

1.40 S40

- **Type:** Academic paper
- **Credibility:** Medium — peer-reviewed at MSR, measuring open-source projects that may not resemble a company codebase.
- **Relevance:** Part VII. Feature-flag debt measured rather than asserted.
- **URL:** <https://cmustrudel.github.io/papers/msr20flags.pdf>

The span relied on: across 100 projects holding 7,593 flags, "half of the flags are removed within 15 months, but also that 25 % of flags remain for a very long time or forever (more than 8 years)".

1.41 S41

- **Type:** Primary source
- **Credibility:** High — the project's own frequently-asked-questions page, recording a defect against itself.
- **Relevance:** Part IV. The exponential merge is a known defect of long standing, stated by the project that carries it.
- **URL:** <https://darcs.net/FAQ/Performance>

The span relied on: the answer to "Is the exponential merge problem fixed yet?" is "Answer from 2012-08-02 — No".

1.42 S42

- **Type:** Primary source
- **Credibility:** High — an archived mailing-list post, dated and first-person.
- **Relevance:** Part IV. Mercurial began from the same trigger, two weeks later, with different stated goals.
- **URL:** <https://lkml.iu.edu/hypermail/linux/kernel/0504.2/0670.html>

The spans relied on: the announcement is dated "Wed Apr 20 2005" and titled "Mercurial v0.1 - a minimal scalable distributed SCM"; the stated aims are "to initially be as simple (and thereby hackable) as possible" and "to be as scalable as possible".

1.43 S43

- **Type:** Blog / editorial
- **Credibility:** Medium — a science-education researcher's summary of an established teaching sequence rather than a primary study.
- **Relevance:** Every demonstration in the deck. Why the prediction is written down before the command runs.
- **URL:** <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

The spans relied on: the method suits "topics areas where students are likely to have strong intuitions which do not match scientific accounts of the world"; predicting first gives "some sense of ownership in the following activity – a stake in the outcome as it were".

1.44 S44

- **Type:** Reference
- **Credibility:** High — vendor documentation for the mechanism that turns a convention into a constraint.
- **Relevance:** Part III. What branch protection actually restricts, which is where centralisation now lives.
- **URL:** <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>

The span relied on: branch protection rules "define whether collaborators can delete or force push to the branch and set requirements for any pushes to the branch, such as passing status checks or a linear commit history".

1.45 S45

- **Type:** Other
- **Credibility:** Medium — a long-established open-source teaching tool rather than a reference work, and widely used in university courses.
- **Relevance:** Part VIII. The reading recommendation that makes branch and merge movement visible, named so that a student can find it.
- **URL:** <https://github.com/pcottle/learnGitBranching>

The span relied on: the project is a browser sandbox with paced exercises that draws the commit graph as the learner manipulates it.

1.46 S46

- **Type:** Reference
- **Credibility:** High — the manual page for the machinery that performs rename detection, maintained with the code.
- **Relevance:** Part IV. Rename detection runs while two commits are compared, and it is a similarity threshold rather than a recorded fact.
- **URL:** <https://git-scm.com/docs/gitdiffcore>

The spans relied on: "and the contents of the deleted file fileX is similar enough to the contents of the created file file0, then rename detection merges these filepairs and creates"; "In both rename and copy detection, the same 'extent of changes' algorithm used in diffcore-break is used to determine if two files are 'similar enough', and can be customized to use a similarity score different from the default of 50%".

1.47 S47

- **Type:** Primary source
- **Credibility:** High — the project's own release notes for the version that introduced the flag.
- **Relevance:** Part VI. The restack operation exists in Git itself, and this is the version and the exact spelling.

- **URL:** <https://github.com/git/git/blob/v2.38.0/Documentation/RelNotes/2.38.0.txt>

The span relied on: the release notes state that "git rebase -i" learns to update branches whose tip appear in the rebased range with "--update-refs" option.

2 Vault sources

2.1 V1 — Git's Object Model and Design Trade-offs

This note's central claim is that Git is not, at its core, a version-control system: it is a content-addressable filesystem with a version-control interface written on top of it, and that distinction explains why the command-line interface feels inconsistent while the data underneath is small and regular. The evidence is the object model itself — four object types (blob, tree, commit, tag), each named by the SHA-1 hash of its own content, so identical content is deduplicated automatically. Because a commit's hash covers its tree and parent hashes, and a tree's hash covers its children, a single commit hash transitively authenticates an entire project state and history — a Merkle DAG. Refs (branches, HEAD) are the one mutable layer: small files holding a hash or a symbolic pointer to another ref. The note walks this bottom-up using Git's own "plumbing" commands (`hash-object`, `write-tree`, `commit-tree`, `update-ref`) to show that ordinary commands are just "add objects" or "move refs," and traces the origin story — Linus Torvalds writing Git in days in April 2005 after losing free access to BitKeeper — as the source of its speed- and integrity-first, ergonomics-second trade-offs.

The note is equally explicit about what it does not claim. It does not treat Git's interface as well-designed: it cites a peer-reviewed conceptual-design analysis identifying specific violations, such as staging and working versions not being independent of one another. It presents the absence of rename tracking as a deliberate modeling choice — quoting Torvalds directly — rather than a defect, while still naming its practical cost. On the pending SHA-1-to-SHA-256 transition, the note treats the question of timing as explicitly unsettled: Git's own governing document states there is no planned release date for Git 3.0, and the note flags that some secondary sources claiming otherwise conflict with that primary source, without resolving which is right. It similarly frames large-scale mitigations (partial clone, sparse checkout, LFS) as bolt-ons with their own costs, not as evidence the core model scales natively, and treats a contrastive tool (Jujutsu) only as a way to distinguish essential from incidental design choices, not as an endorsement.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Git — BreakingChanges documentation](#) — The Git project (fetched 2026-09-23) **HIGH**
- [Git — Wikipedia](#) — Wikipedia contributors (fetched 2026-09-23) **MEDIUM**
- [Re: Handling renames](#) — Linus Torvalds, git mailing list, 14 April 2005 — Linus Torvalds / public-inbox.org git archive (2005) **HIGH**
- [Version Control \(Git\) — MIT Missing Semester](#) — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL **HIGH**
- [Git from the Inside Out](#) — Mary Rose Cook **HIGH**
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 (ACM) **HIGH**
- [Jujutsu — Git comparison](#) — The Jujutsu project **HIGH**
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project **HIGH**
- [Tracking renamed files in Git](#) — Ole Begemann (2025) **MEDIUM**
- [Git — partial-clone documentation](#) — The Git project **HIGH**
- [git-lfs/docs/spec.md](#) — The Git LFS project **HIGH**
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog **HIGH**
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) **MEDIUM**
- [Content-Addressing File System Objects](#) — Nix Reference Manual — The Nix project **HIGH**
- [pcottle/learnGitBranching](#) — Peter Cottle **HIGH**
- [Learning resources: Version control with Git](#) — Princeton Research Computing **MEDIUM**
- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**

- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance](#) — GitLab Docs — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**
- [Migrate to Git 3.0, SHA-256, and reftables](#) — cited in the note's Grounding gap as an unread secondary source to backfill, not yet graded
- [LWN.net Articles/1094575](#) — cited in the note's Grounding gap as an unread secondary source to backfill, not yet graded

2.2 V2 — Git's origin, object model, and design trade-offs

This note argues that Git is not, at its core, a version-control system but a content-addressable filesystem with a version-control interface added on top, and that this explains why the command surface feels inconsistent while the data model beneath it is small and regular. It lays out that model: four immutable object types — blob, tree, commit, tag — each named by the hash of its own content and linked into a Merkle DAG, so one commit hash transitively authenticates the whole project snapshot and every commit behind it, plus a mutable layer of refs — plain files holding a hash — through which branches, HEAD, and detached HEAD are expressed. Every Git command, it states, either adds objects or moves refs, never edits in place: `commit --amend`, `rebase`, and `cherry-pick` build replacement objects and repoint refs, leaving old objects to eventual garbage collection. This rests on Git's own internals documentation (Pro Git), MIT's Missing Semester, and Mary Rose Cook's essay, and is demonstrated with a five-command plumbing walkthrough (`hash-object`, `cat-file`, `write-tree`, `commit-tree`, `update-ref`) reconstructing `add`, `commit`, and `branch` from first principles. It traces the design to its origin — Torvalds writing Git in days in April 2005 after kernel development lost BitKeeper — and sets Git's snapshot-based, local, self-deduplicating design against CVS/Subversion's deltas, and against Mercurial and Jujutsu as other resolutions on the same distributed, hash-addressed foundation.

The note is explicit about what it does not claim. It does not defend Git's command-line interface as well designed: it cites a peer-reviewed conceptual-design analysis of specific defects — non-orthogonal staging and working versions, a `commit` that conflates two unrelated purposes — as documented research, not its own judgment. It does not claim Git tracks renames as a fact in its data model, quoting Torvalds's 2005 statement that Git tracks data, not the history of how it was produced. On the pending SHA-1-to-SHA-256 transition, it treats the Git 3.0 release date as unsettled: the project's own breaking-changes document commits to no date, and secondary sources claiming SHA-256 is already default, or a late-2026 Git 3.0, are flagged as conflicting with that primary source. It also discloses a process limitation: most planned search engines hit usage

limits during compilation, so claims rest on the engines that worked and on primary sources fetched directly, leaving two secondary articles on the timeline unread and un-attributed.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Git — BreakingChanges documentation](#) — The Git project HIGH
- [Git — Wikipedia](#) — Wikipedia contributors MEDIUM
- [Re: Handling renames](#) — Linus Torvalds, git mailing list, 14 April 2005 — Linus Torvalds / public-inbox.org git archive HIGH
- [Version Control \(Git\) — MIT Missing Semester](#) — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL HIGH
- [Git from the Inside Out](#) — Mary Rose Cook HIGH
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 (ACM) HIGH
- [Jujutsu — Git comparison](#) — The Jujutsu project HIGH
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project HIGH
- [Tracking renamed files in Git](#) — Ole Begemann (2025) MEDIUM
- [Git — partial-clone documentation](#) — The Git project HIGH
- [git-lfs/docs/spec.md](#) — The Git LFS project HIGH
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog HIGH
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) MEDIUM
- [Content-Addressing File System Objects](#) — Nix Reference Manual — The Nix project HIGH
- [pcottle/learnGitBranching](#) — Peter Cottle HIGH
- [Learning resources: Version control with Git](#) — Princeton Research Computing MEDIUM

- [git concepts simplified](#) — Sitaram Chamarty MEDIUM
- [How I teach Git](#) — Thomas Broyer (2023) MEDIUM
- [Git Is Not Fine](#) — Bill Jings LOW
- [Improving monorepo performance](#) — GitLab Docs — GitLab HIGH
- [Think Like \(a\) Git](#) — Sam Livingston-Gray MEDIUM
- <https://www.sitepoint.com/migrate-to-git-3-0-sha-256-and-reftables/> (no title given in the note)
- <https://lwn.net/Articles/1094575/> (no title given in the note)

2.3 V3 — Git's Origin, Object Model, and Design Trade-offs

This note establishes that Git is not, at its core, a version-control system: it is a content-addressable filesystem with a version-control interface layered on top, and that distinction explains most of what confuses newcomers. The evidence is the object model itself — four immutable object types (blob, tree, commit, tag) named by the hash of their own content and linked into a Merkle DAG, with a small set of mutable references (branches, HEAD) as the only writable layer. Every Git command either adds new objects or moves a reference; nothing is edited in place, which is why amend, rebase, and cherry-pick build replacement objects rather than changing history. The note traces this design to Git's origin — written by Linus Torvalds in days in April 2005, under deadline pressure after the Linux kernel lost its free BitKeeper license — and argues the resulting priorities (speed, fully local operation, cryptographic integrity) are still legible in trade-offs everyone now inherits. It also explains why a whole-tree snapshot per commit is affordable: unchanged blobs and trees are reused by hash identity rather than recopied, so storage cost tracks changed content, with delta compression applied only later, at the packfile layer, purely for compression rather than correctness.

The note is equally explicit about what it does not claim. It treats scale features — partial clone, sparse checkout, the commit-graph, Git LFS — as later additions bolted onto a core model with no native notion of "part of a repository," and documents where each one leaks into surrounding tooling rather than presenting them as solved. It also flags an unresolved question rather than picking a side: Git's own governing document states there is no planned release date for the SHA-256/reftable transition in Git 3.0, while secondary web sources the note found describe SHA-256 as already the default in earlier releases and predict a late-2026 date for 3.0 — the note treats this as an open contradiction, prefers the primary source, and uses the disagreement as a worked example of source grading rather than resolving it. A further open item, recorded in the note's own grounding-gap tracker, is that the secondary articles making those claims have not

yet been read and attributed to named authors. Finally, the note frames itself as reference grounding for a future teaching session rather than the finished lesson, and defers course-design choices — pacing, exercise design, register — to that later document.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Git — BreakingChanges documentation](#) — The Git project (fetched 2026-09-23) HIGH
- [Git — Wikipedia](#) — Wikipedia contributors (fetched 2026-09-23) MEDIUM
- [Re: Handling renames](#) — Linus Torvalds, git mailing list, 14 April 2005 — Linus Torvalds / public-inbox.org git archive (2005) HIGH
- [Version Control \(Git\) — MIT Missing Semester](#) — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL HIGH
- [Git from the Inside Out](#) — Mary Rose Cook HIGH
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 (ACM) HIGH
- [Jujutsu — Git comparison](#) — The Jujutsu project HIGH
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project HIGH
- [Tracking renamed files in Git](#) — Ole Begemann (2025) MEDIUM
- [Git — partial-clone documentation](#) — The Git project HIGH
- [git-lfs/docs/spec.md](#) — The Git LFS project HIGH
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog HIGH
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) MEDIUM
- [Content-Addressing File System Objects](#) — Nix Reference Manual — The Nix project HIGH
- [pcottle/learnGitBranching](#) — Peter Cottle HIGH

- [Learning resources: Version control with Git](#) — Princeton Research Computing

MEDIUM

- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**
- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance](#) — GitLab Docs — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**
- <https://www.sitepoint.com/migrate-to-git-3-0-sha-256-and-reftables/> — cited in the note's Grounding gap section as an unread secondary source, no title or author given
- <https://lwn.net/Articles/1094575/> — cited in the note's Grounding gap section as an unread secondary source, no title or author given

2.4 V4 — Git's origin, object model, and design trade-offs

This note argues that Git is, at its core, a content-addressable filesystem with a version-control interface layered on top, and that this data model — not the command-line surface — is what actually explains the tool. It lays out the model in four parts: content addressing (identical content always yields the same hash, so deduplication is automatic), four object types (blob, tree, commit, tag), a Merkle DAG in which a commit's hash transitively authenticates the entire project state and history beneath it, and refs as the sole mutable layer (a branch is one file holding one hash; `HEAD` is usually a symbolic pointer to a branch). From this the note draws its central claim: every Git command either adds objects or moves refs, and operations like `commit --amend` and `rebase` never edit history in place, they build new objects and repoint refs, leaving the old ones for eventual garbage collection. It backs this with worked plumbing sequences (`hash-object`, `write-tree`, `commit-tree`, `update-ref`) reconstructing what `add / commit / branch` do, and grounds the origin story in a dated 2005 timeline — Git was written by Linus Torvalds in days, after Linux kernel development lost free access to BitKeeper, with explicit design goals of speed, full local operation, and non-linear development at scale.

The note is explicit about what it does not claim. It is stated up front to be reference grounding for a future course session, not the teaching deck itself — pedagogical sequencing, pacing, and register are left to that deck. On the still-open question of when Git 3.0 (with SHA-256 and reftable as defaults) will ship, the note treats this as genuinely unsettled: the project's own breaking-changes document states no release date exists, while secondary web sources it found claim SHA-256 is already default and place Git 3.0 in late 2026 — a direct conflict the note flags rather than resolves, naming it as a case where the primary source should be preferred and treating the secondary claims as

needing per-article attribution it has not yet done. It also documents that four of five planned search engines were unavailable during its research due to usage-limit errors, so it does not claim engine-level triangulation on its recency claims, only that its load-bearing claims are each anchored to a primary or official source it did fetch directly.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Git — BreakingChanges documentation](#) — The Git project (fetched 2026-09-23) HIGH
- [Git — Wikipedia](#) — Wikipedia contributors (fetched 2026-09-23) MEDIUM
- [Re: Handling renames](#) — Linus Torvalds, git mailing list, 14 April 2005 — Linus Torvalds / public-inbox.org git archive (2005) HIGH
- [Version Control \(Git\)](#) — MIT Missing Semester — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL HIGH
- [Git from the Inside Out](#) — Mary Rose Cook HIGH
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 (ACM) HIGH
- [Jujutsu — Git comparison](#) — The Jujutsu project HIGH
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project HIGH
- [Tracking renamed files in Git](#) — Ole Begemann (2025) MEDIUM
- [Git — partial-clone documentation](#) — The Git project HIGH
- [git-lfs/docs/spec.md](#) — The Git LFS project HIGH
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog HIGH
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) MEDIUM
- [Content-Addressing File System Objects](#) — Nix Reference Manual — The Nix project HIGH
- [pcottle/learnGitBranching](#) — Peter Cottle HIGH

- [Learning resources: Version control with Git](#) — Princeton Research Computing

MEDIUM

- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**
- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance](#) — GitLab Docs — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**
- <https://www.sitepoint.com/migrate-to-git-3-0-sha-256-and-reftables/> — cited in the note's Grounding gap as a secondary source still to be read and attributed per-claim
- <https://lwn.net/Articles/1094575/> — cited in the note's Grounding gap as a secondary source still to be read and attributed per-claim

2.5 V5 — Git's Content-Addressable Object Model and Design Trade-offs

This note argues that Git is not, at its core, a version-control system but a content-addressable filesystem with a version-control interface layered on top — a characterization it draws from Git's own official documentation — and that this data model, not the command-line interface built over it, is what should be taught and understood first. It lays out the model in full: four immutable object types (blob, tree, commit, tag), each named by the hash of its own content, linked into a Merkle DAG, plus one mutable layer of references (branches, HEAD) that point into that graph. It grounds this in Git's origin — written by Linus Torvalds in a matter of days in April 2005, after Linux kernel development lost free access to BitKeeper, with a well-documented timeline running from first commit to self-hosting the next day — and argues that the tool's original priorities (speed, fully local operation, cryptographic integrity, and cheap non-linear branching) still explain its present-day behavior, now that roughly 95% of developers report using it as their primary version control system. The note demonstrates the model concretely by reconstructing Git's everyday commands from five low-level "plumbing" primitives, and explains why commits behave as full snapshots yet cost little: unchanged content is reused by hash, and delta compression is applied only later, as a storage-layer optimization inside packfiles.

The note is explicit about what it does not resolve. It treats the CLI's mismatch with the data model, weak rename tracking, poor handling of large binaries, and monorepo scale as places where the core model has no native answer and instead relies on bolt-on mitigations such as LFS and partial clone, each with its own admitted costs. On the pending SHA-1-to-SHA-256 hash transition, it treats the release timeline as genuinely unsettled: it reports that Git's own governing document states no planned release date

for Git 3.0, notes that other web sources it consulted claim SHA-256 is already the default and place Git 3.0 in late 2026, and defers to the primary source without claiming to have adjudicated the disagreement, leaving it as an item to be checked by reading the conflicting secondary articles directly. It also discloses, rather than hides, a methodological limitation: several of the search engines it planned to use for research returned usage-limit errors, so it acknowledges narrower engine triangulation on the questions where sources disagree.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Git — BreakingChanges documentation](#) — The Git project HIGH
- [Git — Wikipedia](#) — Wikipedia contributors MEDIUM
- [Re: Handling renames](#) — Linus Torvalds, git mailing list, 14 April 2005 — Linus Torvalds / public-inbox.org HIGH
- [Version Control \(Git\) — MIT Missing Semester](#) — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL HIGH
- [Git from the Inside Out](#) — Mary Rose Cook HIGH
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 HIGH
- [Jujutsu — Git comparison](#) — The Jujutsu project HIGH
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project HIGH
- [Tracking renamed files in Git](#) — Ole Begemann (2025) MEDIUM
- [Git — partial-clone documentation](#) — The Git project HIGH
- [git-lfs/docs/spec.md](#) — The Git LFS project HIGH
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog HIGH
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) MEDIUM

- [Content-Addressing File System Objects — Nix Reference Manual](#) — The Nix project **HIGH**
- [pcottle/learnGitBranching](#) — Peter Cottle **HIGH**
- [Learning resources: Version control with Git](#) — Princeton Research Computing **MEDIUM**
- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**
- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance](#) — GitLab Docs — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**
- [Migrate to Git 3.0: SHA-256 and reftables](#) — SitePoint
- [LWN.net Article 1094575](#) — LWN.net

2.6 V6 — Git's Origin, Object Model, and Design Trade-offs

This note argues that Git is not fundamentally a version-control system but a content-addressable filesystem with a version-control interface layered on top, and that this reframing is the single most useful lens for understanding the tool. It grounds that claim in Git's object model: four immutable object types (blob, tree, commit, tag) addressed by content hash and linked into a Merkle DAG, plus one mutable layer of references (branches, HEAD, tags) that are simply files containing hashes. Every Git command, the note claims, either adds objects to this graph or moves a reference within it — nothing is ever edited in place. This account rests on the official Pro Git documentation for the object types, the plumbing-command sequence, and the packfile mechanism; on MIT's Missing Semester and Mary Rose Cook's essay for the bottom-up pedagogical argument; and on Wikipedia's densely footnoted account of Git's origin — written by Linus Torvalds in April 2005 within days, under the deadline created by the loss of BitKeeper access for Linux kernel development, and optimized from the outset for speed, local operation, and non-linear branching rather than for user-facing ergonomics.

The note is explicit about what it does not settle. It does not claim the content-addressed Merkle DAG design is unique to Git — it deliberately places IPFS, Nix, and Bitcoin in the same design family to show the pattern is general, not to claim priority for any of them. It does not adjudicate the correctness of Git's user-facing CLI, instead citing a peer-reviewed conceptual-design critique (Gitless) and Jujutsu's contrastive redesign as documented positions on where the interface strains against the model, without endorsing a resolution. And on the timing of Git 3.0's move to SHA-256 as the default hash, the note treats the question as genuinely unsettled: the project's own

breaking-changes document states no release date has been set, while secondary write-ups it also consulted claim SHA-256 is already default and place Git 3.0 in late 2026 — a direct conflict the note flags rather than resolves, preferring the primary source but not treating the matter as closed.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Git — BreakingChanges documentation](#) — The Git project HIGH
- [Git — Wikipedia](#) — Wikipedia contributors MEDIUM
- [Re: Handling renames](#) — Linus Torvalds, git mailing list, 14 April 2005 — Linus Torvalds / public-inbox.org git archive HIGH
- [Version Control \(Git\) — MIT Missing Semester](#) — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL HIGH
- [Git from the Inside Out](#) — Mary Rose Cook HIGH
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 (ACM) HIGH
- [Jujutsu — Git comparison](#) — The Jujutsu project HIGH
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project HIGH
- [Tracking renamed files in Git](#) — Ole Begemann (2025) MEDIUM
- [Git — partial-clone documentation](#) — The Git project HIGH
- [git-lfs/docs/spec.md](#) — The Git LFS project HIGH
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog HIGH
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) MEDIUM
- [Content-Addressing File System Objects](#) — Nix Reference Manual — The Nix project HIGH
- [pcottle/learnGitBranching](#) — Peter Cottle HIGH

- [Learning resources: Version control with Git](#) — Princeton Research Computing

MEDIUM

- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**
- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance](#) — GitLab Docs — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**
- <https://www.sitepoint.com/migrate-to-git-3-0-sha-256-and-reftables/> — cited in the note's Grounding gap as an unread secondary write-up to backfill, not graded
- <https://lwn.net/Articles/1094575/> — cited in the note's Grounding gap as an unread secondary write-up to backfill, not graded

2.7 V7 — Git's Origin, Object Model, and Design Trade-offs

This note's central claim is that Git is not fundamentally a version-control system but a content-addressable filesystem, with the familiar commands layered on top as a user interface added later and somewhat inconsistently. It grounds that claim in the object model itself: four immutable object types — blob, tree, commit, and tag — each named by the hash of its own content, plus a single mutable layer of references (branch files and HEAD) pointing into the resulting graph. Because each object's hash covers the hashes of everything beneath it, one commit hash transitively authenticates the entire tree and history behind it — a Merkle DAG, not a strict Merkle tree. It supports this with a worked plumbing sequence (`hash-object`, `write-tree`, `commit-tree`, `update-ref`) that reconstructs `git add`, `commit`, and `branch` from lower-level operations, and traces the design to Linus Torvalds writing Git in days during April 2005, under deadline pressure after Linux kernel development lost free access to BitKeeper, with speed, local operation, and non-linear branching as explicit goals ahead of ergonomics. It also catalogues later scale mitigations (partial clone, sparse checkout, commit-graph, Git LFS) layered on a core with no native notion of a partial repository, and situates the same content-addressed Merkle DAG structure in IPFS, Nix, and Bitcoin as a reusable pattern rather than a Git-specific idiosyncrasy.

The note draws a firm line around what remains unsettled. It documents, without resolving, the mismatch between Git's CLI and its own data model, citing a peer-reviewed study that names specific violations — staging and working versions are not orthogonal, and `commit` serves two unrelated purposes — while treating Gitless's redesign as one documented response, not a verdict. It states Git tracks no rename fact at all, relying on a similarity heuristic at diff time, and quotes Torvalds' own 2005 explanation without arguing for or against it. On the SHA-1-to-SHA-256 transition, it distinguishes a

verified claim — the project's own document states no release date exists for Git 3.0 — from conflicting secondary reporting that the change already shipped, marking that conflict as open rather than adjudicated, and leaves attribution of the specific misstatement to a named author as unfinished. It also discloses that most planned search engines hit usage-limit errors, so recency claims rest on fewer triangulating sources than intended. It presents itself as reference grounding for a future deck, leaving pacing, exercises, and simplification to that downstream artifact.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Git — BreakingChanges documentation](#) — The Git project HIGH
- [Git — Wikipedia](#) — Wikipedia contributors MEDIUM
- [Re: Handling renames](#) — Linus Torvalds, git mailing list, 14 April 2005 — Linus Torvalds / public-inbox.org git archive HIGH
- [Version Control \(Git\) — MIT Missing Semester](#) — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL HIGH
- [Git from the Inside Out](#) — Mary Rose Cook HIGH
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 (ACM) HIGH
- [Jujutsu — Git comparison](#) — The Jujutsu project HIGH
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project HIGH
- [Tracking renamed files in Git](#) — Ole Begemann (2025) MEDIUM
- [Git — partial-clone documentation](#) — The Git project HIGH
- [git-lfs/docs/spec.md](#) — The Git LFS project HIGH
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog HIGH
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) MEDIUM

- [Content-Addressing File System Objects — Nix Reference Manual](#) — The Nix project **HIGH**
- [pcottle/learnGitBranching](#) — Peter Cottle **HIGH**
- [Learning resources: Version control with Git](#) — Princeton Research Computing **MEDIUM**
- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**
- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance — GitLab Docs](#) — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**
- [Migrate to Git 3.0, SHA-256, and reftables](#) — SitePoint (named in the note's Grounding gap as an uncited secondary source to backfill, not yet graded)
- [LWN.net Articles/1094575](#) — LWN.net (named in the note's Grounding gap as an uncited secondary source to backfill, not yet graded)

2.8 V8 — Git's origin, object model, and design trade-offs

The source establishes that Git is, at its core, a content-addressable filesystem with a version-control interface layered on top, rather than a version control system in the traditional sense. It grounds this in the object model: four immutable object types (blob, tree, commit, tag), each named by the hash of its own content, linked into a Merkle DAG so that a single commit hash transitively authenticates an entire project history, plus a mutable layer of refs — plain files holding a hash or a symbolic pointer to another ref — that is the only thing any Git command actually moves. It traces this design to a specific origin: Linus Torvalds wrote Git in days in April 2005 after Linux kernel development lost free access to BitKeeper, optimizing deliberately for speed, local operation, and non-linear branching rather than for interface ergonomics, and documents that origin with a dated 2005 timeline and a first-person quotation from Torvalds' own mailing-list post on why Git deliberately does not track renames. It backs the model with the official Git book, MIT's Missing Semester course, an annotated long-form essay reconstructing the model command-by-command, Wikipedia's corroborated timeline, and the project's own breaking-changes documentation, and it extends the pattern outward by comparing Git to Subversion, Mercurial, and Jujutsu, and to structurally similar systems (IPFS, Nix, Bitcoin) built on the same content-addressed, tamper-evident idea.

The note is explicit about what it does not claim. It does not assert that Git's command-line interface matches its underlying data model — it cites a peer-reviewed conceptual-design analysis arguing the opposite, that staging and working-tree operations are not orthogonal and that `commit` conflates two unrelated purposes. It does not treat scale features such as partial clone, sparse checkout, and LFS as part of the core model, describing them instead as later, leaky additions. On the timing of Git's SHA-256 transition it treats the question as genuinely unsettled: the project's own documentation states there is no planned release date for Git 3.0, while secondary sources found in later research claim SHA-256 is already the default and give a release target, a direct contradiction the note flags rather than resolves, instructing a reader to prefer the primary source. It also leaves open, as an acknowledged gap, which named authors are responsible for those conflicting secondary claims, and records that four of five planned search engines were unavailable during research for quota reasons, leaving engine triangulation on recency questions incomplete.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Git — BreakingChanges documentation](#) — The Git project (fetched 2026-09-23) HIGH
- [Git — Wikipedia](#) — Wikipedia contributors (fetched 2026-09-23) MEDIUM
- [Re: Handling renames](#) — Linus Torvalds, git mailing list, 14 April 2005 — Linus Torvalds / public-inbox.org git archive (2005) HIGH
- [Version Control \(Git\)](#) — MIT Missing Semester — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL HIGH
- [Git from the Inside Out](#) — Mary Rose Cook HIGH
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 (ACM) HIGH
- [Jujutsu — Git comparison](#) — The Jujutsu project HIGH
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project HIGH
- [Tracking renamed files in Git](#) — Ole Begemann (2025) MEDIUM

- [Git — partial-clone documentation](#) — The Git project **HIGH**
- [git-lfs/docs/spec.md](#) — The Git LFS project **HIGH**
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog **HIGH**
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) **MEDIUM**
- [Content-Addressing File System Objects](#) — Nix Reference Manual — The Nix project **HIGH**
- [pcottle/learnGitBranching](#) — Peter Cottle **HIGH**
- [Learning resources: Version control with Git](#) — Princeton Research Computing **MEDIUM**
- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**
- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance](#) — GitLab Docs — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**
- [Migrate to Git 3.0, SHA-256, and reftables](#) — cited in the note's Grounding gap as a secondary source contradicting the primary BreakingChanges documentation
- [LWN.net Articles/1094575](#) — cited in the note's Grounding gap as a secondary source contradicting the primary BreakingChanges documentation

2.9 V9 — Git's Origin, Object Model, and Design Trade-offs

"Git — Origin, Object Model, and Design Trade-offs" establishes that Git is fundamentally a content-addressable filesystem with a version-control interface layered on top, not a version-control system in the ordinary sense — a claim it draws directly from Git's own internals documentation. It lays out the full object model: four immutable object types (blob, tree, commit, tag) named by the hash of their own content, chained into a Merkle DAG so that a single commit hash transitively authenticates a project's entire tree and history, plus one mutable layer — refs, which are simply small files holding a hash. Every Git command reduces to one of two operations, adding objects or moving references, which is why `commit --amend`, `rebase`, and `cherry-pick` build new objects and reassign refs rather than editing history in place. The evidence is drawn heavily from Git's own Pro Git manual, the MIT Missing Semester course, and Mary Rose Cook's "Git from the Inside Out" essay, supplemented by a first-person 2005 mailing-list post from Linus Torvalds and Wikipedia's account of Git's origin during the loss of BitKeeper access. The note also traces where this core model runs out — large binaries, huge

monorepos, and the deliberate absence of rename tracking — and treats scale features such as partial clone, sparse checkout, commit-graph, and LFS as additions bolted onto, rather than native to, the four-object core.

The note is explicit about what it does not settle. On Git's planned move from SHA-1 to SHA-256 hashing, it notes that the project's own breaking-changes document states there is no planned release date for Git 3.0, yet secondary write-ups found during research claim SHA-256 already became the default in earlier releases and forecast a late-2026 ship date — a direct contradiction the note preserves as unresolved rather than adjudicating, and uses as a worked example of preferring primary sources. It likewise declines to take a side on Git's design merits: it reports Jujutsu's and academic critiques of the CLI (the staging area, the absence of forward references between commits, mismatches between commands and the underlying model) as documented positions rather than endorsing or rebutting them, and reports rather than resolves an active disagreement between the project's own documentation and secondary reporting on the SHA-256 timeline. It also discloses its own research-coverage gap directly: four of five planned search engines returned quota errors during research, so the recency claims rest on single-engine coverage rather than genuine cross-engine triangulation, and it names two secondary articles still unread and unattributed as open follow-up work rather than treating the question as closed.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#), Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — A Short History of Git](#), Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — What is Git?](#), Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — Git Internals: Git References](#), Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — Git Internals: Packfiles](#), Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — Git Internals: Plumbing and Porcelain](#), Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Git — BreakingChanges documentation](#), The Git project **HIGH**
- [Git — Wikipedia](#), Wikipedia contributors **MEDIUM**
- [Re: Handling renames — Linus Torvalds](#), git mailing list, 14 April 2005, Linus Torvalds / public-inbox.org git archive **HIGH**
- [Version Control \(Git\) — MIT Missing Semester](#), Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL **HIGH**
- [Git from the Inside Out](#), Mary Rose Cook **HIGH**
- [What's Wrong with Git? A Conceptual Design Analysis](#), Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 (ACM) **HIGH**

- [Jujutsu — Git comparison](#), The Jujutsu project **HIGH**
- [jj-vcs/jj — Jujutsu version control system](#), The Jujutsu project **HIGH**
- [Tracking renamed files in Git](#), Ole Begemann **MEDIUM**
- [Git — partial-clone documentation](#), The Git project **HIGH**
- [git-lfs/docs/spec.md](#), The Git LFS project **HIGH**
- [Get up to speed with partial clone and shallow clone](#), Derrick Stolee / The GitHub Blog **HIGH**
- [IPFS — Content Addressed, Versioned, P2P File System](#), Juan Benet / Protocol Labs **MEDIUM**
- [Content-Addressing File System Objects — Nix Reference Manual](#), The Nix project **HIGH**
- [pcottle/learnGitBranching](#), Peter Cottle **HIGH**
- [Learning resources: Version control with Git](#), Princeton Research Computing **MEDIUM**
- [git concepts simplified](#), Sitaram Chamarty **MEDIUM**
- [How I teach Git](#), Thomas Broyer **MEDIUM**
- [Git Is Not Fine](#), Bill Jings **LOW**
- [Improving monorepo performance — GitLab Docs](#), GitLab **HIGH**
- [Think Like \(a\) Git](#), Sam Livingston-Gray **MEDIUM**
- [Migrate to Git 3.0, SHA-256 and reftables](#) — named in the note's grounding gap as an unread secondary source to backfill
- [LWN.net Articles/1094575](#) — named in the note's grounding gap as an unread secondary source to backfill

2.10 V10 — Git's origin, object model, and design trade-offs

This note argues that Git is best understood not as a version-control tool but as a content-addressable filesystem with a version-control interface layered on top of it — a leaky abstraction whose commands only make sense once the underlying data model is understood. It lays out that model precisely: four immutable object types (blob, tree, commit, tag), each addressed by the hash of its own content, linked into a Merkle DAG so that a single commit hash transitively authenticates an entire project's tree and history, with mutable references — branches and HEAD — forming the only layer of the system that is ever changed in place. It walks through Git's low-level "plumbing" commands to demonstrate that everyday "porcelain" operations (add, commit, branch) reduce to two primitives: adding objects and moving refs, never editing in place. It traces the tool's origin to Linus Torvalds' April 2005 rewrite, produced in days after BitKeeper's

free licence for Linux kernel development was revoked, and it enumerates concrete strengths (structural integrity, fully local operations, free branching, automatic deduplication) and limitations (the CLI's documented mismatch with the data model, weak rename tracking, poor handling of large binaries, a backlog of bolted-on scale features, and SHA-1's cryptographic weakening), each tied to a primary or official source, alongside comparisons to Subversion, Mercurial, and Jujutsu, and cross-domain analogues in IPFS, Nix, and Bitcoin.

The note is careful to separate verified claims from merely reported ones. It treats Git's own `BreakingChanges` documentation as authoritative on the still-undated SHA-256/reftable transition, and it explicitly flags that some secondary web sources claiming SHA-256 is already the default, or that Git 3.0 targets a late-2026 release, contradict that primary source and should be traced to a named author rather than accepted from aggregated search results — a disagreement it treats as open, not resolved. It does not claim that current Git tooling has fixed the conceptual defects it cites from academic research (the CLI's non-orthogonal staging and working versions, for instance): those remain characteristics of stock Git, not problems the note asserts have been solved. It states plainly that it is reference grounding for a future teaching deck rather than the deck's content itself, and it does not evaluate whether Git's design choices were the right ones — only what they are, on what evidence, and what trade-offs followed from them.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Git — BreakingChanges documentation](#) — The Git project HIGH
- [Git — Wikipedia](#) — Wikipedia contributors MEDIUM
- [Re: Handling renames](#) — Linus Torvalds, git mailing list, 14 April 2005 — Linus Torvalds / public-inbox.org HIGH
- [Version Control \(Git\) — MIT Missing Semester](#) — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL HIGH
- [Git from the Inside Out](#) — Mary Rose Cook HIGH

- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 **HIGH**
- [Jujutsu — Git comparison](#) — The Jujutsu project **HIGH**
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project **HIGH**
- [Tracking renamed files in Git](#) — Ole Begemann (2025) **MEDIUM**
- [Git — partial-clone documentation](#) — The Git project **HIGH**
- [git-lfs/docs/spec.md](#) — The Git LFS project **HIGH**
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog **HIGH**
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) **MEDIUM**
- [Content-Addressing File System Objects](#) — Nix Reference Manual — The Nix project **HIGH**
- [pcottle/learnGitBranching](#) — Peter Cottle **HIGH**
- [Learning resources: Version control with Git](#) — Princeton Research Computing **MEDIUM**
- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**
- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance](#) — GitLab Docs — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**
- <https://www.sitepoint.com/migrate-to-git-3-0-sha-256-and-reftables/> — cited in the note's Grounding gap as an unread secondary source to backfill and attribute
- <https://lwn.net/Articles/1094575/> — cited in the note's Grounding gap as an unread secondary source to backfill and attribute

2.11 V11 — Three-Way Merge Computation, Rebase, and First-Class Conflict Representation

This note establishes that a three-way merge is a function of exactly three trees — a common base and two sides — and that Git computes it by diffing each side against the base and applying each hunk where only one side changed, emitting a conflict only where both sides changed the same region. It treats merge commits, rebase, and squash merges as the same function with different callers: they differ only in which trees are fed in and what is done with the resulting tree in the object graph, not in the underlying algebra. Two harder cases are worked through in detail: when the merge base is not

unique (a criss-cross history), Git resolves this by recursively merging the candidate bases into a synthesised virtual base, which can itself carry an unresolved conflict that propagates into the outer merge; and when the merge output cannot be a clean tree, Git's `ort / recursive` machinery must fail and wait for `--continue`, whereas Jujutsu instead stores an unresolved merge as an odd-length ordered list of trees directly inside the commit. The evidence is drawn from Git's own manual pages, the commit and mailing-list threads that switched the default strategy from `recursive` to `ort` in Git 2.34.0, Jujutsu's technical documentation, and — for the criss-cross and stage-1 claims specifically — a reproduction run on Git 2.55.0 during this note's own research.

The note is explicit about what it does not claim. It takes no position on whether teams should prefer merging, rebasing, or squashing — it is grounding for how the computation works, not a workflow recommendation. It treats the `ort` performance figures as the strategy author's own unreplicated benchmarks on his own synthetic cases, not independently confirmed, and notes that independent replication against the old `recursive` backend is no longer even possible now that `recursive` has been redirected to mean `ort`. It presents the criss-cross recursive-merge heuristic as empirically justified (fewer conflicts on one historical test corpus) rather than provably optimal, and documents a known case where the heuristic can flip-flop indefinitely without a principled fix. On Jujutsu, it treats the practical claim that first-class conflicts make everyday merging materially less painful as unsettled — asserted by the project itself and reported sympathetically but without independent measurement by the one outside source consulted — and flags open questions about how Jujutsu's representation interacts with criss-cross topologies and whether Git could adopt a similar storable-conflict model without a data-format change.

Referenced sources

- [git-merge-base\(1\)](#) — The Git project **HIGH**
- [git-merge-strategies\(7\)](#) — The Git project **HIGH**
- [git-merge-file\(1\)](#) — The Git project **HIGH**
- [git-merge\(1\)](#) — The Git project **HIGH**
- [git-rebase\(1\)](#) — The Git project **HIGH**
- [Git 2.34 Release Notes](#) — The Git project **HIGH**
- [Git 2.33 Release Notes](#) — The Git project **HIGH**
- [Change default merge backend from recursive to ort — commit 6a5fb96](#) — Elijah Newren **HIGH**
- [Highlights from Git 2.33](#) — The GitHub Blog **HIGH**
- [Optimizing Git's Merge Machinery, #6](#) — Elijah Newren / Palantir Blog **HIGH**

- [merge-ort.c — "Ostensibly Recursive's Twin" header comment](#) — The Git project HIGH
- [fundamentals of merge-ort implementation \(PATCH v2 00/20\)](#) — Elijah Newren / git mailing list HIGH
- [Conflicts — Jujutsu technical documentation](#) — The Jujutsu project HIGH
- [docs/conflicts.md at v0.42.0 — Jujutsu](#) — The Jujutsu project HIGH
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project HIGH
- [Jujutsu: a new, Git-compatible version control system](#) — LWN.net HIGH
- [Criss-Cross Merge — revctrl.org wiki](#) — revctrl.org revision-control wiki MEDIUM
- [Criss-Cross — Breezy user reference](#) — The Breezy (Bazaar) project MEDIUM

2.12 V12 — Three-Way Merge Algorithms and Conflict Representation in Git and Jujutsu

This note works out how a three-way merge is actually computed and what follows from that mechanism. A merge is a pure function of three trees — a base plus two sides — because a two-way diff cannot distinguish "one side added this" from "the other side deleted it"; only a common ancestor resolves the ambiguity. Merge commits, rebase, and squash merge are then just different callers of the same function, differing only in which trees are fed in and where the output is written. The harder case is when the base itself is not unique: in a criss-cross history Git synthesizes a virtual base by recursively pre-merging the candidate ancestors, a step that can itself conflict and, if so, carries conflict markers inside the synthesized base — a claim the note verifies both against Git's own documentation and by direct reproduction on Git 2.55.0, showing the markers appear at index stage 1. It also traces the `ort` strategy's 2021 promotion to default (Git 2.34.0, correcting a widely repeated "2.33" error), its in-memory redesign that avoids the old quadratic index-array shifting, and contrasts this with Jujutsu, which stores an unresolved merge as an odd-length ordered list of trees directly on the commit, letting operations like rebase always complete without a `--continue` step.

The note is explicit about what it does not claim. It takes no position on whether to rebase or merge, and states outright that it is reference grounding for teaching, not a workflow recommendation. It does not evaluate whether Jujutsu's design is actually an improvement in practice — that question, per the note's own reading of independent reporting, remains unsettled and is left open rather than adjudicated. Several other points are marked as open or only partially addressed: whether `ort`'s benchmarked speedups replicate outside the original author's own three synthetic test cases (no independent benchmark was found, and the comparison is now largely unrunnable since the older strategy no longer exists as a separate implementation); how Jujutsu's conflict

representation behaves under a criss-cross topology specifically; and whether Git could adopt conflict-carrying commits without a storage-format change. The note also flags that the specific correctness bugs fixed by `ort`, as opposed to its performance defects, are named only by class rather than cited individually.

Referenced sources

- [git-merge-base\(1\)](#) HIGH
- [git-merge-strategies\(7\)](#) HIGH
- [git-merge-file\(1\)](#) HIGH
- [git-merge\(1\)](#) HIGH
- [git-rebase\(1\)](#) HIGH
- [Git 2.34 Release Notes](#) HIGH
- [Git 2.33 Release Notes](#) HIGH
- [Change default merge backend from recursive to ort — commit 6a5fb96](#) HIGH
- [Highlights from Git 2.33](#) HIGH
- [Optimizing Git's Merge Machinery, #6](#) HIGH
- [merge-ort.c — "Ostensibly Recursive's Twin" header comment](#) HIGH
- [fundamentals of merge-ort implementation \(PATCH v2 00/20\)](#) HIGH
- [Conflicts — Jujutsu technical documentation](#) HIGH
- [docs/conflicts.md at v0.42.0 — Jujutsu](#) HIGH
- [jj-vcs/jj — Jujutsu version control system](#) HIGH
- [Jujutsu: a new, Git-compatible version control system](#) HIGH
- [Criss-Cross Merge — revctrl.org wiki](#) MEDIUM
- [Criss-Cross — Breezy user reference](#) MEDIUM

2.13 V13 — Three-way merge algorithms and conflict representation

This note establishes that a three-way merge is a function of exactly three trees — a base plus two sides — computed by diffing each side against the base and, hunk by hunk, applying whichever side changed; where both sides changed the same region, the algorithm has no rule that decides and emits a conflict. It treats rebase, squash merge, and an ordinary merge commit as the same function called with different inputs and different placements for its output in the object graph, not as different algorithms. A large part of the note is devoted to the case where the base is not unique — a criss-cross history — and to Git's answer: recursively merging the candidate bases into a synthesised virtual base, which the note confirms, by direct reproduction on Git 2.55.0, can itself conflict, surfacing conflict markers inside index stage 1. It also docu-

ments the 2021 switch of Git's default merge strategy from `recursive` to `ort` (Git 2.34.0, not the widely repeated 2.33), the architectural reason — merging in memory instead of via the index and working tree — and the three reasons the author gave, in order: correctness, extensibility, performance. Finally, it describes Jujutsu's alternative: an unresolved conflict is stored in a commit as an ordered, odd-length list of trees read as an algebraic expression, closed under substitution, with textual conflict markers generated on demand rather than stored.

The note explicitly does not argue for or against rebasing, and takes no workflow or team-process position. It flags several open or unsettled points: no independently replicated benchmark of `ort`'s claimed speedups exists beyond the strategy author's own synthetic cases, and that comparison is now largely unrunnable since `recursive` stopped being a separate implementation in Git 2.50.0. The specific correctness bugs `ort` fixed are not catalogued in one document and are named here only by class, without per-bug citation. Jujutsu's one automatic conflict-resolution rule is acknowledged by the Jujutsu project itself to be lossy under rebase-and-rebase-back, and whether Jujutsu's conflict representation actually makes everyday merging less painful — as opposed to merely completing where Git would stop — is stated by the note's own independent source (LWN) as an open question. Also unresolved in the note: how Jujutsu's tree-list conflicts behave under a criss-cross topology, and whether Git could store an unresolved conflict without a data-format change.

Referenced sources

- [git-merge-base\(1\)](#) HIGH
- [git-merge-strategies\(7\)](#) HIGH
- [git-merge-file\(1\)](#) HIGH
- [git-merge\(1\)](#) HIGH
- [git-rebase\(1\)](#) HIGH
- [Git 2.34 Release Notes](#) HIGH
- [Git 2.33 Release Notes](#) HIGH
- [Change default merge backend from recursive to ort — commit 6a5fb96](#) HIGH
- [Highlights from Git 2.33](#) HIGH
- [Optimizing Git's Merge Machinery, #6](#) HIGH
- [merge-ort.c — "Ostensibly Recursive's Twin" header comment](#) HIGH
- [fundamentals of merge-ort implementation \(PATCH v2 00/20\)](#) HIGH
- [Conflicts — Jujutsu technical documentation](#) HIGH
- [docs/conflicts.md at v0.42.0 — Jujutsu](#) HIGH
- [jj-vcs/jj — Jujutsu version control system](#) HIGH

- [Jujutsu: a new, Git-compatible version control system](#) **HIGH**
- [Criss-Cross Merge — revctrl.org wiki](#) **MEDIUM**
- [Criss-Cross — Breezy user reference](#) **MEDIUM**

2.14 V14 — Three-way merge computation, criss-cross bases, and Jujutsu's first-class conflicts

This note establishes that a three-way merge is a pure function of three trees — a common base and two sides — computed by diffing each side against the base and, hunk by hunk, taking whichever side changed; overlap of changed line ranges, not meaning, is the entire conflict criterion. Merge commits, rebases, and squash merges are shown to be the same function with different inputs and different destinations for the output, differing only in the commit graph they leave behind. The harder case is when the base is not unique, as in a criss-cross history: Git resolves this by recursively merging the candidate bases into a synthesised virtual base, a step verified by direct reproduction on Git 2.55.0 showing conflict markers inside index stage 1 itself. The note also documents, and corrects, the history of Git's default strategy switch from `recursive` to `ort`, pinning it to Git 2.34.0 (15 November 2021) against primary release notes and the switching commit, rather than the commonly repeated but wrong 2.33 date. It contrasts this with Jujutsu, which stores an unresolved merge as an ordered, odd-length list of trees linked from the commit — a value a commit can hold — which is why Jujutsu's rebases and merges always complete without a `--continue` step, unlike Git's, which must halt because its merge machinery can only return a tree or fail.

The note explicitly does not argue for or against rebasing, merging, or squashing as a workflow choice; it stays at the mechanism level throughout. It is also careful about what its evidence does and doesn't support: conflict detection is textual and line-range-based rather than semantic, so it is both over- and under-sensitive by construction, and the note treats this as a designed trade-off rather than a bug. The `ort` strategy's large performance gains are reported as the strategy author's own unreplicated benchmarks on three synthetic cases, not independently verified figures. Jujutsu's design is credited with buying completion, not correctness — its one automatic conflict-resolution rule is explicitly lossy, by the project's own admission. Several questions are left open rather than resolved: whether `ort`'s speedups generalize beyond the author's benchmarks (now largely unrunnable, since `recursive` no longer exists as a separate implementation), how Jujutsu's conflict representation behaves under a criss-cross topology, and whether Git could adopt a conflict-carrying commit without changing its on-disk object format.

Referenced sources

- [git-merge-base\(1\)](#) — The Git project **HIGH**
- [git-merge-strategies\(7\)](#) — The Git project **HIGH**
- [git-merge-file\(1\)](#) — The Git project **HIGH**
- [git-merge\(1\)](#) — The Git project **HIGH**
- [git-rebase\(1\)](#) — The Git project **HIGH**
- [Git 2.34 Release Notes](#) — The Git project **HIGH**
- [Git 2.33 Release Notes](#) — The Git project **HIGH**
- [Change default merge backend from recursive to ort — commit 6a5fb96](#) — Elijah Newren **HIGH**
- [Highlights from Git 2.33](#) — The GitHub Blog **HIGH**
- [Optimizing Git's Merge Machinery, #6](#) — Elijah Newren / Palantir Blog **HIGH**
- [merge-ort.c — "Ostensibly Recursive's Twin" header comment](#) — The Git project **HIGH**
- [fundamentals of merge-ort implementation \(PATCH v2 00/20\)](#) — Elijah Newren / git mailing list **HIGH**
- [Conflicts](#) — Jujutsu technical documentation — The Jujutsu project **HIGH**
- [docs/conflicts.md at v0.42.0](#) — Jujutsu — The Jujutsu project **HIGH**
- [jj-vcs/jj](#) — Jujutsu version control system — The Jujutsu project **HIGH**
- [Jujutsu: a new, Git-compatible version control system](#) — LWN.net **HIGH**
- [Criss-Cross Merge](#) — revctrl.org wiki — revctrl.org revision-control wiki **MEDIUM**
- [Criss-Cross](#) — Breezy user reference — The Breezy (Bazaar) project **MEDIUM**

2.15 V15 — Three-Way Merge Computation and Conflict Representation in Git and Jujutsu

This note establishes that a Git merge is a deterministic function of exactly three trees — a base and two sides — and that a conflict arises only where both sides changed a common segment of lines relative to that base, a criterion stated in the plumbing command `git merge-file` and reused by every higher-level operation built on it. It works out the two knobs that distinguish those higher-level operations: which three trees are fed to the function, and where the resulting tree is written — a merge commit records two parents, a rebase replays the function once per commit and writes a new chain, a squash merge writes the result tree under a single parent. It then follows the harder case in detail: when a criss-cross history produces more than one best common ancestor, Git recursively merges the candidate bases into a synthesized virtual base before

running the outer merge, and that inner merge can itself conflict, in which case the conflict is carried into the virtual base rather than aborting the outer one. This mechanism is confirmed by terminal output reproduced on Git 2.55.0, showing conflict markers appearing directly inside stage 1 of the index during a constructed criss-cross merge. The note contrasts this with Jujutsu, which stores an unresolved conflict as an odd-length ordered list of trees linked from the commit itself, read as an algebraic expression and materialized into markers only on demand, which removes the need for commands like `--continue`.

The note is explicit about what it does not claim. It takes no position on rebasing versus merging as a workflow choice, and it does not evaluate whether Jujutsu's design is a practical improvement, noting that no source it consulted measures everyday usability rather than mechanism. It treats Git's recursive-base heuristic as justified only empirically, by reduced conflicts on a Linux kernel test corpus, not by any proof of correctness, and in pathological criss-cross histories it describes the approach as order-dependent with no guaranteed convergence. Several questions are left open: whether the `ort` strategy's large benchmark speedups, measured only by the strategy's own author against three synthetic cases, generalize elsewhere — a comparison the note says is now largely unrunnable, since the prior `recursive` implementation no longer exists separately; how Jujutsu's conflict representation behaves when a criss-cross topology meets an already-conflicted commit; and whether Git could store an unresolved conflict without a change to its on-disk object format. The note also names a genuine cost in Jujutsu's approach rather than treating it as strictly superior: its one automatic conflict-resolution rule is described by the project itself as lossy under a rebase-and-rebase-back sequence, and storing unresolved conflicts inside commit history means some commits may not build, which complicates bisection.

Referenced sources

- [git-merge-base\(1\)](#) — The Git project **HIGH**
- [git-merge-strategies\(7\)](#) — The Git project **HIGH**
- [git-merge-file\(1\)](#) — The Git project **HIGH**
- [git-merge\(1\)](#) — The Git project **HIGH**
- [git-rebase\(1\)](#) — The Git project **HIGH**
- [Git 2.34 Release Notes](#) — The Git project **HIGH**
- [Git 2.33 Release Notes](#) — The Git project **HIGH**
- [Change default merge backend from recursive to ort — commit 6a5fb96](#) — Elijah Newren **HIGH**
- [Highlights from Git 2.33](#) — The GitHub Blog **HIGH**
- [Optimizing Git's Merge Machinery, #6](#) — Elijah Newren / Palantir Blog **HIGH**

- [merge-ort.c — "Ostensibly Recursive's Twin" header comment](#) — The Git project HIGH
- [fundamentals of merge-ort implementation \(PATCH v2 00/20\)](#) — Elijah Newren / git mailing list, public-inbox archive HIGH
- [Conflicts — Jujutsu technical documentation](#) — The Jujutsu project HIGH
- [docs/conflicts.md at v0.42.0 — Jujutsu](#) — The Jujutsu project HIGH
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project HIGH
- [Jujutsu: a new, Git-compatible version control system](#) — LWN.net HIGH
- [Criss-Cross Merge — revctrl.org wiki](#) — revctrl.org revision-control wiki MEDIUM
- [Criss-Cross — Breezy user reference](#) — The Breezy (Bazaar) project MEDIUM

2.16 V16 — Git's Remote-Tracking Refs and the Fetch/Push/Pull Boundary

This note establishes that a Git remote is two configuration values — a name bound to a URL and a fetch refspec — and that the refspec's right-hand side creates a third ref namespace, `refs/remotes/*`, stored locally but describing another repository. Working from Git's own glossary and manuals at a pinned version, plus command transcripts run against live repositories, it shows that `fetch` only adds objects and moves that remote-tracking namespace, touching neither local branches nor the working tree; that `push` only adds objects to the remote and asks it to move its own branch refs, subject to a fast-forward rule the remote enforces as a stated safety feature; and that `pull` is `fetch` plus a local integration step. A reproduced transcript shows that `git status`'s ahead/behind figures and `git branch -vv`'s upstream annotations are computed entirely from the cached remote-tracking ref, so they report no change at all across a remote rewind and a branch deletion until a fetch actually runs. Reading Git's commit history and a maintainer's mailing-list review of the manual's own wording, the note also reconciles why `git-pull(1)` calls `--ff-only` "the default" while the unconfigured code path instead refuses outright on divergence: the manual states a policy and the code states a mechanism, and one precedence asymmetry between the two — an explicit `--ff-only` can refuse where the unset default would rebase or merge successfully — remains genuinely unresolved rather than a documentation defect.

The note is explicit about what it excludes: it is scoped to one remote and one refspec, leaving out multiple remotes, fork/upstream topologies, custom refspecs, and triangular push workflows. It flags its "objects plus refs" framing as a wire-level simplification — real transfers also negotiate capabilities and shallow boundaries, and `.git/FETCH_HEAD` is a side channel the two-namespace story omits. Several questions stay open: whether Git's divergence refusal is pedagogically better than a silent merge is called unverified, with no located study; a bare, non-explicit `--force-with-lease`'s doc-

umented behavior is cited from the manual but unreproduced, since `git push` itself was unavailable on the note's runner; and how much practical `origin/main` confusion comes from the refspec's silent renaming versus ordinary cache staleness is called genuinely open, since the one empirical study of Git difficulty it locates categorizes by command, not concept, and cannot apportion the two. The note likewise finds no design discussion explaining why Git surfaces no last-fetched timestamp in its porcelain.

Referenced sources

- *Documentation/glossary-content.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/glossary-content.adoc> **HIGH**
- *Documentation/git-pull.adoc*, `git-pull(1)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-pull.adoc> **HIGH**
- *builtin/pull.c* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/builtin/pull.c> **HIGH**
- *advice.c* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/advice.c> **HIGH**
- *Documentation/git-push.adoc*, `git-push(1)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-push.adoc> **HIGH**
- *Documentation/git-fetch.adoc*, `git-fetch(1)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch.adoc> **HIGH**
- *Documentation/pull-fetch-param.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/pull-fetch-param.adoc> **HIGH**
- *Documentation/config/branch.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/branch.adoc> **HIGH**
- *Documentation/config/receive.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/receive.adoc> **HIGH**
- *Documentation/revisions.adoc*, `gitrevisions(7)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/revisions.adoc> **HIGH**
- *Documentation/git-fetch-pack.adoc*, `git-fetch-pack(1)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch-pack.adoc> **HIGH**
- *Documentation/git-upload-pack.adoc*, `git-upload-pack(1)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-upload-pack.adoc> **HIGH**
- *Documentation/gitprotocol-pack.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/gitprotocol-pack.adoc> **HIGH**

- *Documentation/git-ls-remote.adoc*, `git-ls-remote(1)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-ls-remote.adoc> **HIGH**
- *Documentation/git-for-each-ref.adoc*, `git-for-each-ref(1)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-for-each-ref.adoc> **HIGH**
- *Documentation/git-status.adoc*, `git-status(1)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-status.adoc> **HIGH**
- *Git Internals — The Refspec* (Pro Git, 2nd ed.) — Scott Chacon and Ben Straub / Apress — <https://git-scm.com/book/en/v2/Git-Internals-The-Refspec> **HIGH**
- *pull: abort by default when fast-forwarding is not possible* (commit 031e2f7ae195069d00d21cde906fce5b0318dbdd) — Elijah Newren / Git project — <https://github.com/git/git/commit/031e2f7ae195069d00d21cde906fce5b0318dbdd> **HIGH**
- *pull: make --rebase and --no-rebase override pull.ff=only* (commit adc27d6a9374) — Elijah Newren / Git project — <https://github.com/git/git/commit/adc27d6a9374> **HIGH**
- *pull: don't say that merge is "the default strategy"* (commit 71076d0edde4) — Alex Henrie / Git project — <https://github.com/git/git/commit/71076d0edde4> **HIGH**
- *doc: git-pull: clarify options for integrating remote branch* (commit 59b28f928b2b) — Julia Evans / Git project — <https://github.com/git/git/commit/59b28f928b2b> **HIGH**
- *Re: [PATCH v2 2/4] doc: git-pull: clarify options for integrating remote branch* (maintainer review) — Junio C Hamano / git@vger.kernel.org — <https://public-inbox.org/git/xmqqqzvddqon.fsf@gitster.g/> **HIGH**
- *Re: pull.rebase config vs. --ff-only on command line* (mailing-list thread, superseded) — Felipe Contreras and others / git@vger.kernel.org — http://public-inbox.org/git/60f5b9a9e29d8_13f2e22084b@natae.notmuch/ **HIGH**
- *Documentation/git-branch.adoc*, `git-branch(1)` — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-branch.adoc> **HIGH**
- *push.default: Rename 'tracking' to 'upstream'* (commit 53c403116) — Johan Herland / Git project — <https://github.com/git/git/commit/53c403116> **HIGH**
- *The most confusing git terminology* — Mark Longair — <https://longair.net/blog/2012/05/07/the-most-confusing-git-terminology/> **MEDIUM**
- *Git remote branches and Git's missing terminology* — Mark Jason Dominus, The Universe of Discourse — <https://blog.plover.com/prog/git-remote-branches.html> **MEDIUM**
- *Why Git is hard* — roadrunnertwice — <https://roadrunnertwice.dreamwidth.org/596185.html> **MEDIUM**

- *Do Developers Really Know How to Use Git Commands? A Large-Scale Study Using Stack Overflow* — Wenhua Yang, Chong Zhang, Minxue Pan, Chang Xu, Yu Zhou, Zhiqiu Huang / ACM TOSEM — https://cs.nju.edu.cn/changxu/1_publications/22/TOSEM22.pdf **HIGH**
- *Dealing with diverged git branches* — Julia Evans — <https://jvns.ca/blog/2024/02/01/dealing-with-diverged-git-branches/> **MEDIUM**

2.17 V17 — Remotes, Remote-Tracking Refs, and the Fetch/Push/Pull Boundary

This note treats a Git remote as two configuration values — a name bound to a URL and a fetch refspec — rather than a connection, and argues that nearly all practical confusion in team use traces to the third ref namespace, `refs/remotes/*`, that the refspec creates. It walks through what each of fetch, push and pull actually does to objects and refs: fetch only adds objects and moves remote-tracking refs, push only writes to the remote's own branch refs under the remote's enforced fast-forward rule, and pull is fetch plus a local integration step whose outcome is governed by configuration rather than a fixed default. The author backs each claim with two kinds of evidence: direct quotation from Git's own documentation and source code read at a pinned tag (v2.55.0), and first-person command transcripts reproduced against real and scratch repositories — showing, for instance, that `rev-parse origin/main` and `git status` never consult the network and so can silently report stale ahead/behind counts, and that an unconfigured `git pull` refuses on divergence while `git pull --ff-only` fails through a different code path with a different message on identical state.

The note is explicit about its boundary: it covers exactly one remote and one refspec, deliberately excluding multiple remotes, fork/upstream topologies, custom or negative refspecs, and triangular push workflows, leaving those out of scope by design rather than by oversight. It does not claim to have resolved everything it investigated. It flags as unsettled whether the confusion commonly attributed to `origin/main` stems more from the refspec's silent renaming or from the remote-tracking cache simply going stale, noting that the one large empirical study it locates categorizes by command rather than by concept and so cannot apportion the two even in principle. It likewise leaves open whether Git's omission of a last-fetched timestamp from `status` and `branch -vv` is a considered design choice or simply unbuilt, and names one reproduction — the bare form of `--force-with-lease` — that it could not complete on its runner and cites rather than reproduces. Throughout, it treats itself as describing mechanism rather than recommending workflow, and corrects one of its own earlier inferences about `pull`'s documentation once the commit history falsified it.

Referenced sources

- *Documentation/glossary-content.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/glossary-content.adoc> **HIGH**
- *Documentation/git-pull.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-pull.adoc> **HIGH**
- *builtin/pull.c* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/builtin/pull.c> **HIGH**
- *advice.c* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/advice.c> **HIGH**
- *Documentation/git-push.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-push.adoc> **HIGH**
- *Documentation/git-fetch.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch.adoc> **HIGH**
- *Documentation/pull-fetch-param.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/pull-fetch-param.adoc> **HIGH**
- *Documentation/config/branch.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/branch.adoc> **HIGH**
- *Documentation/config/receive.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/receive.adoc> **HIGH**
- *Documentation/revisions.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/revisions.adoc> **HIGH**
- *Documentation/git-fetch-pack.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch-pack.adoc> **HIGH**
- *Documentation/git-upload-pack.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-upload-pack.adoc> **HIGH**
- *Documentation/gitprotocol-pack.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/gitprotocol-pack.adoc> **HIGH**
- *Documentation/git-ls-remote.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-ls-remote.adoc> **HIGH**
- *Documentation/git-for-each-ref.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-for-each-ref.adoc> **HIGH**
- *Documentation/git-status.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-status.adoc> **HIGH**
- *Git Internals — The Refspec* (Pro Git, 2nd ed.) — Scott Chacon and Ben Straub — <https://git-scm.com/book/en/v2/Git-Internals-The-Refspec> **HIGH**

- *pull: abort by default when fast-forwarding is not possible* — Elijah Newren / Git project — <https://github.com/git/git/commit/031e2f7ae195069d00d21cde906fce5b0318dbdd> **HIGH**
- *pull: make --rebase and --no-rebase override pull.ff=only* — Elijah Newren / Git project — <https://github.com/git/git/commit/ad27d6a9374> **HIGH**
- *pull: don't say that merge is "the default strategy"* — Alex Henrie / Git project — <https://github.com/git/git/commit/71076d0edde4> **HIGH**
- *doc: git-pull: clarify options for integrating remote branch* — Julia Evans / Git project — <https://github.com/git/git/commit/59b28f928b2b> **HIGH**
- *Re: [PATCH v2 2/4] doc: git-pull: clarify options for integrating remote branch* — Junio C Hamano — <https://public-inbox.org/git/xmqqqzvddqon.fsf@gitster.g/> **HIGH**
- *Re: pull.rebase config vs. --ff-only on command line* — Felipe Contreras and others — http://public-inbox.org/git/60f5b9a9e29d8_13f2e22084b@natae.notmuch/ **HIGH**
- *Documentation/git-branch.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-branch.adoc> **HIGH**
- *push.default: Rename 'tracking' to 'upstream'* — Johan Herland / Git project — <https://github.com/git/git/commit/53c403116> **HIGH**
- *The most confusing git terminology* — Mark Longair — <https://longair.net/blog/2012/05/07/the-most-confusing-git-terminology/> **MEDIUM**
- *Git remote branches and Git's missing terminology* — Mark Jason Dominus — <https://blog.plover.com/prog/git-remote-branches.html> **MEDIUM**
- *Why Git is hard* — roadrunnertwice — <https://roadrunnertwice.dreamwidth.org/596185.html> **MEDIUM**
- *Do Developers Really Know How to Use Git Commands? A Large-Scale Study Using Stack Overflow* — Wenhua Yang et al. / ACM TOSEM — https://cs.nju.edu.cn/changxu/1_publications/22/TOSEM22.pdf **HIGH**
- *Dealing with diverged git branches* — Julia Evans — <https://jvns.ca/blog/2024/02/01/dealing-with-diverged-git-branches/> **MEDIUM**

2.18 V18 — Remotes, Remote-Tracking Refs, and Refspecs in Git

This note extends an earlier note's claim — a Git repository is objects plus references, and every command adds objects or moves references — across the network boundary: `fetch`, `push`, and `pull` are that same operation with one new element, a third ref namespace, `refs/remotes/*`, stored locally but describing the remote's state as of the last fetch. A remote is two configuration values, not a connection — a name bound to a URL and a fetch refspec, both written by `clone` — and transcripts show that `fetch` only

adds objects and moves remote-tracking refs, leaving local branches and the working tree untouched, while `push` is the mirror write on the remote's `refs/heads/*`, gated by a fast-forward rule the remote itself enforces. Reproductions establish that ahead/behind counts are a claim about the last fetch rather than a live query, that the `+ force` flag is safe on remote-tracking refs but dangerous on local branches, and that `--force-with-lease` is a compare-and-swap against a cached value rather than an unconditional overwrite like `--force`. The central claim is that `pull` is not simply "fetch then merge": the glossary, the `git-pull(1)` manual, and the `builtin/pull.c` source describe three different things, and reproduced cases show the unset default deferring to `pull.rebase` configuration while an explicit `--ff-only` flag overrides that same configuration — so the default and the flag named after it can disagree on identical, divergent state.

The note covers exactly one remote and one refspec, excluding multiple remotes, fork/upstream topologies, and triangular push workflows, and flags "objects plus refs" as a simplification that omits capability negotiation, shallow/partial transfers, and the `FETCH_HEAD` side channel. Having read the commit trail behind the three conflicting descriptions, the note withdraws its own earlier framing of the `pull` discrepancy as a "documentation defect": the manual's wording was a deliberate 2025 addition, endorsed by the Git maintainer as a correct statement of policy rather than mechanism, and the only thing left unreconciled is the precedence asymmetry between the unset default and an explicit `--ff-only`. Several questions remain open and unverified: whether the refusal is pedagogically better than a silent merge, how much reported `origin/main` confusion traces to silent renaming versus caching staleness, and whether omitting a last-fetched timestamp from any porcelain command is deliberate or simply unbuilt. The bare form of `--force-with-lease` is cited from documentation but not reproduced, since the runner's `git push` was unavailable.

Referenced sources

- Documentation/glossary-content.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/glossary-content.adoc> **HIGH**
- Documentation/git-pull.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-pull.adoc> **HIGH**
- builtin/pull.c — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/builtin/pull.c> **HIGH**
- advice.c — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/advice.c> **HIGH**
- Documentation/git-push.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-push.adoc> **HIGH**
- Documentation/git-fetch.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch.adoc> **HIGH**

- Documentation/pull-fetch-param.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/pull-fetch-param.adoc> **HIGH**
- Documentation/config/branch.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/branch.adoc> **HIGH**
- Documentation/config/receive.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/receive.adoc> **HIGH**
- Documentation/revisions.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/revisions.adoc> **HIGH**
- Documentation/git-fetch-pack.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch-pack.adoc> **HIGH**
- Documentation/git-upload-pack.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-upload-pack.adoc> **HIGH**
- Documentation/gitprotocol-pack.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/gitprotocol-pack.adoc> **HIGH**
- Documentation/git-ls-remote.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-ls-remote.adoc> **HIGH**
- Documentation/git-for-each-ref.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-for-each-ref.adoc> **HIGH**
- Documentation/git-status.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-status.adoc> **HIGH**
- Git Internals — The Refspec (Pro Git, 2nd ed.) — Scott Chacon and Ben Straub / Apress — <https://git-scm.com/book/en/v2/Git-Internals-The-Refspec> **HIGH**
- pull: abort by default when fast-forwarding is not possible (commit 031e2f7a) — Elijah Newren, Git project — <https://github.com/git/git/commit/031e2f7ae195069d00d21cde906f5b0318dbdd> **HIGH**
- pull: make --rebase and --no-rebase override pull.ff=only (commit adc27d6a9374) — Elijah Newren, Git project — <https://github.com/git/git/commit/adc27d6a9374> **HIGH**
- pull: don't say that merge is "the default strategy" (commit 71076d0edde4) — Alex Henrie, Git project — <https://github.com/git/git/commit/71076d0edde4> **HIGH**
- doc: git-pull: clarify options for integrating remote branch (commit 59b28f928b2b) — Julia Evans, Git project — <https://github.com/git/git/commit/59b28f928b2b> **HIGH**
- Re: [PATCH v2 2/4] doc: git-pull: clarify options for integrating remote branch — Junio C Hamano, git@vger.kernel.org — <https://public-inbox.org/git/xmqqqzvd-dqon.fsf@gitster.g/> **HIGH**

- Re: pull.rebase config vs. --ff-only on command line — Felipe Contreras and others, git@vger.kernel.org — http://public-inbox.org/git/60f5b9a9e29d8_13f2e22084b@natae.notmuch/ **HIGH**
- Documentation/git-branch.adoc — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-branch.adoc> **HIGH**
- push.default: Rename 'tracking' to 'upstream' (commit 53c403116) — Johan Herland, Git project — <https://github.com/git/git/commit/53c403116> **HIGH**
- The most confusing git terminology — Mark Longair — <https://longair.net/blog/2012/05/07/the-most-confusing-git-terminology/> **MEDIUM**
- Git remote branches and Git's missing terminology — Mark Jason Dominus, The Universe of Discourse — <https://blog.plover.com/prog/git-remote-branches.html> **MEDIUM**
- Why Git is hard — roadrunnertwice — <https://roadrunnertwice.dreamwidth.org/596185.html> **MEDIUM**
- Do Developers Really Know How to Use Git Commands? A Large-Scale Study Using Stack Overflow — Wenhua Yang, Chong Zhang, Minxue Pan, Chang Xu, Yu Zhou, Zhiqiu Huang / ACM TOSEM — https://cs.nju.edu.cn/changxu/1_publications/22/TOSEM22.pdf **HIGH**
- Dealing with diverged git branches — Julia Evans — <https://jvns.ca/blog/2024/02/01/dealing-with-diverged-git-branches/> **MEDIUM**

2.19 V19 — Remotes, remote-tracking refs, and refspecs

This note's claim is that Git's remote machinery adds exactly one new idea on top of the local objects-plus-refs model: a third ref namespace, `refs/remotes/*`, that belongs cleanly to neither side. A remote is not a connection but two stored configuration values — a name bound to a URL, and a fetch refspec naming which refs to copy and where. From there the three verbs partition cleanly and are each verified by command-line reproduction against a real repository pinned to Git 2.55.0, not citation alone: `fetch` only adds objects and moves the local `refs/remotes/origin/*` refs, leaving local branches and the working tree untouched; `push` adds objects to the remote and asks it to move its own `refs/heads/*`, subject to a fast-forward rule the remote itself enforces; `pull` is fetch plus a local integration step. A stale checkout showed `git status` reporting no divergence while the live server was a commit ahead, demonstrating that every ahead/behind number is a comparison between two local refs, accurate only as of the last fetch. The note's one contested finding is that Git's glossary, its `pull` manual page, and its `pull` source code each describe the diverged-branches default differently; read-

ing the commit history behind all three resolves this as a deliberate, dated set of wording changes rather than an inconsistency, though a precedence asymmetry between the unset default and an explicit `--ff-only` flag remains genuinely unreconciled.

The note is explicitly scoped to one remote and one refspec, leaving multiple remotes, fork/upstream triangular workflows, and custom or negative refspecs out of bounds by design. It flags its own "objects plus refs" framing as a wire-level simplification, since real transfers also negotiate capabilities and shallow boundaries. It pulls back from an earlier draft's claim that the `pull` manual's wording is a "documentation defect," noting the code is self-consistent and the manual's phrasing was reviewed and accepted by the project's maintainer as a description of policy rather than mechanism. It leaves several questions explicitly open: whether the refusal on divergent branches is pedagogically better than a silent merge or merely relocates the same confusion, whether any user-facing Git output ever surfaces how stale a remote-tracking ref is (none found), and how much of the practical `origin/main` confusion is attributable to the ref's silent rename versus its caching — a split no source it found attempts to apportion. It makes no workflow recommendation.

Referenced sources

- Documentation/glossary-content.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/glossary-content.adoc> **HIGH**
- Documentation/git-pull.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-pull.adoc> **HIGH**
- builtin/pull.c — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/builtin/pull.c> **HIGH**
- advice.c — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/advice.c> **HIGH**
- Documentation/git-push.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-push.adoc> **HIGH**
- Documentation/git-fetch.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch.adoc> **HIGH**
- Documentation/pull-fetch-param.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/pull-fetch-param.adoc> **HIGH**
- Documentation/config/branch.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/branch.adoc> **HIGH**

- Documentation/config/receive.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/receive.adoc> **HIGH**
- Documentation/revisions.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/revisions.adoc> **HIGH**
- Documentation/git-fetch-pack.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch-pack.adoc> **HIGH**
- Documentation/git-upload-pack.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-upload-pack.adoc> **HIGH**
- Documentation/gitprotocol-pack.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/gitprotocol-pack.adoc> **HIGH**
- Documentation/git-ls-remote.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-ls-remote.adoc> **HIGH**
- Documentation/git-for-each-ref.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-for-each-ref.adoc> **HIGH**
- Documentation/git-status.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-status.adoc> **HIGH**
- Git Internals — The Refspec (Pro Git, 2nd ed.), Scott Chacon and Ben Straub / Apress — <https://git-scm.com/book/en/v2/Git-Internals-The-Refspec> **HIGH**
- pull: abort by default when fast-forwarding is not possible — commit 031e2f7ae195069d00d21cde906fce5b0318dbdd, Elijah Newren et al / Git project (2021-07-22) — <https://github.com/git/git/commit/031e2f7ae195069d00d21cde906fce5b0318dbdd> **HIGH**
- pull: make --rebase and --no-rebase override pull.ff=only — commit adc27d6a9374, Elijah Newren / Git project (2021-07-22) — <https://github.com/git/git/commit/adc27d6a9374> **HIGH**
- pull: don't say that merge is "the default strategy" — commit 71076d0edde4, Alex Henrie / Git project (2021-11-18) — <https://github.com/git/git/commit/71076d0edde4> **HIGH**
- doc: git-pull: clarify options for integrating remote branch — commit 59b28f928b2b, Julia Evans / Git project (2025-10-15) — <https://github.com/git/git/commit/59b28f928b2b> **HIGH**

- Re: [PATCH v2 2/4] doc: git-pull: clarify options for integrating remote branch — Junio C Hamano, git@vger.kernel.org (2025-10-08) — <https://public-inbox.org/git/xm-qqqvzddqon.fsf@gitster.g/> **HIGH**
- Re: pull.rebase config vs. --ff-only on command line — Felipe Contreras and others, git@vger.kernel.org (2021), superseded — http://public-inbox.org/git/60f5b9a9e29d8_13f2e22084b@natae.notmuch/ **HIGH**
- Documentation/git-branch.adoc — Git project (2026), at tag v2.55.0 — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-branch.adoc> **HIGH**
- push.default: Rename 'tracking' to 'upstream' — commit 53c403116, Johan Herland / Git project (2011-02-16) — <https://github.com/git/git/commit/53c403116> **HIGH**
- The most confusing git terminology — Mark Longair (2012-05-07) — <https://longair.net/blog/2012/05/07/the-most-confusing-git-terminology/> **MEDIUM**
- Git remote branches and Git's missing terminology — Mark Jason Dominus, The Universe of Discourse (2017-06-17) — <https://blog.plover.com/prog/git-remote-branches.html> **MEDIUM**
- Why Git is hard — roadrunnertwice (2023-09-24) — <https://roadrunner-twice.dreamwidth.org/596185.html> **MEDIUM**
- Do Developers Really Know How to Use Git Commands? A Large-Scale Study Using Stack Overflow — Yang, Zhang, Pan, Xu, Zhou, Huang / ACM TOSEM (2022) — https://cs.nju.edu.cn/changxu/1_publications/22/TOSEM22.pdf **HIGH**
- Dealing with diverged git branches — Julia Evans (2024-02-01) — <https://jvns.ca/blog/2024/02/01/dealing-with-diverged-git-branches/> **MEDIUM**

2.20 V20 — Git's Remote, Fetch, Push, and Pull Mechanics

This note extends the "a repository is objects plus references" model across the network boundary, arguing that `fetch` and `push` are that same add-objects-and-move-refs operation performed against a remote, with one genuinely new element: a third ref namespace, `refs/remotes/*`, that caches the remote's branches locally. A remote is defined as nothing more than a name bound to a URL plus a fetch refspec; `clone` writes both. The note separates the three verbs cleanly: `fetch` is the only read, adding objects and moving only `refs/remotes/origin/*`, touching neither local branches nor the working tree; `push` is the only write, asking the remote to move its own `refs/heads/*` subject to a fast-forward rule the remote itself enforces; `pull` is `fetch` plus a local integration step. Through direct command reproduction, the note demonstrates that `origin/main` is a local file, not a live query — `git rev-parse origin/main` and `git ls-remote origin` can disagree, and `git status`'s ahead/behind figures, `git branch -vv`'s `[ahead N]`, and even `[gone]` are all computed from that local cache and therefore lag

reality by exactly one fetch. It also works through a documented three-way discrepancy about what `pull` does by default: Git's glossary calls it an unconditional merge, the manual names `--ff-only` as the default, and the source code's actual unset behavior is to refuse when branches have diverged, deferring to `pull.rebase` / `pull.ff` configuration otherwise — all reproduced as four distinct behavioral cases. Evidence throughout is Git's own pinned-version source and documentation plus first-person command transcripts.

The note explicitly limits itself to one remote and one refspec, excluding multiple remotes, fork/upstream topologies, triangular push workflows, and most wire-protocol detail beyond one summary sentence. It does not claim its own "objects plus refs" framing is complete at the wire level, noting that real transfers also negotiate capabilities and write side-channel state like `FETCH_HEAD`. It explicitly declines to say whether the merge/rebase refusal on divergence is pedagogically better than a silent default, calling that an open empirical question. It also flags as unsettled how much of the common `origin/main` confusion is attributable to the remote-tracking rename versus simple staleness — several sources treat these as distinct failure modes, but none apportions them — and whether Git's omission of a last-fetch timestamp from its status reporting is a deliberate design choice or simply unbuilt. The note separately documents, as a correction to its own earlier draft, that a wording choice in Git's manual it had suspected of being stale documentation was in fact a deliberate, recently-added clarification.

Referenced sources

- *Documentation/glossary-content.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/glossary-content.adoc> **HIGH**
- *Documentation/git-pull.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-pull.adoc> **HIGH**
- *builtin/pull.c*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/builtin/pull.c> **HIGH**
- *advice.c*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/advice.c> **HIGH**
- *Documentation/git-push.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-push.adoc> **HIGH**
- *Documentation/git-fetch.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch.adoc> **HIGH**
- *Documentation/pull-fetch-param.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/pull-fetch-param.adoc> **HIGH**
- *Documentation/config/branch.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/branch.adoc> **HIGH**

- *Documentation/config/receive.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/receive.adoc> **HIGH**
- *Documentation/revisions.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/revisions.adoc> **HIGH**
- *Documentation/git-fetch-pack.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch-pack.adoc> **HIGH**
- *Documentation/git-upload-pack.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-upload-pack.adoc> **HIGH**
- *Documentation/gitprotocol-pack.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/gitprotocol-pack.adoc> **HIGH**
- *Documentation/git-ls-remote.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-ls-remote.adoc> **HIGH**
- *Documentation/git-for-each-ref.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-for-each-ref.adoc> **HIGH**
- *Documentation/git-status.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-status.adoc> **HIGH**
- *Git Internals — The Refspec* (Pro Git, 2nd ed.), Scott Chacon and Ben Straub — <https://git-scm.com/book/en/v2/Git-Internals-The-Refspec> **HIGH**
- *pull: abort by default when fast-forwarding is not possible*, commit 031e2f7ae195069d00d21cde906fce5b0318dbdd — <https://github.com/git/git/commit/031e2f7ae195069d00d21cde906fce5b0318dbdd> **HIGH**
- *pull: make --rebase and --no-rebase override pull.ff=only*, commit adc27d6a9374 — <https://github.com/git/git/commit/adc27d6a9374> **HIGH**
- *pull: don't say that merge is "the default strategy"*, commit 71076d0edde4 — <https://github.com/git/git/commit/71076d0edde4> **HIGH**
- *doc: git-pull: clarify options for integrating remote branch*, commit 59b28f928b2b — <https://github.com/git/git/commit/59b28f928b2b> **HIGH**
- *Re: [PATCH v2 2/4] doc: git-pull: clarify options for integrating remote branch* (maintainer review), Junio C Hamano — <https://public-inbox.org/git/xmqqqzvd-dqon.fsf@gitster.g/> **HIGH**
- *Re: pull.rebase config vs. --ff-only on command line* (mailing-list thread, superseded), Felipe Contreras and others — http://public-inbox.org/git/60f5b9a9e29d8_13f2e22084b@natae.notmuch/ **HIGH**
- *Documentation/git-branch.adoc*, Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-branch.adoc> **HIGH**
- *push.default: Rename 'tracking' to 'upstream'*, commit 53c403116 — <https://github.com/git/git/commit/53c403116> **HIGH**

- *The most confusing git terminology*, Mark Longair — <https://longair.net/blog/2012/05/07/the-most-confusing-git-terminology/> **MEDIUM**
- *Git remote branches and Git's missing terminology*, Mark Jason Dominus — <https://blog.plover.com/prog/git-remote-branches.html> **MEDIUM**
- *Why Git is hard*, roadrunnertwice — <https://roadrunnertwice.dreamwidth.org/596185.html> **MEDIUM**
- *Do Developers Really Know How to Use Git Commands? A Large-Scale Study Using Stack Overflow*, Wenhua Yang et al. / ACM TOSEM — https://cs.nju.edu.cn/changxu/1_publications/22/TOSEM22.pdf **HIGH**
- *Dealing with diverged git branches*, Julia Evans — <https://jvns.ca/blog/2024/02/01/dealing-with-diverged-git-branches/> **MEDIUM**

2.21 V21 — Git remotes, remote-tracking refs, and what fetch, push, and pull do to them

This note explains what Git's `fetch`, `push`, and `pull` commands actually do to objects and references once a remote repository is involved, building on a companion note's claim that a Git repository is nothing but objects plus references and that every command either adds objects or moves references. It argues that a remote is just two configuration values — a name bound to a URL and a fetch refspec — and that the network boundary adds exactly one new thing, a local ref namespace (`refs/remotes/*`) that caches what the remote last reported rather than the remote itself. Every claim is backed by a quotation from Git's own documentation or source code at a single pinned version, paired with first-person command transcripts run in the same environment: a fresh clone's recorded refspec, `fetch` leaving local branches and the working tree untouched while moving only remote-tracking refs, a divergent `git pull` with nothing configured refusing outright, and a `--force-with-lease` `push` rejected as "stale info" once the cached value no longer matches the server. The most developed finding concerns `git pull`: its manual page's stated default (fast-forward-only) and its actual unconfigured behavior (an outright refusal pending configuration) are shown to be distinct, a discrepancy traced through the project's commit history to a deliberate 2021 behavior change and a 2025 documentation clarification that the maintainer reviewed and endorsed as a description of policy rather than mechanism.

The note is explicitly scoped to one remote and one refspec, leaving multiple remotes, fork/upstream triangular workflows, and custom or negative refsspecs out of scope by design. It treats the "objects plus references" model as a simplification of the wire protocol, which also negotiates capabilities and shallow or partial transfer boundaries that the note does not otherwise cover. On its most contested point — why the manual and

the code describe `pull`'s default differently — it concludes the two are not in error but describe different levels (policy versus mechanism), while flagging one precedence asymmetry between the unconfigured default and an explicit `--ff-only` flag as genuinely unreconciled. It leaves open whether reported user confusion about `origin/main` owes more to the ref-renaming behavior or to staleness, since the one large empirical study it found cannot separate the two even in principle, and it marks as explicitly unverified whether Git's refusal-over-silent-merge design for divergent branches is pedagogically preferable to the alternative, noting that no study of that question was located.

Referenced sources

- *Documentation/glossary-content.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/glossary-content.adoc> **HIGH**
- *Documentation/git-pull.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-pull.adoc> **HIGH**
- *builtin/pull.c* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/builtin/pull.c> **HIGH**
- *advice.c* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/advice.c> **HIGH**
- *Documentation/git-push.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-push.adoc> **HIGH**
- *Documentation/git-fetch.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch.adoc> **HIGH**
- *Documentation/pull-fetch-param.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/pull-fetch-param.adoc> **HIGH**
- *Documentation/config/branch.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/branch.adoc> **HIGH**
- *Documentation/config/receive.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/config/receive.adoc> **HIGH**
- *Documentation/revisions.adoc* (gitrevisions(7)) — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/revisions.adoc> **HIGH**
- *Documentation/git-fetch-pack.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-fetch-pack.adoc> **HIGH**
- *Documentation/git-upload-pack.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-upload-pack.adoc> **HIGH**
- *Documentation/gitprotocol-pack.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/gitprotocol-pack.adoc> **HIGH**

- *Documentation/git-ls-remote.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-ls-remote.adoc> **HIGH**
- *Documentation/git-for-each-ref.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-for-each-ref.adoc> **HIGH**
- *Documentation/git-status.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-status.adoc> **HIGH**
- *Git Internals — The Refspec* (Pro Git, 2nd ed.) — Scott Chacon and Ben Straub — <https://git-scm.com/book/en/v2/Git-Internals-The-Refspec> **HIGH**
- *pull: abort by default when fast-forwarding is not possible* (commit 031e2f7a) — Elijah Newren / Git project — <https://github.com/git/git/commit/031e2f7ae195069d00d21cde906f5b0318dbdd> **HIGH**
- *pull: make --rebase and --no-rebase override pull.ff=only* (commit adc27d6a) — Elijah Newren / Git project — <https://github.com/git/git/commit/adc27d6a9374> **HIGH**
- *pull: don't say that merge is "the default strategy"* (commit 71076d0e) — Alex Henrie / Git project — <https://github.com/git/git/commit/71076d0edde4> **HIGH**
- *doc: git-pull: clarify options for integrating remote branch* (commit 59b28f92) — Julia Evans / Git project — <https://github.com/git/git/commit/59b28f928b2b> **HIGH**
- *Re: [PATCH v2 2/4] doc: git-pull: clarify options for integrating remote branch* (maintainer review) — Junio C Hamano — <https://public-inbox.org/git/xmqqqzvd-dqon.fsf@gitster.g/> **HIGH**
- *Re: pull.rebase config vs. --ff-only on command line* — Felipe Contreras and others — http://public-inbox.org/git/60f5b9a9e29d8_13f2e22084b@natae.notmuch/ **HIGH**
- *Documentation/git-branch.adoc* — Git project — <https://raw.githubusercontent.com/git/git/v2.55.0/Documentation/git-branch.adoc> **HIGH**
- *push.default: Rename 'tracking' to 'upstream'* (commit 53c403116) — Johan Herland / Git project — <https://github.com/git/git/commit/53c403116> **HIGH**
- *The most confusing git terminology* — Mark Longair — <https://longair.net/blog/2012/05/07/the-most-confusing-git-terminology/> **MEDIUM**
- *Git remote branches and Git's missing terminology* — Mark Jason Dominus — <https://blog.plover.com/prog/git-remote-branches.html> **MEDIUM**
- *Why Git is hard* — roadrunnertwice — <https://roadrunnertwice.dreamwidth.org/596185.html> **MEDIUM**
- *Do Developers Really Know How to Use Git Commands? A Large-Scale Study Using Stack Overflow* — Wenhua Yang, Chong Zhang, Minxue Pan, Chang Xu, Yu Zhou, Zhiqiu Huang / ACM TOSEM — https://cs.nju.edu.cn/changxu/1_publications/22/TOSEM22.pdf **HIGH**

- *Dealing with diverged git branches* — Julia Evans — <https://jvns.ca/blog/2024/02/01/dealing-with-diverged-git-branches/> MEDIUM

2.22 V22 — The relocation of version-control centralisation from the data model into policy

This note argues that distributed version control does not eliminate a single point of authority; it removes that point from the data model and leaves it to be rebuilt as policy. The argument rests on separating three independent axes — centralised versus decentralised authority, locking versus merge-based concurrency, and file- versus file-set-oriented units — which the historical lineage (SCCS, RCS, CVS, Subversion, Perforce, BitKeeper, Git) correlates but does not logically require. Some capabilities a central server provided are only enforceable because the server was structural: exclusive locking (traced to RCS's "exclusive responsibility" rule and re-implemented by Subversion's `svn lock` and Perforce's `+l` filetype), per-path access control (Subversion's `authz` file and Perforce's `protections` table, both server-side tables consulted per command), and paying large-binary storage cost once rather than once per clone. Offline capability is shown to be structural rather than a feature, including a first-hand reproduction in which a Subversion working copy — holding only the pristine text of one checked-out revision — answered `status / diff / info / cat` with its repository made unreachable, but failed `log`, `blame`, `update`, and `commit`. Forges (GitHub branch protection) and Radicle's cryptographic quorum are offered as evidence that authority returns as policy once distribution removes it from the data model, and Epic Games' and Unity's documentation corroborate that game studios still rely on Perforce-style exclusive locking for unmergeable binary assets.

The note is explicit about what it does not claim. "Distributed systems cannot offer locking" is true of the data model but not of practice, since Git LFS locking and forge features implement locking as policy at a single authoritative remote — the limit is that policy cannot bind two peers who exchange work directly, not that locking is unavailable. It also flags that "authority is merely a social convention" understates the real cost of relocating authority once branch protection, CI, and identity all live at one forge. Several points are left open rather than resolved: whether centralised systems retain a genuine or merely legacy advantage for large binaries is called contested; Perforce's market-share claim among AAA studios is sourced only to vendor marketing and is marked unverified; the semantics of Git LFS locking were not researched; and the off-line reproduction used a renamed, unreachable local repository rather than a genuine network partition, a gap the note states rather than papers over.

Referenced sources

- [Making Sense of Revision-Control Systems](#) — Bryan O'Sullivan / Communications of the ACM (2009-09-01) **HIGH**
- [Understanding Version-Control Systems \(DRAFT\)](#) — Eric S. Raymond / catb.org (undated draft) **MEDIUM**
- [RCS—A System for Version Control](#) — Walter F. Tichy / Department of Computer Sciences, Purdue University **HIGH**
- [Version Control with Subversion — Locking](#) — Collins-Sussman, Fitzpatrick & Pilato / svnbook.red-bean.com (1.7 edition) **HIGH**
- [Version Control with Subversion — Path-Based Authorization](#) — Collins-Sussman, Fitzpatrick & Pilato / svnbook.red-bean.com (1.7 edition) **HIGH**
- [Set protections with p4 protect](#) — Perforce Software / P4 Server Administration Documentation **HIGH**
- [How protections are implemented](#) — Perforce Software / P4 Server Administration Documentation (2025.1) **HIGH**
- [Exclusive File Locking](#) — Perforce Software / Perforce Knowledge Base, article 3114 (2022-04-10) **HIGH**
- [Using Perforce as Source Control \(Unreal Engine\)](#) — Epic Games / Unreal Engine Documentation **HIGH**
- [dev.epicgames.com/documentation/en-us/unreal-engine/using-perforce-as-source-control-for-unreal-engine](#) — Epic Games / Unreal Engine Documentation (successor page to [9], referenced in its credibility note)
- [Plugin for Unreal — Unity Version Control](#) — Unity Technologies / Unity Documentation **MEDIUM**
- [BitKeeper for Kernel Developers](#) — Val Henson & Jeff Garzik / Proceedings of the Ottawa Linux Symposium (2002) **HIGH**
- [Interview with BitKeeper author Larry McVoy](#) — Jeremy Andrews (KernelTrap) / Linux.com (2002-05-28) **MEDIUM**
- [BitKeeper](#) — Wikipedia — Wikipedia contributors (fetched 2026-09-28) **MEDIUM**
- [BitKeeper goes open source](#) — LWN.net (2016-05-11) **HIGH**
- [About protected branches](#) — GitHub / GitHub Docs **HIGH**
- [Canonical References in Radicle: A Hands-On Guide](#) — Radicle Garden (2026-06-26) **MEDIUM**
- [The Essence of Git: Concepts for P2P Replication](#) — Erick Lavoie / arXiv:2609.13555 (2026) **MEDIUM**

- [What is version control: centralized vs. DVCS](#) — Giancarlo Lionetti / Atlassian (2012-02-14) **MEDIUM**
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Git](#) — Wikipedia — Wikipedia contributors **MEDIUM**

2.23 V23 — The relocation of centralisation from data model to policy

This note argues that centralised and distributed version control differ structurally, not merely in deployment topology: a centralised system makes a single point of authority a fact of its data model, while a distributed system removes that fact, so authority returns only as a convention — one that forges, such as GitHub, now implement as policy through branch-protection rules requiring approving reviews before a merge. The consequence is not symmetric. Several guarantees a central server provided were enforceable only because the server was structural: exclusive locking, per-path access control, and paying binary-history storage once rather than on every clone. The note traces the locking lineage from RCS's exclusive-checkout rule, through CVS's merge-based break with it, to Subversion's deliberate reintroduction of locking scoped to unmergeable content, and shows Perforce implementing the same exclusive-open guarantee at server scale, with Unreal Engine's and Unity's documentation independently converging on the same binary-asset workflow. It documents per-path access control as a server-side table consulted per command in both Subversion and Perforce, with no Git-native equivalent, and treats forge branch protection — and Radicle's cryptographic-quorum reconstruction of it once the server is removed — as the same re-centralisation pattern that BitKeeper's licence, rather than its architecture, imposed two decades earlier.

The note is explicit about its limits. It discloses that its most-cited source has a conflict of interest — a Mercurial contributor who was also party to the BitKeeper licensing dispute — and qualifies "distributed systems cannot by their nature offer locking" as true of the data model but false of practice, since Git LFS locking and forge features implement locking as policy at a single authoritative remote. It marks Perforce's claimed market share among AAA studios as vendor-sourced and unverified by independent survey, distinguishing that figure from the underlying locking mechanism, which it treats as verified through two competing first-party sources. It leaves open whether centralised systems retain a genuine or merely legacy advantage on large binaries, calling this contested among the sources it cites. Its offline-versus-online demonstration used a renamed, unreachable Subversion repository rather than an actual network partition, a simulation the note states rather than treats as equivalent. It also flags that

treating authority as a single scalar does not fit systems with genuinely partitioned per-path authority, and leaves several questions — including Git LFS locking's exact semantics — open.

Referenced sources

- [Making Sense of Revision-Control Systems](#) — Bryan O'Sullivan / Communications of the ACM (2009-09-01) **HIGH**
- [Understanding Version-Control Systems \(DRAFT\)](#) — Eric S. Raymond / catb.org (undated draft) **MEDIUM**
- [RCS—A System for Version Control](#) — Walter F. Tichy / Department of Computer Sciences, Purdue University **HIGH**
- [Version Control with Subversion — Locking](#) — Collins-Sussman, Fitzpatrick & Pilato / svnbook.red-bean.com (1.7 edition) **HIGH**
- [Version Control with Subversion — Path-Based Authorization](#) — Collins-Sussman, Fitzpatrick & Pilato / svnbook.red-bean.com (1.7 edition) **HIGH**
- [Set protections with p4 protect](#) — Perforce Software / P4 Server Administration Documentation **HIGH**
- [How protections are implemented](#) — Perforce Software / P4 Server Administration Documentation (2025.1) **HIGH**
- [Exclusive File Locking](#) — Perforce Software / Perforce Knowledge Base, article 3114 (2022-04-10) **HIGH**
- [Using Perforce as Source Control \(Unreal Engine\)](#) — Epic Games / Unreal Engine Documentation (4.27 edition) **HIGH**
- [Using Perforce as Source Control for Unreal Engine \(successor page\)](#) — Epic Games / dev.epicgames.com (current, serving UE 5.8)
- [Plugin for Unreal — Unity Version Control](#) — Unity Technologies / Unity Documentation **MEDIUM**
- [BitKeeper for Kernel Developers](#) — Val Henson & Jeff Garzik / Proceedings of the Ottawa Linux Symposium (2002) **HIGH**
- [Interview with BitKeeper author Larry McVoy](#) — Jeremy Andrews (KernelTrap) / Linux.com (2002-05-28) **MEDIUM**
- [BitKeeper](#) — Wikipedia — Wikipedia contributors (fetched 2026-09-28) **MEDIUM**
- [BitKeeper goes open source](#) — LWN.net (2016-05-11) **HIGH**
- [About protected branches](#) — GitHub / GitHub Docs **HIGH**
- [Canonical References in Radicle: A Hands-On Guide](#) — Radicle Garden (2026-06-26) **MEDIUM**

- [The Essence of Git: Concepts for P2P Replication](#) — Erick Lavoie / arXiv:2609.13555 (2026) **MEDIUM**
- [What is version control: centralized vs. DVCS](#) — Giancarlo Lionetti / Atlassian (2012-02-14) **MEDIUM**
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Git — Wikipedia](#) — Wikipedia contributors **MEDIUM**

2.24 V24 — Centralisation relocated from the data model into policy

This note argues that centralised and distributed version control differ in where authority lives, not whether it exists: a centralised system makes a single point of authority a structural fact of its data model, while a distributed system removes that fact without removing the need for an authoritative copy, so authority returns as policy, a social convention that forges such as GitHub now implement as branch-protection rules. The consequence is asymmetric: some capabilities a central server provided were only enforceable because the server was structural, exclusive locking, per-path access control, and paying the storage cost of large binary history once rather than on every clone, and policy layered above a peer-to-peer data model can approximate these for a shared remote but cannot bind two peers who exchange work directly. The note traces this through the RCS-to-Git locking lineage (locking as the original, exclusive-responsibility concurrency model; CVS and Git replacing it with merge-by-default; Subversion and Perforce retaining it deliberately for content that cannot be merged, including game-engine binary assets), through Subversion's and Perforce's server-side per-path authorization tables, and through GitHub's protected branches and Radicle's cryptographic quorum, read here as a decentralised system rebuilding branch protection from scratch once the server is gone.

The note is explicit about what it does not claim. It does not argue centralised systems are straightforwardly superior: its sources disagree, one calling distribution strictly dominant, another describing distributed tools as still behind on large binaries, a disagreement the note calls contested rather than resolved. It flags that its most-cited source has a disclosed conflict of interest, a Mercurial contributor and party to the BitKeeper licensing dispute, and treats his comparative verdicts, as distinct from his checkable descriptive claims, with caution. The claim that distributed systems cannot by nature offer locking is qualified as true of the data model and false of practice, since Git LFS locking and forge features implement it as policy at a single authoritative remote; the honest form is that locking cannot be enforced between arbitrary peers, not that it is unavailable anywhere. Several questions stay explicitly open: whether any in-

dependent, non-vendor source corroborates Perforce's claimed dominance in game-industry binary storage; what Git LFS file locking actually guarantees; how the offline-capability contrast holds under a real network partition rather than the simulated unreachable-repository test reproduced here; and whether per-path access control and full offline history are genuinely mutually exclusive or simply never co-implemented.

Referenced sources

- [Making Sense of Revision-Control Systems](#) — Bryan O'Sullivan / Communications of the ACM **HIGH**
- [Using Perforce as Source Control \(Unreal Engine\) — successor page](#) — Epic Games / Unreal Engine Documentation
- [Understanding Version-Control Systems \(DRAFT\)](#) — Eric S. Raymond / catb.org **MEDIUM**
- [RCS—A System for Version Control](#) — Walter F. Tichy / Department of Computer Sciences, Purdue University **HIGH**
- [Version Control with Subversion — Locking](#) — Collins-Sussman, Fitzpatrick & Pilato / svnbook.red-bean.com **HIGH**
- [Version Control with Subversion — Path-Based Authorization](#) — Collins-Sussman, Fitzpatrick & Pilato / svnbook.red-bean.com **HIGH**
- [Set protections with p4 protect](#) — Perforce Software / P4 Server Administration Documentation **HIGH**
- [How protections are implemented](#) — Perforce Software / P4 Server Administration Documentation (2025.1) **HIGH**
- [Exclusive File Locking](#) — Perforce Software / Perforce Knowledge Base, article 3114 **HIGH**
- [Using Perforce as Source Control \(Unreal Engine\)](#) — Epic Games / Unreal Engine Documentation **HIGH**
- [Plugin for Unreal — Unity Version Control](#) — Unity Technologies / Unity Documentation **MEDIUM**
- [BitKeeper for Kernel Developers](#) — Val Henson & Jeff Garzik / Proceedings of the Ottawa Linux Symposium (2002) **HIGH**
- [Interview with BitKeeper author Larry McVoy](#) — Jeremy Andrews (KernelTrap) / Linux.com **MEDIUM**
- [BitKeeper — Wikipedia](#) — Wikipedia contributors **MEDIUM**
- [BitKeeper goes open source](#) — LWN.net **HIGH**
- [About protected branches](#) — GitHub / GitHub Docs **HIGH**
- [Canonical References in Radicle: A Hands-On Guide](#) — Radicle Garden **MEDIUM**

- [The Essence of Git: Concepts for P2P Replication](#) — Erick Lavoie / arXiv:2609.13555 (2026) **MEDIUM**
- [What is version control: centralized vs. DVCS](#) — Giancarlo Lionetti / Atlassian **MEDIUM**
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Git — Wikipedia](#) — Wikipedia contributors **MEDIUM**

2.25 V25 — Centralisation relocated from the data model into policy

This note argues that centralised and distributed version control are usually contrasted as a deployment difference — one server versus many copies — when the sharper distinction is structural: a centralised system makes a single point of authority a fact of its data model, while a distributed system removes that fact but not the need for an authoritative copy, so authority returns as a convention that forges now implement as policy. It draws on Raymond's taxonomy to separate three independent variables (centralised/decentralised, locking/merge-before-commit/commit-before-merge, file/fileset-oriented) and traces the lineage from SCCS and RCS's exclusive-locking model through CVS's shift to merge-based collaboration and Subversion's deliberate re-introduction of locking for unmergeable binary content. It documents what only a structural single server could enforce — exclusive locking, per-path access control, and paying binary-history storage once rather than on every clone — and reproduces the offline-versus-connected asymmetry directly: a Subversion working copy holds only the pristine text of one revision, so status and diff work with the repository unreachable while log and blame fail. It then traces how Git, having removed centralisation from the data model, saw it return as policy at forges (GitHub branch protection) and, twenty years earlier, as BitKeeper's licence-mandated metadata server — evidence that BitKeeper's architecture was distributed and only its licence, not its design, was the constraint Git escaped.

The note is explicit about what it does not claim. It does not assert that distributed systems can never approximate locking in practice — only that they cannot enforce it between arbitrary peers talking directly to each other, and it flags Git LFS file locking as a real counter-example whose exact semantics it has not sourced. It treats Perforce's claimed dominance in AAA game development as reported rather than verified, since the only evidence found was vendor marketing and tertiary repetition rather than an independent survey. It also names its own scalar treatment of "authority" as a boundary that does not fit systems with legitimately partitioned per-subtree authority, such as a Perforce depot with delegated sub-protections. Several questions are left open: what Git LFS locking actually guarantees, how the offline contrast behaves under a real

network partition rather than a simulated one, whether any centralised system combines per-path access control with offline history, and whether Jujutsu's first-class conflicts change the locking calculus at all.

Referenced sources

- [Making Sense of Revision-Control Systems](#) — Bryan O'Sullivan / Communications of the ACM (2009-09-01) HIGH
- [Understanding Version-Control Systems \(DRAFT\)](#) — Eric S. Raymond / catb.org (undated draft) MEDIUM
- [RCS—A System for Version Control](#) — Walter F. Tichy / Department of Computer Sciences, Purdue University HIGH
- [Version Control with Subversion — Locking](#) — Collins-Sussman, Fitzpatrick & Pilato / svnbook.red-bean.com (1.7 edition) HIGH
- [Version Control with Subversion — Path-Based Authorization](#) — Collins-Sussman, Fitzpatrick & Pilato / svnbook.red-bean.com (1.7 edition) HIGH
- [Set protections with p4 protect](#) — Perforce Software / P4 Server Administration Documentation HIGH
- [How protections are implemented](#) — Perforce Software / P4 Server Administration Documentation (2025.1) HIGH
- [Exclusive File Locking](#) — Perforce Software / Perforce Knowledge Base, article 3114 (2022-04-10) HIGH
- [Using Perforce as Source Control \(Unreal Engine\)](#) — Epic Games / Unreal Engine Documentation HIGH
- [Plugin for Unreal — Unity Version Control](#) — Unity Technologies / Unity Documentation MEDIUM
- [BitKeeper for Kernel Developers](#) — Val Henson & Jeff Garzik / Proceedings of the Ottawa Linux Symposium (2002) HIGH
- [Interview with BitKeeper author Larry McVoy](#) — Jeremy Andrews (KernelTrap) / Linux.com (2002-05-28) MEDIUM
- [BitKeeper](#) — Wikipedia — Wikipedia contributors (fetched 2026-09-28) MEDIUM
- [BitKeeper goes open source](#) — LWN.net (2016-05-11) HIGH
- [About protected branches](#) — GitHub / GitHub Docs HIGH
- [Canonical References in Radicle: A Hands-On Guide](#) — Radicle Garden (2026-06-26) MEDIUM
- [The Essence of Git: Concepts for P2P Replication](#) — Erick Lavoie / arXiv:2609.13555 (2026) MEDIUM

- [What is version control: centralized vs. DVCS](#) — Giancarlo Lionetti / Atlassian (2012-02-14) **MEDIUM**
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [Git](#) — Wikipedia — Wikipedia contributors **MEDIUM**

2.26 V26 — Jujutsu as a redesign of Git's user model

This note treats Jujutsu (`jj`) as a controlled experiment: it keeps Git's storage layer — the content-addressed graph of blobs, trees and commits, with the commit ID identical to the Git commit ID when the Git backend is used — and replaces everything else. Three changes carry the redesign. The working copy is itself a commit, snapshotted automatically at the start of almost every command, which is what eliminates the index as a user-facing concept rather than merely hiding it. Every mutating command is recorded as an operation in its own append-only DAG, so the repository's state history is itself version-controlled and `jj undo` is an ordinary traversal of that log; the note documents that this log was built to enable lock-free concurrency, and that undo is a byproduct of that mechanism rather than the reason for it. Each commit also carries a change ID, a second identifier stable across amend and rebase, which lets a change be referred to consistently even as its content hash changes underneath it — a generalization of a problem Gerrit's Change-Id trailer solved at the tooling layer since 2009. None of this requires a new storage primitive: change IDs and conflicted trees are carried inside Git's own commit format via non-standard headers and placeholder directories, and the operation log lives entirely outside Git in Jujutsu's own store. Because the substitution happens strictly above an unmodified object graph, the note treats what survives (the Merkle DAG) as essential to Git and what does not (index, stash, per-branch working state, repository locking) as consequences of Git's interface rather than of its data model.

The note is explicit about what it does not claim. It does not argue that Jujutsu is a finished or recommended tool: as of the version checked (v0.45.1, 2026-09-03) the project describes itself as experimental, with an unstable on-disk format and an unstable command surface, even though its Git-compatibility layer is considered stable. It records unresolved tensions rather than papering over them — for instance, an independent commentator's claim that the auto-commit behavior can be "turned off" is checked against the project's own FAQ and found to be true only for newly added files, not for edits to already-tracked ones. It leaves open questions it says no source answers: how often change-ID identity actually survives a realistic squash-and-force-push review cycle through a forge, and whether the operation log's concurrency guarantees have

any documented production use outside Google. It also declines to adjudicate whether the redesign is ultimately an improvement, citing an independent reviewer's reserved verdict that this "remains to be seen." The conflict-representation half of the Git/Jujutsu contrast is deliberately left to a sibling note and is not repeated here.

Referenced sources

- [Working copy — Jujutsu docs](#) **HIGH**
- [Operation log — Jujutsu docs](#) **HIGH**
- [Git compatibility — Jujutsu docs](#) **HIGH**
- [Glossary — Jujutsu docs](#) **HIGH**
- [Git comparison — Jujutsu docs](#) **HIGH**
- [FAQ — Jujutsu docs](#) **HIGH**
- [docs/FAQ.md at v0.45.1 — jj-vcs/jj](#) **HIGH**
- [README.md at v0.45.1 — jj-vcs/jj](#) **HIGH**
- [docs/config.md at v0.45.1 — jj-vcs/jj](#) **HIGH**
- [Releases — jj-vcs/jj](#) **HIGH**
- [Jujutsu: a new, Git-compatible version control system — Daroc Alden, LWN.net](#) **HIGH**
- [Jujutsu is great for the wrong reason](#) **MEDIUM**
- [Tech Notes: The Jujutsu version control system — Evan Martin](#) **MEDIUM**
- [Gerrit Code Review — Change-Ids](#) **HIGH**
- [How we built stacked PRs without a new git workflow — Mergify](#) **MEDIUM**
- [gitoxide](#)
- [pre-commit.com](#)

2.27 V27 — Jujutsu's redesign of Git's user model while keeping Git's storage

This note establishes that Jujutsu (`jj`) is a version control system that keeps Git's content-addressed storage entirely intact while replacing almost everything above it. Because Jujutsu's only production-ready backend is Git itself — its commit ID is literally the Git commit ID — the project functions as a controlled experiment that isolates what Git's object model requires from what Git's interface merely does. Three changes carry the redesign, each sourced to the project's own documentation and cross-checked against independent accounts: the working copy is itself a commit, snapshotted automatically at the start of nearly every command, which removes the index as a user-facing concept rather than replacing it; every mutating command is recorded as an oper-

ation in an append-only DAG, so the repository's own history is version-controlled and `jj undo`, `jj op revert`, and `jj op restore` are ordinary traversals of it — a mechanism the project states it built for lock-free concurrency, with undo as a side benefit; and each commit carries a change ID, a second identifier stable across amendment and rebase, generalizing a pattern Gerrit's Change-Id trailer established in 2009. The pedagogical point the note draws from this is that Git concepts usually taught as things Git simply has — the index, stash, and reset's three flavors — turn out to be consequences of a UI model layered on the object graph, not properties of the content-addressed graph itself.

The note is explicit about what it does not claim. It does not recommend adopting Jujutsu; it is grounded for teaching purposes only, as a counterfactual for students who already use Git by rote. It treats Jujutsu's own concepts — change IDs and the operation log — as having no native Git representation, surviving only through non-standard commit headers and placeholder directories whose fidelity degrades outside Jujutsu's own layer, and it catalogs real feature gaps (hooks, submodules, partial clones, Git LFS). The project itself is pre-1.0, experimental, and both its library API and CLI are explicitly unstable. The note also leaves several questions open rather than resolving them: whether Jujutsu's interface is ultimately an improvement over Git's, independent sources disagree and reserve judgment; whether its lock-free concurrency design has a documented production deployment outside Google is unaddressed; and how often change identity survives a realistic multi-developer round trip through a forge is unquantified.

Referenced sources

- [Working copy — Jujutsu docs](#) — The Jujutsu project **HIGH**
- [Operation log — Jujutsu docs](#) — The Jujutsu project **HIGH**
- [Git compatibility — Jujutsu docs](#) — The Jujutsu project **HIGH**
- [Glossary — Jujutsu docs](#) — The Jujutsu project **HIGH**
- [Git comparison — Jujutsu docs](#) — The Jujutsu project **HIGH**
- [FAQ — Jujutsu docs](#) — The Jujutsu project **HIGH**
- [docs/FAQ.md at v0.45.1 — jj-vcs/jj](#) — The Jujutsu project **HIGH**
- [README.md at v0.45.1 — jj-vcs/jj](#) — The Jujutsu project **HIGH**
- [docs/config.md at v0.45.1 — jj-vcs/jj](#) — The Jujutsu project **HIGH**
- [Releases — jj-vcs/jj](#) — The Jujutsu project **HIGH**
- [Jujutsu: a new, Git-compatible version control system](#) — Daroc Alden, LWN.net **HIGH**
- [Jujutsu is great for the wrong reason](#) — felesatra.moe **MEDIUM**
- [Tech Notes: The Jujutsu version control system](#) — Evan Martin **MEDIUM**

- [Gerrit Code Review — Change-Ids](#) — The Gerrit Code Review project HIGH
- [How we built stacked PRs without a new git workflow](#) — Mergify MEDIUM
- [gitoxide](#) — the Rust library named as Jujutsu's Git backend implementation
- [pre-commit.com](#) — referenced in the note's quotation of Jujutsu issue #405 on hook support

2.28 V28 — Patch-based version control's commutation model

This note lays out the rival to Git's snapshot model: version-control systems (Darcs, and later Pijul) in which the change itself, not the tree state, is the object with a durable identity, and the tree state is derived by applying a set of such changes. Once a change is first-class, the central operation becomes asking whether two changes commute — whether applying them in either order reaches the same state with each retaining "the same meaning" in its new position. Where commutation holds, order carries no information; where it fails, the pair is either dependent or in conflict, and in Darcs's own formalization those are literally the same undefined-commutation fact. Darcs separates a patch into identity, context, and effect, with identity alone invariant under commutation, and builds cherry-picking directly out of the commutation operator rather than treating it as a separate feature. Pijul keeps that algebra but reframes the merge as a categorical pushout — a universal, non-heuristic construction — and replaces Darcs's pairwise-commutation bookkeeping with a materialized graph of labelled line-vertices that acts as a cache of everything already applied. The payoff the note documents on both sides: a change carries the same identity wherever it is applied, so cherry-picking never fragments into content-identical but differently-hashed copies the way Git's rebuilt objects do, and merging can proceed by set difference over patch identities instead of re-diffing two endpoint trees.

The note is explicit about what remains unresolved. Darcs's own FAQ still answers "no" (as of 2012, unchanged through 2026) to whether its exponential-time merge blowup is fixed, and two first-hand practitioner accounts disagree about how often it bites in practice without either being shown wrong. Pijul's claimed complexity advantage over Darcs is asserted by its own author rather than independently benchmarked, and its atomic operations (line insert/delete, file add/delete/rename) are deliberately narrower than Darcs's arbitrary-patch generality — a trade made explicitly for the pushout construction to work. The note also records a live, unresolved dispute over whether patch-level dependency tracking has any value given that it cannot see semantic dependencies between patches, and it notes that the snapshot model retains an affordance patch systems lose: `git bisect` walks a total order of complete historical states, while Darcs's local, partially-ordered patch log does not reconstruct "the version

whoever wrote the bug was actually looking at." The note treats this purely as reference grounding for teaching Git's design by contrast, not as a recommendation to adopt either alternative tool, and flags that neither tool was actually run on the runner that produced it — every behavioral claim rests on project documentation and independent secondary description, not reproduction.

Referenced sources

- [Darcs — Theory/GaneshPatchAlgebra](#) — Ganesh Sittampalam / Darcs project wiki HIGH
- [An Algebra of Patches](#) — Ian Lynagh (2006) HIGH
- [A formalization of darcs patch theory using inverse semigroups](#) — Judah Jacobson, UCLA Computational and Applied Mathematics Report 09-83 (2009) HIGH
- [Snapshots or Patches? Does It Matter?](#) — Sinclair Target (2025-08-18) MEDIUM
- [Darcs — Frequently Asked Questions \(Performance\)](#) — Darcs project (answer dated 2012-08-02; page read 2026-09-28) HIGH
- [Hacker News comment 16807818](#) — "I haven't used Pijul, but I did use Darcs for several large production projects" — Anonymous commenter (2018-04-13) LOW
- [Hacker News comment 43061764](#) — on Darcs' exponential merge corner-case — woadwarrior01 (2025-02-15) LOW
- [A Categorical Theory of Patches](#) — Samuel Mimram and Cinzia Di Giusto, MFPS — Mathematical Foundations of Programming Semantics, ENTCS 298 (2013) 283–307 HIGH
- [Pijul — Model](#) — Pijul project MEDIUM
- [The Pijul manual — Theory](#) — Pijul project HIGH
- [xit — docs/patch.md](#) — xit project MEDIUM
- [Pijul for git users | Lobsters](#) — Lobsters discussion thread MEDIUM
- [Pijul — A Distributed Version Control System](#) — Pijul project MEDIUM
- [Hacker News comment 47736092](#) — "Speaking as a former Darcs user" — WolfeReader (2026-04-12) LOW
- [Theory of the Pijul Version Control System | Lobsters](#) — Pierre-Étienne Meunier (pmeunier), self-identified Pijul author, 2019-05-25 MEDIUM
- [Notes on Pijul](#) — toastal (2023-11-23) MEDIUM
- [crates.io API — pijul crate](#) — crates.io registry (queried 2026-09-28) HIGH
- [Hackage — darcs package](#) — Hackage registry (queried 2026-09-28) HIGH
- [hub.darcs.net — darcs/darcs-screened changes](#) — Darcs project repository hosting (read 2026-09-28) HIGH
- [Darcs — FrontPage](#) — Darcs project (read 2026-09-28) HIGH

2.29 V29 — The algebra of commuting changes in Darcs and Pijul

This note establishes patch-based version control as a distinct model from Git's snapshot approach, using Darcs and Pijul as its two implementations. Here the change itself, not the tree state, is the object with a durable identity, which makes it meaningful to ask whether two changes commute — whether applying them in either order reaches the same state, each keeping its own meaning. Where commutation holds, order carries no information; where it fails, the pair is either dependent or in conflict, and Darcs treats those as one relation with two faces. That substitution is what lets a patch keep the same identity everywhere it is applied, so cherry-picking a change and later merging the branch it came from does not manufacture an artificial conflict, unlike Git, where cherry-pick necessarily builds a new object with a new hash. The note also traces the model's cost: Darcs' original algorithm, testing commutation pairwise against history, can be exponential in the structure of a conflict, a defect its own FAQ still records as unfixed. Pijul keeps the same algebra — formalized independently of Pijul as a pushout in a category of files and patches — but replaces the pairwise testing with a materialized graph that caches what has already been applied, treated here as the actual repair, a change of data structure rather than of theory.

The note is explicit about its limits. It is reference grounding for a university course, not a recommendation to adopt either tool, and it does not repeat the Git object model or three-way-merge mechanics covered elsewhere. Several points are left open: no independent benchmark compares Pijul's performance against Git or Darcs on a real repository, so its claimed fix for Darcs' blowup is theoretically motivated but empirically unaudited; whether Darcs' exponential case is still reachable under current usage is untested, since the FAQ's answer dates to 2012; Pijul's community size cannot be measured from public data; and whether Jujutsu's conflict handling formally derives from Darcs' construction or only echoes it is unresolved. Neither tool was installed or run on the runner — Darcs because it is unpackaged there, Pijul because no Rust toolchain was available — so every behavioral claim about either tool, including the central cherry-pick contrast, rests on documentation and secondary accounts rather than direct reproduction.

Referenced sources

- [Darcs — Theory/GaneshPatchAlgebra](#), Ganesh Sittampalam / Darcs project wiki — **HIGH**
- [An Algebra of Patches](#), Ian Lynagh (2006) — **HIGH**
- [A formalization of darcs patch theory using inverse semigroups](#), Judah Jacobson, UCLA Computational and Applied Mathematics Report 09-83 (2009) — **HIGH**
- [Snapshots or Patches? Does It Matter?](#), Sinclair Target (2025-08-18) — **MEDIUM**

- [Darcs — Frequently Asked Questions \(Performance\)](#), Darcs project (answer dated 2012-08-02) — **HIGH**
- [Hacker News comment 16807818](#) — "I haven't used Pijul, but I did use Darcs for several large production projects", Anonymous commenter (2018-04-13) — **LOW**
- [Hacker News comment 43061764](#) — on Darcs' exponential merge corner-case, woad-warrior01 (2025-02-15) — **LOW**
- [A Categorical Theory of Patches](#), Samuel Mimram and Cinzia Di Giusto, MFPS — Mathematical Foundations of Programming Semantics, ENTCS 298 (2013) — **HIGH**
- [Pijul — Model](#), Pijul project — **MEDIUM**
- [The Pijul manual — Theory](#), Pijul project — **HIGH**
- [xit — docs/patch.md](#), xit project — **MEDIUM**
- [Pijul for git users | Lobsters](#), Lobsters discussion thread — **MEDIUM**
- [Pijul — A Distributed Version Control System](#), Pijul project (homepage) — **MEDIUM**
- [Hacker News comment 47736092](#) — "Speaking as a former Darcs user", WolfeReader (2026-04-12) — **LOW**
- [Theory of the Pijul Version Control System | Lobsters](#), Pierre-Étienne Meunier (2019-05-25) — **MEDIUM**
- [Notes on Pijul](#), toastal (2023-11-23) — **MEDIUM**
- [crates.io API — pijul crate](#), crates.io registry (queried 2026-09-28) — **HIGH**
- [Hackage — darcs package](#), Hackage registry (queried 2026-09-28) — **HIGH**
- [hub.darcs.net — darcs/darcs-screened changes](#), Darcs project repository hosting (read 2026-09-28) — **HIGH**
- [Darcs — FrontPage](#), Darcs project (read 2026-09-28) — **HIGH**

2.30 V30 — Patch theory, as implemented by Darcs and Pijul

This note describes a rival model of version control in which the change itself, not the snapshot, is the object with identity. Git stores snapshots and computes changes by diffing two of them; Darcs and Pijul instead store the change, give it a durable identity, and derive the tree by applying a set of changes. Once a change is first-class, two changes can be asked whether they commute — whether applying them in either order reaches the same state — and Darcs' own formalisation treats "a conflict" and "commutation is undefined here" as the same fact. The note grounds this in the projects' own technical documentation and in three independent formalisations of Darcs' patch algebra, one of them a peer-reviewed paper defining Pijul's merge as a category-theoretic pushout. Two consequences follow directly from identity: a cherry-picked change keeps the same identity wherever applied, so merging afterward recognises it rather

than re-deriving it by diffing trees, unlike Git's cherry-pick, which builds a new object; and because the model tracks a full dependency graph rather than two endpoint trees, it never needs to synthesise a common base, avoiding the ambiguity a three-way merge faces when no unique greatest lower bound exists.

The note is explicit that this is reference grounding for teaching, not a recommendation to adopt either tool. Darcs' own FAQ still concedes, as of 2012 and unchanged through 2026, that its representation can make merges exponentially slow in certain conflict topologies, and two first-hand practitioner accounts disagree about how often this bites in practice. Pijul's complexity claims — that its redesign fixes Darcs' blowup — are asymptotic arguments and project-stated design goals, not independently benchmarked results; the note names this an open gap. Neither tool was runner-reproduced: `darcs` is unavailable in the runner image and no Rust toolchain was present to build Pijul, so every behavioural claim, including the central cherry-pick contrast, rests on documentation and secondary description rather than execution. The note also flags unsettled questions it does not resolve: whether Jujutsu's conflict algebra is a formal derivation of Darcs' conflictors or only an inspiration, and whether patch theory's limited real-world adoption (versus its influence on tools like Jujutsu and xit) reflects an inherent barrier or a contingent one.

Referenced sources

- [Darcs — Theory/GaneshPatchAlgebra](#) — Ganesh Sittampalam / Darcs project wiki HIGH
- [An Algebra of Patches](#) — Ian Lynagh (2006) HIGH
- [A formalization of darcs patch theory using inverse semigroups](#) — Judah Jacobson, UCLA Computational and Applied Mathematics Report 09-83 (2009) HIGH
- [Snapshots or Patches? Does It Matter?](#) — Sinclair Target (2025-08-18) MEDIUM
- [Darcs — Frequently Asked Questions \(Performance\)](#) — Darcs project (answer dated 2012-08-02; page read 2026-09-28) HIGH
- [Hacker News comment 16807818](#) — "I haven't used Pijul, but I did use Darcs for several large production projects" — Anonymous commenter (2018-04-13) LOW
- [Hacker News comment 43061764](#) — on Darcs' exponential merge corner-case — woadwarrior01 (2025-02-15) LOW
- [A Categorical Theory of Patches](#) — Samuel Mimram and Cinzia Di Giusto, MFPS — Mathematical Foundations of Programming Semantics, ENTCS 298 (2013) 283–307 HIGH
- [Pijul — Model](#) — Pijul project MEDIUM
- [The Pijul manual — Theory](#) — Pijul project HIGH
- [xit — docs/patch.md](#) — xit project MEDIUM

- [Pijul for git users | Lobsters](#) — Lobsters discussion thread **MEDIUM**
- [Pijul — A Distributed Version Control System](#) — Pijul project **MEDIUM**
- [Hacker News comment 47736092 — "Speaking as a former Darcs user"](#) — WolfeReader (2026-04-12) **LOW**
- [Theory of the Pijul Version Control System | Lobsters](#) — Pierre-Étienne Meunier (pmeunier), self-identified Pijul author, 2019-05-25 **MEDIUM**
- [Notes on Pijul](#) — toastal (2023-11-23) **MEDIUM**
- [crates.io API — pijul crate](#) — crates.io registry (queried 2026-09-28) **HIGH**
- [Hackage — darcs package](#) — Hackage registry (queried 2026-09-28) **HIGH**
- [hub.darcs.net — darcs/darcs-screened changes](#) — Darcs project repository hosting (read 2026-09-28) **HIGH**
- [Darcs — FrontPage](#) — Darcs project (read 2026-09-28) **HIGH**

2.31 V31 — the Gitless critique of Git's conceptual design

This note examines research by Santiago Perez De Rosso and Daniel Jackson (MIT) arguing that Git's usability problems are not vague dissatisfaction but demonstrable defects under a specific theory of conceptual design. The theory defines a "concept" as something a user must understand to operate a system, each concept invented to serve a "motivating purpose"; an "operational misfit" occurs when a concept's prescribed behavior fails that purpose in some scenario. Applying five design criteria — Motivation, Coherence, Fulfillment, Non-division, and Decoupling — the authors' 2016 OOPSLA paper (which supersedes an earlier 2013 paper built on a different, rejected set of criteria) identifies seven operational misfits in Git, each traced to a specific concept and to the criterion it violates. For example, the `commit` concept violates Coherence because it serves two goals — making changes persistent and grouping logically related changes — that pull in opposite directions; the stash violates Motivation because its purpose connects to no genuine version-control need, which the note reads as evidence the stash patches a deeper flaw (branch/working-directory coupling) rather than serving a real requirement. The authors validated the misfits' reality against a large sample of highly-upvoted Stack Overflow questions, then tested a redesign — Gitless — in an 11-participant user study. Gitless eliminates the staging area and generalizes the branch concept (so each branch carries its own working state) to make the stash unnecessary, addressing four of the seven misfits, with net-positive but mixed usability results across six tasks.

The note is explicit about what this work does not establish. The authors themselves decline to claim Gitless is a "better" version control system — only that it better satisfies the purpose-mapping their method evaluates, under controlled, task-specific condi-

tions. Both papers restrict their analysis to local operations, excluding the entire remote/sync surface (origin versus upstream, `pull`'s rebase behavior) that the note flags as a major gap for a collaboration-focused course. The claim that Git's staging area is "inessential" is asserted rather than demonstrated with usage-frequency data, and the note records named practitioners disputing it on purpose grounds — arguing staging serves a logical-commit-grouping need the papers' own concept table already assigns it, so the live disagreement is over visibility, not existence of the purpose. The 11-participant user study's external validity is thin by the authors' own statement (all MIT-affiliated, none self-identified as Git novices, with the "novice" subgroup defined by task-completion clustering rather than prior experience). The note leaves several questions open rather than resolved: whether the Stack Overflow correlation was ever replicated on later data, whether Git's own maintainers considered these findings, why the Gitless project went dormant after 2019, and whether Gitless's branch semantics actually outperform Git's own 2026 autostash-on-checkout feature for real work. The note reports and stress-tests these claims; it does not argue the underlying design questions are settled.

Referenced sources

- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez De Rosso & Daniel Jackson, Onward! 2013 **HIGH**
- [Purposes, Concepts, Misfits, and a Redesign of Git](#) — Santiago Perez De Rosso & Daniel Jackson, OOPSLA 2016 **HIGH**
- [ACM Digital Library record — What's wrong with git?: a conceptual design analysis](#) — ACM / Onward! 2013 proceedings **HIGH**
- [OpenAlex work records for DOIs 10.1145/2983990.2984018 and 10.1145/2509578.2509584](#) — OpenAlex **HIGH**
- [Gitless](#) — MIT Software Design Group project page — MIT CSAIL Software Design Group **HIGH**
- [Gitless](#) — Santiago Perez De Rosso et al. **HIGH**
- [gitless-vcs/gitless](#) — repository, releases and commits — GitHub REST API via `gh api` **HIGH**
- [gitless commit ff36038e8](#) — "new switch command" — GitHub REST API **HIGH**
- [Git 2.23.0 release notes](#) — The Git project **HIGH**
- [git-switch\(1\) at tag v2.23.0](#) — The Git project **HIGH**
- [Git 2.51.0 release notes](#) — The Git project **HIGH**
- [git commit f609dc4f7](#) — "builtin: unmark git-switch and git-restore as experimental" — The Git project **HIGH**
- [Git 2.55.0 release notes](#) — The Git project **HIGH**

- [git commit c07039ebc — "checkout -m: autostash when switching branches"](#) — The Git project **HIGH**
- [git-checkout\(1\) on git.git master](#) — The Git project **HIGH**
- [Jujutsu documentation — Git comparison and Conflicts](#) — The Jujutsu project **HIGH**
- [jj-vcs/jj README](#) — The Jujutsu project **HIGH**
- [Purposes, concepts, misfits, and a redesign of Git — the morning paper](#) — Adrian Colyer **MEDIUM**
- [What's Wrong With Git? A Conceptual Design Analysis](#) — Michael J. Tsai **MEDIUM**
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Hacker News discussion — Hacker News, December 2013 **LOW**
- [Making it easier to collaborate on code](#) — MIT CSAIL news office **MEDIUM**

2.32 V32 — Git's two families of monorepo scaling fixes

This note argues that Git's object model — content-addressed blobs, trees, and whole-tree-snapshot commits — does not itself degrade with scale; what fails is the layered-on assumption that a client can cheaply hold, transfer, and walk the entire object graph on every operation. From that framing it argues that every production fix for Git at monorepo scale collapses into exactly two families. "Fetch less" (shallow clone, partial clone, sparse checkout) genuinely gives the client an incomplete repository and tells Git so. "Fake completeness" (Microsoft's VFS for Git, Google's Piper/CitC, Meta's Sapling/EdenFS) transfers even less but presents what looks like a full working tree, materialising files on demand through a virtual filesystem — the first over a real Git repository, the other two over non-Git systems built for the purpose. The note supports this with primary-source evidence: Git's own partial-clone design document, controlled clone-time measurements on `torvalds/linux` comparing full, shallow, blobless and treeless clones, and first-party accounts from Microsoft, Google and Meta of the systems each built. It treats commit-graph, multi-pack-index, and Git LFS as adjacent but distinct from either family: the first two are local lookup accelerators that change nothing about what crosses the network, and LFS addresses blob weight rather than repository structure.

The note is explicit that fetch-less fixes defer cost rather than remove it — a treeless clone costs the server more CPU than a full clone does, and later commands such as `git reset --hard` become markedly slower — and that partial clone trades Git's offline-completeness for a standing dependency on a remote server. It notes that sparse checkout remains formally experimental even in the Git version it cites, and that fake-completeness systems are largely unavailable outside the companies that built them: Eden-

FS and Meta's Mononoke server are not publicly supported, and Google's CitC has never been released at all. It flags one contested point — whether VFS for Git is "deprecated" in favor of `scalar` — as an artefact-dependent question it resolves by checking the live repository rather than repeating the popular claim. Several questions it leaves explicitly open: why Microsoft quietly removed a maintenance-mode notice from VFS for Git's README without public explanation, whether Meta has published timing figures comparable to Microsoft's Windows-repository numbers, and whether a third family of fix beyond the two it identifies might exist. It states plainly that the two-family taxonomy describes the fixes that were actually built, not a proof that no other approach is possible.

Referenced sources

- [Partial Clone Design Notes](#) — Git project **HIGH**
- [git-clone\(1\)](#) — Git project **HIGH**
- [git-sparse-checkout\(1\)](#) — Git project **HIGH**
- [Scalar — design document](#) — Git project **HIGH**
- [Git Commit-Graph Design Notes](#) — Git project **HIGH**
- [Multi-Pack-Index \(MIDX\) Design Notes](#) — Git project **HIGH**
- [scalar — command documentation](#) — Git project / git-scm.com **HIGH**
- [The Story of Scalar](#) — Derrick Stolee & Victoria Dye / GitHub Blog **HIGH**
- [Announcing GVFS \(Git Virtual File System\)](#) — Saeed Noursalehi / Microsoft Azure DevOps Blog **HIGH**
- [Git Virtual File System Architecture](#) — Microsoft Learn **HIGH**
- [The largest Git repo on the planet](#) — Brian Harry / Microsoft **HIGH**
- [microsoft/VFSForGit — Readme.md at master](#) — Microsoft **HIGH**
- [microsoft/VFSForGit — Readme.md at 4ee8c1c9](#) — Microsoft **HIGH**
- [Why Google Stores Billions of Lines of Code in a Single Repository](#) — Rachel Potvin & Josh Levenberg / Communications of the ACM **HIGH**
- [Sapling: Source control that's user-friendly and scalable](#) — Durham Goode / Meta Engineering **HIGH**
- [facebook/sapling — README.md](#) — Meta **HIGH**
- [EdenFS Overview](#) — Meta **HIGH**
- [Git support modes | Sapling](#) — Meta / sapling-scm.com **HIGH**
- [Git LFS Specification](#) — git-lfs project **HIGH**
- [git-lfs-faq\(7\)](#) — git-lfs project **HIGH**
- [Large Object Promisors](#) — Git project **HIGH**
- [Why You Shouldn't Use Git LFS](#) — Gregory Szorc **MEDIUM**

- [Large files with Git: LFS and git-annex](#) — Antoine Beaupré / LWN.net **HIGH**
- [Git clone: a data-driven study on cloning behaviors](#) — Solmaz Abbaspoursani / GitHub Blog **HIGH**
- [About Git Large File Storage](#) — GitHub Docs **HIGH**
- [You probably shouldn't bother with \(free tier\) Github LFS](#) — wlcx.cc **MEDIUM**
- [Beyond GVFS: more details on optimizing Git for large repositories](#) — Edward Thomson / Microsoft Azure DevOps Blog **HIGH**
- [GVFS Updates: More Performance, More Availability](#) — Edward Thomson / Microsoft Azure DevOps Blog **HIGH**
- [Supercharge your Git workflows](#) — Darwin Sanoy / GitLab **MEDIUM**

2.33 V33 — Trunk-based versus branch-based Git workflows

This note argues that the five named branching models (Gitflow, GitHub Flow, GitLab Flow, trunk-based development, Release Flow) are not five different philosophies but variations on one underlying variable: how long a change is allowed to sit outside the mainline before it merges. Branch topology is treated as mostly naming; branch lifetime is the variable that predicts cost. The case for short-lived branches draws on two independent lines of evidence: mining studies of real repositories showing merge-conflict probability and defect-proneness rising with branch duration and size, and survey-based delivery-performance research (DORA) associating elite performers with few active branches and at least daily merges. The note works through the mechanics each landing method (fast-forward, merge commit, squash, rebase) leaves on mainline history, the preconditions short-lived integration requires (fast CI, automated testing, feature flags, small batches), and platform features — protected branches and merge queues — that turn a convention into an enforced constraint. It documents the release-branch variant (Microsoft's Release Flow) as trunk-based and release-branched at once, and works through the contested question of whether Gitflow is obsolete, concluding the actual dispute is narrower than it appears: nobody defends long-lived feature branches, only long-lived release branches for specific cases.

The note is explicit about what this evidence does not establish. DORA's findings are self-reported and correlational, not causal, and its benchmark methodology has changed across report years. The headline conflict-probability lift from the mining study is qualified by a small effect size, with change size found to be a stronger predictor than duration alone. The preconditions for short-lived integration are treated as non-optional: where CI is slow or test coverage thin, frequent merging is said to invert the expected outcome rather than merely reduce its benefit. Some contexts — multiple supported product versions, compliance regimes requiring documented review gates —

are said to require long-lived branches regardless of delivery-performance data. Feature flags are named as a replacement cost (runtime debt) rather than a free win, backed by measured flag-lifetime data. Several claims are flagged in the note itself as inference or unverified rather than sourced, including the "upstream first" policy commonly attributed to GitLab Flow and the causal link between monorepo structure and trunk-based development.

Referenced sources

- [A successful Git branching model](#) — Vincent Driessen / nvie.com HIGH
- [Patterns for Managing Source Code Branches](#) — Martin Fowler / martinowler.com HIGH
- [GitHub flow](#) — GitHub / GitHub Docs HIGH
- [What is GitLab Flow?](#) — GitLab / about.gitlab.com MEDIUM
- [Trunk Based Development — Introduction](#) — Paul Hammant / trunkbaseddevelopment.com HIGH
- [Short-Lived Feature Branches](#) — Paul Hammant / trunkbaseddevelopment.com HIGH
- [DORA — Capabilities: Trunk-based development](#) — DORA (DevOps Research and Assessment) / dora.dev HIGH
- [How Microsoft develops with DevOps](#) — Microsoft / Microsoft Learn HIGH
- [About protected branches](#) — GitHub / GitHub Docs HIGH
- [Managing a merge queue](#) — GitHub / GitHub Docs HIGH
- [Merge trains](#) — GitLab / GitLab Docs HIGH
- [Attributes that may raise the occurrence of merge conflicts](#) — Menezes, Trindade, Pimentel, Plastino, Murta & Costa / Journal of Software Engineering Research and Development 9(1) (2021) HIGH
- [An empirical investigation into merge conflicts and their effect on software quality](#) — Brindescu, Ahmed, Jensen & Sarma / Empirical Software Engineering (2020) HIGH
- [Why Google Stores Billions of Lines of Code in a Single Repository](#) — Rachel Potvin & Josh Levenberg / Communications of the ACM (2016) HIGH
- [Pro Git — Basic Branching and Merging](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [About pull request merges](#) — GitHub / GitHub Docs HIGH
- [Merge strategies and squash merge](#) — Microsoft / Microsoft Learn HIGH
- [Git Branching Strategies in 2026: GitHub Flow, GitFlow, GitLab Flow, Trunk-Based & Release Flow](#) — DeployHQ (2026) MEDIUM

- [Git-Flow, GitHub-Flow, Gitlab-Flow and Trunk Based Development explained](#) — Steven Giesel / [steven-giesel.com](#) (2022-07-23) **MEDIUM**
- [Choosing a Git workflow: a decision guide for real teams](#) — [gitflow.dev](#) (2026-06-07) **LOW**
- [Capture the Feature Flag: Detecting Feature Flags in Open-Source](#) — Jens Meinicke, Juan Hoyos, Bogdan Vasilescu, Christian Kästner / MSR 2020 **HIGH**
- [Feature Toggle Dynamics in Large-Scale Systems: Prevalence, Growth, Lifespan, and Benchmarking](#) — Xhevahire Tërnavë / arXiv preprint 2604.15872 (2026) **MEDIUM**

2.34 V34 — Branch lifetime as the variable behind branching-strategy trade-offs

This note argues that the named branching strategies — Gitflow, GitHub Flow, GitLab Flow, trunk-based development, and Microsoft's Release Flow — differ mainly along one variable, how long a change is allowed to sit outside the mainline before it integrates, rather than in the topology diagrams usually used to present them. It grounds this in two independent empirical lines: a mining study of 182,273 merge scenarios across 80 projects finding that conflict probability rises with the duration of the branch being integrated, and a separate study of 36,122 merge commits finding that lines involved in a conflict resolution are roughly twice as likely to be defect-prone, with semantic conflicts around twenty-six times as likely. Survey-based delivery-performance research separately associates higher throughput and stability with teams that keep few active branches and merge to trunk at least daily. The note treats the two lines as complementary rather than decisive, since one measures conflicts rather than outcomes and the other is self-reported and correlational, but notes they fail in different ways. It also works through each landing mechanism — fast-forward, merge commit, squash, and rebase — and how each affects bisecting and revert granularity, and uses Microsoft's Release Flow as a worked example of a model that is simultaneously trunk-based and release-branched.

The note is explicit that it is reference grounding for a course deck rather than teaching material itself, and it repeatedly separates what a source actually states from folklore built on top of it — for instance, pointing out that GitHub's current documentation for GitHub Flow no longer asserts that the mainline must always be deployable, even though that belief is widely attached to the model. It flags a limit in its own strongest empirical claim: branch duration has only a small effect size in the cited conflict study, and the number of changed files, commits, and lines predicts conflicts more strongly than elapsed time does, so a duration figure on its own overstates what the study found.

It does not claim short-lived branches are correct in every context: it names conditions – supporting multiple released product versions simultaneously, and compliance regimes requiring documented separation-of-duties review – under which long-lived release branches remain necessary regardless of delivery-performance data. Several claims are marked as the author's own inference rather than something an external source states outright, including the claim that no cited position defends long-lived feature branches, and one attribution – an "upstream first" policy commonly credited to GitLab Flow – is flagged as unverified against GitLab's own current documentation.

Referenced sources

- [A successful Git branching model](#) – Vincent Driessen / nvie.com **HIGH**
- [Patterns for Managing Source Code Branches](#) – Martin Fowler **HIGH**
- [GitHub flow](#) – GitHub Docs **HIGH**
- [What is GitLab Flow?](#) – GitLab **MEDIUM**
- [Trunk Based Development – Introduction](#) – Paul Hammant / trunkbaseddevelopment.com **HIGH**
- [Short-Lived Feature Branches](#) – Paul Hammant / trunkbaseddevelopment.com **HIGH**
- [DORA – Capabilities: Trunk-based development](#) – DORA (DevOps Research and Assessment) **HIGH**
- [How Microsoft develops with DevOps](#) – Microsoft Learn **HIGH**
- [About protected branches](#) – GitHub Docs **HIGH**
- [Managing a merge queue](#) – GitHub Docs **HIGH**
- [Merge trains](#) – GitLab Docs **HIGH**
- [Attributes that may raise the occurrence of merge conflicts](#) – Menezes, Trindade, Pimentel, Plastino, Murta & Costa / JSERD 9(1), 2021 **HIGH**
- [An empirical investigation into merge conflicts and their effect on software quality](#) – Brindescu, Ahmed, Jensen & Sarma / Empirical Software Engineering, 2020 **HIGH**
- [Why Google Stores Billions of Lines of Code in a Single Repository](#) – Rachel Potvin & Josh Levenberg / Communications of the ACM **HIGH**
- [Pro Git – Basic Branching and Merging](#) – Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [About pull request merges](#) – GitHub Docs **HIGH**
- [Merge strategies and squash merge](#) – Microsoft Learn **HIGH**
- [Git Branching Strategies in 2026: GitHub Flow, GitFlow, GitLab Flow, Trunk-Based & Release Flow](#) – DeployHQ **MEDIUM**

- [Git-Flow, GitHub-Flow, Gitlab-Flow and Trunk Based Development explained](#) — Steven Giesel **MEDIUM**
- [Choosing a Git workflow: a decision guide for real teams](#) — gitflow.dev **LOW**
- [Capture the Feature Flag: Detecting Feature Flags in Open-Source](#) — Jens Meinicke, Juan Hoyos, Bogdan Vasilescu & Christian Kästner / MSR 2020 **HIGH**
- [Feature Toggle Dynamics in Large-Scale Systems: Prevalence, Growth, Lifespan, and Benchmarking](#) — Xhevahire Tërnavë / arXiv preprint 2604.15872 **MEDIUM**
- dora.community — the redirect host the note records for the DORA capability page above, as of the note's 2026-09-27 access date; the note assigns no separate credibility grade to this host.
- docs.gitlab.com/topics/gitlab_flow/ — GitLab's product-documentation page for GitLab Flow, which the note records as unreachable (a 302 to an authentication endpoint) on 2026-09-27 while trying to verify an "upstream first" attribution; no credibility grade given.
- cacm.acm.org — one of two hosts the note records as returning a 403 to automated fetch when it tried to reach the full Communications of the ACM text behind the Google monorepo source above; no credibility grade given.
- dl.acm.org — the other host the note records as returning the same 403 for that text; no credibility grade given.

2.35 V35 — Trunk-Based versus Branch-Based Development

"Branching Strategies — Trunk-Based vs Branch-Based Development" argues that the five commonly taught branching models (Gitflow, GitHub Flow, GitLab Flow, trunk-based development, Microsoft's Release Flow) differ mainly along one variable — how long a change is allowed to live outside the mainline — and that nearly every documented cost and benefit follows from that variable rather than from branch topology. It grounds this in two independent empirical threads: a mining study of 182,273 merge scenarios across 80 projects that finds conflict probability rising with branch duration, and a study of 36,122 merge commits finding that lines involved in a conflict resolution are roughly twice as likely to be defect-prone, with semantic conflicts about 26 times more so. Survey-based DORA research adds that teams with few active branches and at least daily merges show higher delivery and stability. The note works through the mechanics that make short-lived integration viable — fast CI, automated testing, feature flags, branch-by-abbreviation for larger changes — and the concrete history-shape consequences of each Git merge mechanism (fast-forward, merge commit, squash, rebase). It uses Microsoft's Release Flow, Google's monorepo, Netflix's post-incident adoption, and a Bloom & Wild before/after figure as worked examples of the practice at scale.

The note is explicit about what this evidence does not establish. It flags DORA's methodology as self-reported and correlational, noting that nothing in the dataset shows trunk-based development causes elite performance rather than accompanying the testing investment that makes it survivable. It qualifies its own headline conflict statistic — a 161% rise beyond 30 days — by reporting that the underlying study found only a small effect size for duration, with change size a stronger predictor. It treats several claims as explicitly unsettled: whether Gitflow is "obsolete" is presented as a live, two-sided dispute rather than resolved, since Gitflow's own author scoped rather than retracted his 2010 recommendation for explicitly versioned or multi-version products; the claim that monorepos favor trunk-based development because of atomic cross-project changes is marked as the note's own inference, not something any cited source states; and GitLab Flow's "upstream-first" hotfix convention is marked unverified because it does not appear on GitLab's current documentation. It also concedes that feature flags substitute runtime debt for merge debt, and that flags frequently outlive their intended removal date.

Referenced sources

- [A successful Git branching model](#) — Vincent Driessen / nvie.com **HIGH**
- [Patterns for Managing Source Code Branches](#) — Martin Fowler / martinowler.com **HIGH**
- [GitHub flow](#) — GitHub / GitHub Docs **HIGH**
- [What is GitLab Flow?](#) — GitLab / about.gitlab.com **MEDIUM**
- [Trunk Based Development — Introduction](#) — Paul Hammant / trunkbaseddevelopment.com **HIGH**
- [Short-Lived Feature Branches](#) — Paul Hammant / trunkbaseddevelopment.com **HIGH**
- [DORA — Capabilities: Trunk-based development](#) — DORA (DevOps Research and Assessment) / dora.dev **HIGH**
- [How Microsoft develops with DevOps](#) — Microsoft / Microsoft Learn **HIGH**
- [About protected branches](#) — GitHub / GitHub Docs **HIGH**
- [Managing a merge queue](#) — GitHub / GitHub Docs **HIGH**
- [Merge trains](#) — GitLab / GitLab Docs **HIGH**
- [Attributes that may raise the occurrence of merge conflicts](#) — José William Menezes, Bruno Trindade, João Felipe Pimentel, Alexandre Plastino, Leonardo Murta & Catarina Costa / Journal of Software Engineering Research and Development 9(1), 2021 **HIGH**

- [An empirical investigation into merge conflicts and their effect on software quality](#) — Caius Brindescu, Iftexhar Ahmed, Carlos Jensen & Anita Sarma / Empirical Software Engineering, 2020 **HIGH**
- [Why Google Stores Billions of Lines of Code in a Single Repository](#) — Rachel Potvin & Josh Levenberg / Communications of the ACM, 2016 **HIGH**
- [Pro Git — Basic Branching and Merging](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [About pull request merges](#) — GitHub / GitHub Docs **HIGH**
- [Merge strategies and squash merge](#) — Microsoft / Microsoft Learn **HIGH**
- [Git Branching Strategies in 2026: GitHub Flow, GitFlow, GitLab Flow, Trunk-Based & Release Flow](#) — DeployHQ **MEDIUM**
- [Git-Flow, GitHub-Flow, Gitlab-Flow and Trunk Based Development explained](#) — Steven Giesel / steven-giesel.com **MEDIUM**
- [Choosing a Git workflow: a decision guide for real teams](#) — gitflow.dev **LOW**
- [Capture the Feature Flag: Detecting Feature Flags in Open-Source](#) — Jens Meinicke, Juan Hoyos, Bogdan Vasilescu, Christian Kästner / MSR 2020 **HIGH**
- [Feature Toggle Dynamics in Large-Scale Systems: Prevalence, Growth, Lifespan, and Benchmarking](#) — Xhevahire Tërnavaj / arXiv preprint 2604.15872, 2026 **MEDIUM**

2.36 V36 — Branching Strategies, Trunk-Based versus Branch-Based Development

This note argues that the five named branching models students encounter (Gitflow, GitHub Flow, GitLab Flow, trunk-based development, and Microsoft's Release Flow) differ mainly along one variable — how long a change is allowed to sit outside the mainline — rather than being five distinct philosophies, following Martin Fowler's framing of "Integration Frequency" as the organizing spectrum. It backs the case for short-lived branches with two independent empirical lines: mining studies finding that merge-conflict probability and defect-proneness both rise with branch duration and divergence, and DORA's survey-based delivery-performance research finding that elite performers keep few active branches and merge at least daily. It works through the mechanics that make this variable operative — that branch topology and integration frequency are separable (a four-hour pull-request branch is trunk-based; a four-week one is feature branching in disguise), that a "healthy" trunk is a testing property enforced by feature flags and branch-by-abstraction rather than a branching property, and that the three landing mechanisms (fast-forward, merge commit, squash, rebase) trade off differently against `git bisect`, revert granularity, and history shape. It documents worked examples at scale (Microsoft's Release Flow, Google's monorepo, Netflix's post-

incident adoption) and treats the Gitflow-obsolescence question as genuinely contested but narrower than usually presented: nobody defends long-lived feature branches, and the live dispute is only about long-lived release branches for versioned or multi-version products.

The note is explicit about the limits of its own evidence. It flags that DORA's methodology is self-reported and correlational, so nothing in the dataset establishes that trunk-based development causes elite performance rather than accompanying the CI and test investment that makes it survivable. It qualifies its own headline conflict-probability figures by noting the underlying study found only a small effect size for branch duration specifically, with change size a stronger predictor than time — meaning the practice worth teaching is smaller batches, not shorter calendars for their own sake. It names contexts where its own recommendation does not apply: products supporting multiple released versions simultaneously, and compliance regimes that mandate documented review gates. It marks several of its own claims explicitly unresolved — that a monorepo's atomicity property favors trunk-based development, that the feature-versus-release-branch framing of the Gitflow debate is correct, and that GitLab Flow requires an "upstream first" hotfix discipline — labeling each as an inference or unverified claim rather than presenting it as sourced fact. It does not evaluate whether the five models it surveys are good or bad in any given organization's specific context, treating branching strategy instead as a diagnostic for CI and testing maturity.

Referenced sources

- [A successful Git branching model](#) — Vincent Driessen / nvie.com (2010, with a 2020-03-05 note of reflection) HIGH
- [Patterns for Managing Source Code Branches](#) — Martin Fowler / martinowler.com (2020-05-28) HIGH
- [GitHub flow](#) — GitHub / GitHub Docs HIGH
- [What is GitLab Flow?](#) — GitLab / about.gitlab.com MEDIUM
- [Trunk Based Development — Introduction](#) — Paul Hammant / trunkbaseddevelopment.com HIGH
- [Short-Lived Feature Branches](#) — Paul Hammant / trunkbaseddevelopment.com HIGH
- [DORA — Capabilities: Trunk-based development](#) — DORA (DevOps Research and Assessment) / dora.dev HIGH
- [How Microsoft develops with DevOps](#) — Microsoft / Microsoft Learn HIGH
- [About protected branches](#) — GitHub / GitHub Docs HIGH
- [Managing a merge queue](#) — GitHub / GitHub Docs HIGH
- [Merge trains](#) — GitLab / GitLab Docs HIGH

- [Attributes that may raise the occurrence of merge conflicts](#) — José William Menezes, Bruno Trindade, João Felipe Pimentel, Alexandre Plastino, Leonardo Murta & Catarina Costa / Journal of Software Engineering Research and Development 9(1) (2021) **HIGH**
- [An empirical investigation into merge conflicts and their effect on software quality](#) — Caius Brindescu, Iftekhar Ahmed, Carlos Jensen & Anita Sarma / Empirical Software Engineering (2020) **HIGH**
- [Why Google Stores Billions of Lines of Code in a Single Repository](#) — Rachel Potvin & Josh Levenberg / Communications of the ACM (2016) **HIGH**
- [Pro Git — Basic Branching and Merging](#) — Scott Chacon & Ben Straub / git-scm.com **HIGH**
- [About pull request merges](#) — GitHub / GitHub Docs **HIGH**
- [Merge strategies and squash merge](#) — Microsoft / Microsoft Learn (2025-10-27) **HIGH**
- [Git Branching Strategies in 2026: GitHub Flow, GitFlow, GitLab Flow, Trunk-Based & Release Flow](#) — DeployHQ (2026) **MEDIUM**
- [Git-Flow, GitHub-Flow, Gitlab-Flow and Trunk Based Development explained](#) — Steven Giesel / steven-giesel.com (2022-07-23) **MEDIUM**
- [Choosing a Git workflow: a decision guide for real teams](#) — gitflow.dev (2026-06-07) **LOW**
- [Capture the Feature Flag: Detecting Feature Flags in Open-Source](#) — Jens Meinicke, Juan Hoyos, Bogdan Vasilescu, Christian Kästner / MSR 2020 **HIGH**
- [Feature Toggle Dynamics in Large-Scale Systems: Prevalence, Growth, Lifespan, and Benchmarking](#) — Xhevahire Tërnavia / arXiv preprint 2604.15872 (2026) **MEDIUM**

2.37 V37 — Trunk-based versus branch-based Git development strategies

This note treats the five commonly-taught branching models — Gitflow, GitHub Flow, GitLab Flow, trunk-based development, and Microsoft's Release Flow — not as five separate philosophies but as different settings of a single variable: how long a change is allowed to remain outside the mainline before it merges. It supports that framing from two independent directions. Mining studies of real repositories find that merge-conflict probability rises with branch duration and size (one study of 182,273 merge scenarios across 80 projects reports roughly 40% fewer conflicts under a one-day branch lifetime and over 150% more beyond thirty days), and a separate study of 143 open-source projects finds that lines touched by a conflict resolution are roughly twice as likely to be buggy, rising to about 26 times for entangled "semantic" conflicts. Survey-based deliv-

ery research (DORA) separately associates teams that keep three or fewer active branches and merge at least daily with higher throughput and stability. The note then works through the mechanics that make short-lived integration possible or difficult: the four Git-level landing mechanisms (fast-forward, merge commit, squash, rebase) and what each preserves on the mainline; platform features such as protected branches and merge queues/merge trains that turn a convention into an enforced constraint; and the precondition that makes any of it safe — fast, trustworthy CI plus feature flags standing in for the isolation that branches used to provide. It uses Microsoft's Release Flow as a documented case where trunk-based development and long-lived release branches coexist, cutting release branches from a shippable trunk and fixing forward (landing a hotfix on the mainline first, then cherry-picking it into the release branch) rather than the reverse.

The note is explicit about what it does not establish. The DORA evidence is self-reported and correlational, so it cannot show that adopting trunk-based development causes better delivery outcomes rather than accompanying the testing and CI investment that makes it survivable. The headline conflict-duration percentages are flagged as overstating the case: the same study reports only a small effect size for duration itself, with the number of changed files, commits, and lines being stronger predictors than elapsed time — meaning "keep branches short" is really a proxy for "keep changes small." It also names contexts where long-lived branches remain necessary regardless — supporting multiple released product versions simultaneously, or compliance regimes requiring documented review gates — and treats the "Gitflow is obsolete" debate as settled only for feature branches; the live disagreement concerns long-lived release branches specifically. Three further claims are marked as the author's own inference rather than sourced conclusions (the monorepo-atomicity argument for trunk-based development, the framing that only release-branch lifetime is contested, and the specific "feature-vs-release" partition of that debate), and GitLab Flow's "upstream first" hotfix policy is marked unverified, since it does not appear on GitLab's current documentation.

Referenced sources

- [A successful Git branching model](#) — Vincent Driessen / nvie.com **HIGH**
- [Patterns for Managing Source Code Branches](#) — Martin Fowler / martinfowler.com **HIGH**
- [GitHub flow](#) — GitHub Docs **HIGH**
- [What is GitLab Flow?](#) — GitLab **MEDIUM**
- [Trunk Based Development — Introduction](#) — Paul Hammant / trunkbaseddevelopment.com **HIGH**
- [Short-Lived Feature Branches](#) — Paul Hammant / trunkbaseddevelopment.com **HIGH**

- [DORA — Capabilities: Trunk-based development](#) — DORA / dora.dev **HIGH**
- [How Microsoft develops with DevOps](#) — Microsoft Learn **HIGH**
- [About protected branches](#) — GitHub Docs **HIGH**
- [Managing a merge queue](#) — GitHub Docs **HIGH**
- [Merge trains](#) — GitLab Docs **HIGH**
- [Attributes that may raise the occurrence of merge conflicts](#) — Menezes, Trindade, Pimentel, Plastino, Murta & Costa / JSERD 9(1), 2021 **HIGH**
- [An empirical investigation into merge conflicts and their effect on software quality](#) — Brindescu, Ahmed, Jensen & Sarma / Empirical Software Engineering, 2020 **HIGH**
- [Why Google Stores Billions of Lines of Code in a Single Repository](#) — Potvin & Levenberg / Communications of the ACM, 2016 **HIGH**
- [Pro Git — Basic Branching and Merging](#) — Chacon & Straub / git-scm.com **HIGH**
- [About pull request merges](#) — GitHub Docs **HIGH**
- [Merge strategies and squash merge](#) — Microsoft Learn **HIGH**
- [Git Branching Strategies in 2026: GitHub Flow, GitFlow, GitLab Flow, Trunk-Based & Release Flow](#) — DeployHQ **MEDIUM**
- [Git-Flow, GitHub-Flow, Gitlab-Flow and Trunk Based Development explained](#) — Steven Giesel **MEDIUM**
- [Choosing a Git workflow: a decision guide for real teams](#) — gitflow.dev **LOW**
- [Capture the Feature Flag: Detecting Feature Flags in Open-Source](#) — Meinicke, Hoyos, Vasilescu, Kästner / MSR 2020 **HIGH**
- [Feature Toggle Dynamics in Large-Scale Systems: Prevalence, Growth, Lifespan, and Benchmarking](#) — Xhevahire Tërnavaj / arXiv preprint 2604.15872, 2026 **MEDIUM**

2.38 V38 — Code review and the pull request as a review mechanism

This note argues that code review and the pull request are a social convention layered on top of Git, not a Git concept: Git supplies commits and refs, not approvals, reviewers, or a notion of a change being "ready." It traces four historical answers to what the reviewable unit is — the emailed patch series Linux has used since 1991, the Gerrit change keyed to a single commit (2008), the Phabricator diff (2007), and the GitHub/GitLab pull request, a branch of arbitrary size (2008 onward) — and argues the choice of unit explains nearly everything downstream, including why stacked pull requests exist as a workaround to recover granularity the commit-based systems already had. Its evidentiary core is three findings: a Microsoft study (Bacchelli and Bird) found that although developers rank "finding defects" as review's top purpose, only 14% of actual re-

view comments concerned defects, while knowledge transfer and alternative solutions dominated; the same study found comprehension, not diligence, is the binding constraint; and a Cisco/SmartBear study of 2,500 reviews covering 3.2 million lines found defect-detection collapses past roughly 400 lines or 90 minutes, with no defects found beyond that point across three styles studied. A Google case study of roughly nine million changes supplies corroborating figures — a one-reviewer median, sub-four-hour review latency — and the note concludes review is a bounded-capacity comprehension activity whose main product is shared understanding, not a defect-finding gate.

The note does not argue review should be abandoned, only that its measured value (comprehension, knowledge distribution) diverges from its stated purpose (catching defects), explaining failure modes such as rubber-stamp review and review delay used as organisational leverage. It defers the Git object model and branch/merge mechanics to two companion notes rather than re-deriving them, and marks some reasoning explicitly as inference — for instance, that a review queue needs a work-in-progress limit because wait time grows non-linearly near capacity. It treats review's knowledge-transfer function as possibly eroding under AI-assisted development, not settled, and treats GitHub's native Stacked PRs feature as similarly unsettled: at writing time it was a private preview, not generally available. It declines to resolve whether asynchronous review or synchronous pairing and mobbing is the better default, presenting that as context-dependent, and flags one of its own sources — the Cisco/SmartBear study — as carrying a vendor interest that qualifies, without discrediting, otherwise rigorous case-study data.

Referenced sources

- [Expectations, Outcomes, and Challenges of Modern Code Review](#), Alberto Bacchelli and Christian Bird, ICSE 2013 — **HIGH**
- [Modern Code Review: A Case Study at Google](#), Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko and Alberto Bacchelli, ICSE-SEIP 2018 — **HIGH**
- [About Gerrit Code Review](#), Gerrit Code Review project — **HIGH**
- [Gerrit Code Review — Changes](#), Gerrit Code Review project documentation — **HIGH**
- [Best Kept Secrets of Peer Code Review](#), Jason Cohen, Steven Teleki and Eric Brown; SmartBear Software — **HIGH** for the case-study data, **MEDIUM** as a vendor publication
- [A Brief History of the Pull Request](#), rdnlsmith (2023-11-19) — **MEDIUM**
- [git-format-patch Documentation](#), Git project — **HIGH**
- [Phabricator](#), Wikipedia — **MEDIUM**

- [GitHub invokes spirit of Phabricator with preview of Stacked PRs](#), Tim Anderson, The Register (2026-04-14) — MEDIUM
- [Stacked Diffs Versus Pull Requests](#), Jackson Gabbard (2018-09-29) — MEDIUM
- [About code owners](#), GitHub Docs — HIGH
- [Code Owners](#), GitLab Docs — HIGH
- [Stacked Diffs \(and why you should know about them\)](#), Gergely Orosz, The Pragmatic Engineer (2023-10-17) — MEDIUM
- [In Praise of Stacked PRs](#), Ben Congdon (2022-07-17) — MEDIUM
- [Stacked PRs: A Better Way to Review Code](#), Travis DePrato, Aviator (2025-03-30) — MEDIUM
- [Stacked Pull Requests](#), Matt Rickard (2022-07-18) — LOW
- [The Standard of Code Review](#), Google engineering practices documentation — HIGH
- [What to Look For In a Code Review](#), Google engineering practices documentation — HIGH
- [Small CLs](#), Google engineering practices documentation — HIGH
- [Conventional Comments](#), Conventional Comments — MEDIUM
- [Speed of Code Reviews](#), Google engineering practices documentation — HIGH
- [From Async Code Reviews to Co-Creation Patterns](#), Dragan Stepanović, InfoQ (2022-11-08) — MEDIUM
- [Trunk Based Development: Stop PR Reviews!](#), Daniel Truempfer (2023-08-22) — LOW
- [Mob Programming — key issues it can solve, benefits, and learnings](#), Futurice — MEDIUM
- [On Pair Programming](#), Birgitta Böckeler and Nina Siessegger, martinowler.com — MEDIUM
- [Code reviews? What about pair programming, and trunk-based development?](#), Curtis Einsmann, Medium (2022-09-12) — LOW

2.39 V39 — Code review and the pull request as a review mechanism

This note argues that code review as most developers encounter it — a pull request on GitHub or a merge request on GitLab — is a social convention layered on top of Git, not a feature of Git itself, and that the pull request is only the fourth of four historical answers to the question of what the reviewable unit should be: the emailed patch series on the Linux kernel mailing list, the single commit keyed by a Change-Id in Gerrit, the

stacked single-commit diff in Phabricator, and finally the branch-sized GitHub pull request, which won on accessibility but lost the granularity the earlier systems had natively — which is why stacked-PR workflows, CODEOWNERS routing and explicit comment-severity labels exist as compensating tooling. Centrally, drawing on a Microsoft study that hand-classified 570 review comments (only 14% concerned defects, versus 29% code-improvement suggestions), a Cisco study of 2,500 reviews (no defects found after 90 minutes, performance dropping past roughly 200-400 lines), and a Google study of about 9 million changes (median one reviewer, 24 lines, under four hours latency), the note concludes that review is a bounded-capacity comprehension activity whose dominant output is shared understanding and knowledge transfer rather than defect-finding, and that review latency behaves as a queue: slow review pressures authors to enlarge pending changes rather than wait, which degrades comprehensibility and lengthens the queue further.

The note is explicit about what it leaves to companion notes and does not re-derive: Git's object model and the parents-never-children commit rule, and the branch/merge mechanics of trunk-based versus branch-based development. It does not claim code review is an effective bug-finding tool — it argues the opposite, that treating it as a defect gate is a category error — and it flags that review's knowledge-transfer benefit may already be eroding where AI, rather than a human peer, authors most of the code. Several of its own claims are marked explicitly as inference or as the author's confident-but-unsourced extrapolation beyond the cited studies, rather than as directly evidenced findings, and GitHub's native "Stacked PRs" feature is treated as an unproven private preview, not a shipped capability. Whether synchronous pairing or mobbing should replace asynchronous review altogether is presented as an open, unresolved tension, with the note settling only on a hybrid default rather than a resolution.

Referenced sources

- [Expectations, Outcomes, and Challenges of Modern Code Review](#) — Alberto Bacchelli and Christian Bird, ICSE 2013 — **HIGH**
- [Modern Code Review: A Case Study at Google](#) — Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko and Alberto Bacchelli, ICSE-SEIP 2018 — **HIGH**
- [About Gerrit Code Review](#) — Gerrit Code Review project — **HIGH**
- [Gerrit Code Review — Changes](#) — Gerrit Code Review project documentation — **HIGH**
- [Best Kept Secrets of Peer Code Review](#) — Jason Cohen, Steven Teleki and Eric Brown; SmartBear Software — **HIGH** for the case-study data, **MEDIUM** as a vendor publication
- [A Brief History of the Pull Request](#) — rdnlsmith (2023-11-19) — **MEDIUM**
- [git-format-patch Documentation](#) — Git project — **HIGH**

- [Phabricator](#) — Wikipedia — **MEDIUM**
- [GitHub invokes spirit of Phabricator with preview of Stacked PRs](#) — Tim Anderson, The Register (2026-04-14) — **MEDIUM**
- [Stacked Diffs Versus Pull Requests](#) — Jackson Gabbard (2018-09-29) — **MEDIUM**
- [About code owners](#) — GitHub Docs — **HIGH**
- [Code Owners](#) — GitLab Docs — **HIGH**
- [Stacked Diffs \(and why you should know about them\)](#) — Gergely Orosz, The Pragmatic Engineer (2023-10-17) — **MEDIUM**
- [In Praise of Stacked PRs](#) — Ben Congdon (2022-07-17) — **MEDIUM**
- [Stacked PRs: A Better Way to Review Code](#) — Travis DePrato, Aviator (2025-03-30) — **MEDIUM**
- [Stacked Pull Requests](#) — Matt Rickard (2022-07-18) — **LOW**
- [The Standard of Code Review](#) — Google engineering practices documentation — **HIGH**
- [What to Look For In a Code Review](#) — Google engineering practices documentation — **HIGH**
- [Small CLs](#) — Google engineering practices documentation — **HIGH**
- [Conventional Comments](#) — Conventional Comments — **MEDIUM**
- [Speed of Code Reviews](#) — Google engineering practices documentation — **HIGH**
- [From Async Code Reviews to Co-Creation Patterns](#) — Dragan Stepanović, InfoQ (2022-11-08) — **MEDIUM**
- [Trunk Based Development: Stop PR Reviews!](#) — Daniel Truemper (2023-08-22) — **LOW**
- [Mob Programming — key issues it can solve, benefits, and learnings](#) — Futurice — **MEDIUM**
- [On Pair Programming](#) — Birgitta Böckeler and Nina Siessegger, martinowler.com — **MEDIUM**
- [Code reviews? What about pair programming, and trunk-based development?](#) — Curtis Einsmann, Medium (2022-09-12) — **LOW**

2.40 V40 — Code Review and the Pull Request as a Review Mechanism

This note treats code review as a social convention layered on top of Git, not a Git feature, and traces the reviewable unit through four generations: the emailed patch series on the Linux kernel mailing list, Gerrit's one-commit-per-change model keyed by a `Change-Id`, Phabricator's stacked diffs, and the GitHub/GitLab pull request, whose unit

is an unconstrained branch. It grounds that history in two empirical studies — Bacchelli and Bird's study of Microsoft's CodeFlow tool and Sadowski et al.'s case study of Google's Critique — to argue that review's stated purpose (finding defects) and its measured output diverge: only about 14% of review comments concern defects, while the dominant output is knowledge transfer, team awareness, and alternative solutions, because review is fundamentally bounded by comprehension cost rather than diligence. A third study, the Cisco MeetingPlace case study, supplies the hard size and duration ceilings (200–400 lines, 60–90 minutes) behind "keep changes small." From there the note builds out mechanisms that follow from this synthesis: CODEOWNERS-based reviewer routing, the stacked-pull-request workaround for the PR's branch-sized review unit (and why squash-merging breaks a stack), author/reviewer responsibilities, comment-severity labelling conventions, and review latency as a queueing phenomenon rather than a fixed delay.

The note is explicit about what it does not claim. It is framed as reference grounding for a future course deck rather than the deck itself, and it deliberately does not re-derive Git's object model or its branch-and-merge machinery, both of which live in companion notes it only references. It treats GitHub's native "Stacked PRs" feature as arriving rather than arrived, since at the time of writing it is a private preview, not generally available. It flags one of its own primary sources, the Cisco/SmartBear study, as high-credibility for its case-study data but only medium-credibility as a vendor publication whose authors sell review tooling. And it leaves two questions it names but does not resolve: whether review's knowledge-transfer function survives as AI writes more of the code, and the broader question of agentic PR volume against fixed human review capacity, which it marks explicitly as a dangling forward-reference for a future note rather than settling here.

Referenced sources

- [Expectations, Outcomes, and Challenges of Modern Code Review](#) — Alberto Bacchelli and Christian Bird **HIGH**
- [Modern Code Review: A Case Study at Google](#) — Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko and Alberto Bacchelli **HIGH**
- [About Gerrit Code Review](#) — Gerrit Code Review project **HIGH**
- [Gerrit Code Review — Changes](#) — Gerrit Code Review project documentation **HIGH**
- [Best Kept Secrets of Peer Code Review](#) — Jason Cohen, Steven Teleki and Eric Brown; SmartBear Software **HIGH** for the case-study data, **MEDIUM** as a vendor publication
- [A Brief History of the Pull Request](#) — rdnlsmith **MEDIUM**
- [git-format-patch Documentation](#) — Git project **HIGH**

- [Phabricator](#) — Wikipedia **MEDIUM**
- [GitHub invokes spirit of Phabricator with preview of Stacked PRs](#) — Tim Anderson, The Register **MEDIUM**
- [Stacked Diffs Versus Pull Requests](#) — Jackson Gabbard **MEDIUM**
- [About code owners](#) — GitHub Docs **HIGH**
- [Code Owners](#) — GitLab Docs **HIGH**
- [Stacked Diffs \(and why you should know about them\)](#) — Gergely Orosz, The Pragmatic Engineer **MEDIUM**
- [In Praise of Stacked PRs](#) — Ben Congdon **MEDIUM**
- [Stacked PRs: A Better Way to Review Code](#) — Travis DePrato, Aviator **MEDIUM**
- [Stacked Pull Requests](#) — Matt Rickard **LOW**
- [The Standard of Code Review](#) — Google engineering practices documentation **HIGH**
- [What to Look For In a Code Review](#) — Google engineering practices documentation **HIGH**
- [Small CLs](#) — Google engineering practices documentation **HIGH**
- [Conventional Comments](#) — Conventional Comments **MEDIUM**
- [Speed of Code Reviews](#) — Google engineering practices documentation **HIGH**
- [From Async Code Reviews to Co-Creation Patterns](#) — Dragan Stepanović, InfoQ **MEDIUM**
- [Trunk Based Development: Stop PR Reviews!](#) — Daniel Truemper **LOW**
- [Mob Programming](#) — key issues it can solve, benefits, and learnings — Futurice **MEDIUM**
- [On Pair Programming](#) — Birgitta Böckeler and Nina Siessegger, martinofowler.com **MEDIUM**
- [Code reviews? What about pair programming, and trunk-based development?](#) — Curtis Einsmann, Medium **LOW**

2.41 V41 — Code Review as a Bounded Comprehension Activity, and the Pull Request's Place in Its History

This note argues that code review is a social convention layered on top of Git, not a feature of Git itself, and that the pull request is only one of four historical answers — alongside the emailed patch series, Gerrit's single commit keyed by a `Change-Id`, and Phabricator's stacked diff — to the question of what unit should be reviewed. It grounds that claim in three findings. A Microsoft study that hand-classified 570 review com-

ments found that finding defects was the top-stated motivation for review, yet "defect" was only the fourth most common comment category at 14%, behind code improvements, knowledge transfer, and team awareness; 91% of 873 surveyed programmers separately confirmed that unfamiliar files take longer to review, which the note reads as evidence that comprehension, not diligence, is the real bottleneck. A ten-month Cisco study of 2,500 reviews found defect detection collapses once a change exceeds roughly 200-400 lines or a review runs past 60-90 minutes, with no defect found past 90 minutes in any of three styles tested. From these the note concludes that review is a bounded-capacity comprehension activity whose dominant output is shared understanding rather than defect-catching, and traces how review latency — modeled as a queue, not a fixed delay — pressures developers to pile further commits onto a pending pull request rather than wait, making the change less comprehensible and the queue longer still.

The note is explicit about its limits. It does not re-derive Git's object model or its branch-and-merge machinery, deferring both to two companion notes it names directly. It treats the "200-400 lines / 60-90 minutes" figures and the "100-300 lines / 30-60 minutes" figures as a ceiling and a recommendation from the same source, not competing claims, correcting what it says was a prior internal citation error. Several points are flagged as interpretation rather than fact: that a work-in-progress limit is the right remedy for a growing review queue is marked "[inference]"; that review's knowledge-transfer role is the benefit most at risk as AI writes more code is marked "Confident" rather than certain. The note declines to adjudicate stacked-diff tooling against pairing and mobbing, instead naming a hybrid "pair by default, pull requests for the exceptional case" as the position worth teaching, and treats GitHub's native Stacked PRs as unsettled — "arriving rather than arrived" — since it remained in private preview at the time of writing.

Referenced sources

- [Expectations, Outcomes, and Challenges of Modern Code Review](#) — Alberto Bacchelli and Christian Bird, ICSE 2013 HIGH
- [Modern Code Review: A Case Study at Google](#) — Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko and Alberto Bacchelli, ICSE-SEIP 2018 HIGH
- [About Gerrit Code Review](#) — Gerrit Code Review project HIGH
- [Gerrit Code Review — Changes](#) — Gerrit Code Review project documentation HIGH
- [Best Kept Secrets of Peer Code Review](#) — Jason Cohen, Steven Teleki and Eric Brown, SmartBear Software HIGH MEDIUM
- [A Brief History of the Pull Request](#) — rdnlsmith (2023-11-19) MEDIUM
- [git-format-patch Documentation](#) — Git project HIGH

- [Phabricator](#) — Wikipedia **MEDIUM**
- [GitHub invokes spirit of Phabricator with preview of Stacked PRs](#) — Tim Anderson, The Register (2026-04-14) **MEDIUM**
- [Stacked Diffs Versus Pull Requests](#) — Jackson Gabbard (2018-09-29) **MEDIUM**
- [About code owners](#) — GitHub Docs **HIGH**
- [Code Owners](#) — GitLab Docs **HIGH**
- [Stacked Diffs \(and why you should know about them\)](#) — Gergely Orosz, The Pragmatic Engineer (2023-10-17) **MEDIUM**
- [In Praise of Stacked PRs](#) — Ben Congdon (2022-07-17) **MEDIUM**
- [Stacked PRs: A Better Way to Review Code](#) — Travis DePrato, Aviator (2025-03-30) **MEDIUM**
- [Stacked Pull Requests](#) — Matt Rickard (2022-07-18) **LOW**
- [The Standard of Code Review](#) — Google engineering practices documentation **HIGH**
- [What to Look For In a Code Review](#) — Google engineering practices documentation **HIGH**
- [Small CLs](#) — Google engineering practices documentation **HIGH**
- [Conventional Comments](#) — Conventional Comments **MEDIUM**
- [Speed of Code Reviews](#) — Google engineering practices documentation **HIGH**
- [From Async Code Reviews to Co-Creation Patterns](#) — Dragan Stepanović, InfoQ (2022-11-08) **MEDIUM**
- [Trunk Based Development: Stop PR Reviews!](#) — Daniel Truemper (2023-08-22) **LOW**
- [Mob Programming — key issues it can solve, benefits, and learnings](#) — Futurice **MEDIUM**
- [On Pair Programming](#) — Birgitta Böckeler and Nina Siessegger, martinowler.com **MEDIUM**
- [Code reviews? What about pair programming, and trunk-based development?](#) — Curtis Einsmann, Medium (2022-09-12) **LOW**

2.42 V42 — Stacked diffs as a code-review workflow

This note describes stacked diffs: decomposing one logical change into an ordered chain of small changes, each reviewed on its own, where change $n+1$ builds on the not-yet-merged change n . It establishes that the practice has two independent origins — Meta's Phabricator/Differential, which stored the dependency as explicit revision

metadata because the review object rather than a branch carried the change, and Google's Gerrit/Critique change-list model, which never used branch-per-review in the first place. The two lineages arrive at the same shape from opposite directions, and Meta's name for the practice, not Google's earlier use of it, is what escaped into wider industry use once Phabricator was open-sourced. The note grounds the mechanism in a structural fact about Git: a commit records its parents and nothing records its children, so Git cannot on its own know that a branch is stacked on another branch, and every stacking tool closes that gap by keeping a parent record outside the commit graph. It reproduces on a runner (Git 2.55.0, Linux AArch64) the two consequences that follow — a pull request's three-dot diff against trunk pulls in the layer below an unmerged branch, and a plain rebase without `--update-refs` silently orphans the branches above the one being rebased — and surveys four tools' restack mechanics (native Git's `--update-refs`, Graphite's `gt restack / gt track`, Meta's stack-native Sapling, and Jujutsu's automatically-moving bookmarks), plus GitHub's native stacked-pull-request feature that reached public preview on 2026-07-30.

The note explicitly does not argue that a team should or should not stack, and does not re-derive the object-model or rebase mechanics it cites from companion notes. It flags several open questions rather than settling them: no controlled study compares review effectiveness or defect detection across stack depths or against an equivalent single large pull request; Graphite's on-disk storage format for its branch-parent metadata is undocumented; adoption of `git rebase --update-refs` since 2022 is untracked; and whether forges other than GitHub have shipped native stacking is unestablished. It treats the dispute over whether stacking improves review quality or only review ergonomics as unresolved, noting that the vendor's defect-rate figures and the counter-argument that "bugs live in the seam between layers" measure different quantities and neither refutes the other. It also declines to treat the vault's separate evidence on small-batch delivery and trunk-based development as evidence for or against stacking specifically, since none of it measures stack-restack dynamics.

Referenced sources

- *git-rebase(1)* — `--update-refs`, Git project / Software Freedom Conservancy — <https://git-scm.com/docs/git-rebase> **HIGH**
- *Git 2.38.0 Release Notes*, Git project (2022) — <https://raw.githubusercontent.com/git/git/v2.38.0/Documentation/RelNotes/2.38.0.txt> **HIGH**
- *Stacked pull requests are now in public preview*, GitHub Changelog (2026-07-30) — <https://github.blog/changelog/2026-07-30-stacked-pull-requests-are-now-in-public-preview/> **HIGH**
- *Stacked pull requests (reference)*, GitHub Docs — <https://docs.github.com/en/pull-requests/reference/stacked-pull-requests> **HIGH**

- *Use other tools with stacked pull requests*, GitHub Docs — <https://docs.github.com/en/pull-requests/reference/use-other-tools-with-stacked-pull-requests> **HIGH**
- *Branches (pull request reference) — diff comparison methods*, GitHub Docs — <https://docs.github.com/en/pull-requests/reference/branches> **HIGH**
- *Restack branches*, Graphite documentation — <https://graphite.com/docs/restack-branches> **HIGH**
- *Track branches*, Graphite documentation — <https://graphite.com/docs/track-branches> **HIGH**
- *Command reference*, Graphite documentation — <https://graphite.com/docs/command-reference> **HIGH**
- *Graphite is joining Cursor*, Cursor / Anysphere (2025-12-19) — <https://cursor.com/blog/graphite> **HIGH**
- *Sapling: Source control that's user-friendly and scalable*, Meta Engineering (2022-11-15) — <https://engineering.fb.com/2022/11/15/open-source/sapling-source-control-scalable/> **HIGH**
- *Working with GitHub*, Jujutsu documentation — <https://docs.jj-vcs.dev/latest/github/> **HIGH**
- *Bookmarks*, Jujutsu documentation — <https://www.jj-vcs.dev/latest/bookmarks/> **HIGH**
- *Phabricator Project History*, Evan Priestley / Phacility — https://secure.phabricator.com/book/phabflavor/article/project_history/ **HIGH**
- *T5000: Using Differential with plain Git, without requiring Arc*, Phabricator issue tracker — <https://secure.phabricator.com/T5000> **HIGH**
- *Software Engineering at Google, Chapter 19: Critique — Google's Code Review Tool*, Winters, Manshreck & Wright (eds.), O'Reilly (2020) — <https://abseil.io/resources/swe-book/html/ch19.html> **HIGH**
- *Stacked Diffs (and why you should know about them)*, Gergely Orosz, The Pragmatic Engineer (2023-10-17) — <https://newsletter.pragmaticengineer.com/p/stacked-diffs> **MEDIUM**
- *GitHub Targets Large Merge Problem with Stacked PRs*, InfoQ (2026-04-29) — <https://www.infoq.com/news/2026/04/github-stacked-prs/> **MEDIUM**
- *Stacked Diffs Versus Pull Requests*, Jackson Gabbard (2018-09-29) — <https://jg.gg/2018/09/29/stacked-diffs-versus-pull-requests/> **MEDIUM**
- *Stacked PRs as Parallel Review*, Rishi Baldawa (2025-11-22) — <https://rishi.baldawa.com/posts/pr-throughput/stacked-prs-parallel-review/> **LOW**

- *GitHub Just Shipped Stacked PRs. They Fix the Wrong Half of Code Review*, Housseem Ben Slama, Software Letters (2026-05-30) — <https://www.softwareletters.com/p/sl-55-git-hub-just-shipped-stacked-prs-they-fix-the-wrong-half-of-code-review> **LOW**

2.43 V43 — Stacked diffs as a code-review workflow

This note explains stacked diffs — decomposing one logical change into an ordered chain of small, individually reviewed changes, each built on the not-yet-merged change below it — and treats the practice as a tooling problem, not a discipline problem. Its central claim is structural: Git records a commit's parents but never its children, so no Git object expresses "branch B depends on branch A"; every stacking tool therefore keeps that relationship in a store outside the commit graph, one that can disagree with the repository. The note traces two independent origins — Meta's Phabricator, which made an individual commit the review unit and stored dependencies as explicit metadata, and Google's Gerrit/Critique change-list model, which never used branch-per-review at all — and demonstrates, with commands run on a live machine, why branch-per-pull-request makes dependent work hard: a pull request's diff computes against the merge base, so an unmerged lower layer leaks into the layer above's diff, and a plain rebase moves only the current branch, silently orphaning the rest of the stack. It surveys how native Git, Graphite, Sapling, Jujutsu, and now GitHub's own public-preview stacked pull requests (from 2026-07-30) each close that gap.

The note is explicit that it is reference grounding for teaching and takes no position on whether a team should stack. It declines to treat the vault's existing merge-conflict, review-effectiveness, DORA, or small-batch evidence as support for stacking specifically, since none of it measures stack-restack dynamics. It states plainly that no controlled study compares review effectiveness or defect rates across stack depths, or against an equivalent single large pull request — every depth threshold it cites is self-described practitioner folklore, not measurement. It leaves open where Graphite stores its branch-parent metadata, how widely Git's native restack flag is actually used, whether any forge besides GitHub has shipped native stacking, and how stacking behaves under merge queues. On whether stacking improves review quality or only ergonomics, it presents both sides of a disputed claim without resolving it, noting that the defect/speed figures and the counter-argument about seam-level bugs measure different things and that neither refutes the other.

Referenced sources

- *git-rebase(1) — --update-refs*, Git project / Software Freedom Conservancy — <https://git-scm.com/docs/git-rebase> **HIGH**

- *Git 2.38.0 Release Notes*, Git project (2022) — <https://raw.githubusercontent.com/git/git/v2.38.0/Documentation/RelNotes/2.38.0.txt> **HIGH**
- *Stacked pull requests are now in public preview*, GitHub Changelog (2026-07-30) — <https://github.blog/changelog/2026-07-30-stacked-pull-requests-are-now-in-public-preview/> **HIGH**
- *Stacked pull requests (reference)*, GitHub Docs — <https://docs.github.com/en/pull-requests/reference/stacked-pull-requests> **HIGH**
- *Use other tools with stacked pull requests*, GitHub Docs — <https://docs.github.com/en/pull-requests/reference/use-other-tools-with-stacked-pull-requests> **HIGH**
- *Branches (pull request reference) — diff comparison methods*, GitHub Docs — <https://docs.github.com/en/pull-requests/reference/branches> **HIGH**
- *Restack branches*, Graphite documentation — <https://graphite.com/docs/restack-branches> **HIGH**
- *Track branches*, Graphite documentation — <https://graphite.com/docs/track-branches> **HIGH**
- *Command reference*, Graphite documentation — <https://graphite.com/docs/command-reference> **HIGH**
- *Graphite is joining Cursor*, Cursor / Anysphere (2025-12-19) — <https://cursor.com/blog/graphite> **HIGH**
- *Sapling: Source control that's user-friendly and scalable*, Meta Engineering (2022-11-15) — <https://engineering.fb.com/2022/11/15/open-source/sapling-source-control-scalable/> **HIGH**
- *Working with GitHub*, Jujutsu documentation — <https://docs.jj-vcs.dev/latest/github/> **HIGH**
- *Bookmarks*, Jujutsu documentation — <https://www.jj-vcs.dev/latest/bookmarks/> **HIGH**
- *Phabricator Project History*, Evan Priestley / Phacility — https://secure.phabricator.com/book/phabflavor/article/project_history/ **HIGH**
- *T5000: Using Differential with plain Git, without requiring Arc*, Phabricator issue tracker — <https://secure.phabricator.com/T5000> **HIGH**
- *Software Engineering at Google, Chapter 19: Critique — Google's Code Review Tool*, Winters, Manshreck & Wright (eds.), O'Reilly (2020), hosted at abseil.io — <https://abseil.io/resources/swe-book/html/ch19.html> **HIGH**
- *Stacked Diffs (and why you should know about them)*, Gergely Orosz, The Pragmatic Engineer (2023-10-17) — <https://newsletter.pragmaticengineer.com/p/stacked-diffs> **MEDIUM**

- *GitHub Targets Large Merge Problem with Stacked PRs*, InfoQ (2026-04-29) — <https://www.infoq.com/news/2026/04/github-stacked-prs/> MEDIUM
- *Stacked Diffs Versus Pull Requests*, Jackson Gabbard (2018-09-29) — <https://jg.gg/2018/09/29/stacked-diffs-versus-pull-requests/> MEDIUM
- *Stacked PRs as Parallel Review*, Rishi Baldawa (2025-11-22) — <https://rishi.baldawa.com/posts/pr-throughput/stacked-prs-parallel-review/> LOW
- *GitHub Just Shipped Stacked PRs. They Fix the Wrong Half of Code Review*, Housseem Ben Slama, Software Letters (2026-05-30) — <https://www.softwareletters.com/p/sl-55-github-just-shipped-stacked-prs-they-fix-the-wrong-half-of-code-review> LOW

2.44 V44 — Continuous Integration and Continuous Delivery

This note distinguishes three practices that are casually collapsed into the single term "CI/CD": continuous integration (developers merging into a shared mainline at least daily, verified by a self-testing automated build), continuous delivery (keeping an artefact releasable to production at any time — a capability), and continuous deployment (a policy under which every change that passes the pipeline is released automatically, with no human gate). It traces each practice to a distinct origin — continuous integration to Kent Beck's Extreme Programming in the 1990s, the deployment pipeline to a 2006 ThoughtWorks conference paper, and the canonical treatment to Humble and Farley's 2010 book — and argues the three form a dependency chain, deployment requiring delivery, which in turn requires integration. Drawing on Fowler's and Humble's own writing, it cites an operational three-question test for genuine CI (daily trunk pushes, a test run on every commit, builds fixed within roughly ten minutes) and reports that fewer than 20% of teams who believe they practice CI actually pass it. It documents the deployment pipeline's staged structure — a fast commit stage held to a five-to-ten-minute budget, followed by slower acceptance and release stages — the build-once-promote-the-artefact discipline, and two supporting techniques: feature flags to decouple deploying code from releasing a feature, and the expand/migrate/contract pattern for backward-compatible database schema changes. It backs these claims with large-scale empirical case studies of Facebook's web and mobile release processes and Google's test infrastructure, concluding that deployment frequency itself does not directly cause higher productivity or quality, but the automation that higher frequency forces does.

The note does not claim continuous deployment suits every team. It frames the choice between delivery (a human decides when to release) and deployment (the pipeline's pass is the release decision) as a business decision rather than a technical one, and it names cases — mobile app-store review, embedded and firmware hardware constraints, certified domains such as avionics (DO-178C) and medical devices (IEC 62304), and on-

premise or air-gapped distribution — where continuous deployment costs more than it is worth, while continuous integration itself remains largely unaffected. Several points are explicitly flagged as inference or unsettled rather than established fact: the "stop the line" phrase is identified as a Lean/Toyota borrowing rather than a verbatim CI/CD coinage; the IEC 62304/FDA compliance claims and the app-store review-latency figures are marked as reported, resting on a vendor guide and an agency blog respectively rather than a primary standard or first-party statement, and the note's own grounding-gap section names these as open items needing better sourcing. It also leaves an internal tension unresolved between two of its own sources on whether feature flags are a precondition of trunk-based development or only become necessary once a team practices continuous deployment, and it does not verify certain terminology — Humble and Farley's "fan-in/fan-out" pipeline vocabulary, and Kent Beck's own primary-source wording for daily mainline integration — against the original texts.

Referenced sources

- [Continuous Integration](#) — Martin Fowler HIGH
- [Continuous Delivery](#) — Martin Fowler HIGH
- [Deployment Pipeline](#) — Martin Fowler HIGH
- [Continuous Delivery: Anatomy of the Deployment Pipeline — The Commit Stage](#) — Jez Humble and David Farley / InformIT HIGH
- [Anatomy of the Deployment Pipeline](#) — ch. 5 excerpt, InformIT HIGH
- [Patterns — Continuous Delivery](#) — Jez Humble HIGH
- [Continuous Integration — Continuous Delivery](#) — Jez Humble HIGH
- [Feature Toggles \(aka Feature Flags\)](#) — Pete Hodgson HIGH
- [Parallel Change](#) — Danilo Sato / martinowler.com HIGH
- [Using the expand and contract pattern](#) — Prisma Data Guide MEDIUM
- [Blue Green Deployment](#) — Martin Fowler HIGH
- [Continuous Delivery \(book page and foreword\)](#) — Jez Humble and David Farley; foreword by Martin Fowler HIGH
- [CruiseControl](#) — Wikipedia MEDIUM
- [Continuous deployment | Technology Radar](#) — Thoughtworks HIGH
- [Automated deployment pipeline | Technology Radar](#) — Thoughtworks HIGH
- [Pipelines as code | Technology Radar](#) — Thoughtworks HIGH
- [Jenkins as a deployment pipeline | Technology Radar](#) — Thoughtworks HIGH
- [Continuous Deployment of Mobile Software at Facebook](#) — Rossi, Shibley, Su, Beck, Savor & Stumm, FSE 2016 HIGH

- [Continuous Deployment at Facebook and OANDA](#) — Savor, Douglas, Gentili, Williams, Beck & Stumm, ICSE 2016 **HIGH**
- [Rapid release at massive scale](#) — Chuck Rossi / Engineering at Meta **HIGH**
- [The State of Continuous Integration Testing @Google](#) — John Micco / Google, ICST 2017 keynote **HIGH**
- [Taming Google-Scale Continuous Testing](#) — Memon et al., ICSE-SEIP 2017 **HIGH**
- [Embedded DevOps: A Survey on the Application of DevOps Practices in Embedded Software and Firmware Development](#) — Parthiv Katapara and Anand Sharma, Nirma University **MEDIUM**
- [App Store Review Time for Mobile Apps in 2026](#) — Jesus Vargas / LOW/CODE **LOW**
- [Release Trains vs Continuous Deployment](#) — Hector Reyes Aleman / Medium **LOW**
- [Formal Verification and Compliance Release Control of Cloud Native Avionics OTA Pipeline Based on GitOps](#) — SPRC Open / FAPM **LOW**
- [IEC 62304 Medical Software Compliance Guide](#) — Jama Software **LOW**

2.45 V45 — Continuous Integration and Continuous Delivery

This note argues that "CI/CD" compresses three distinct, historically separate practices into one initialism, and that the conflation is why a student can run a pipeline daily and still not say what would have to change for their team to deploy on demand. Continuous integration (Kent Beck, Extreme Programming, 1990s) is a property of how a team uses version control — merging to trunk at least daily behind a self-testing build; continuous delivery is a property of the pipeline's artefact — any passing commit could go to production; continuous deployment is the policy that removes the human gate, so passing the pipeline is the release decision. Each depends on the one before it. The claims rest almost entirely on primary sources — Fowler's own articles and bliki entries, and Humble and Farley's book and companion site — quoting Fowler's definitions verbatim and Humble's three-question operational test for whether a team is actually doing CI, alongside his reported finding that fewer than 20% of teams who think they are doing CI can pass it. The note then works through the pipeline's mechanics (commit-stage time budget, build-once-and-promote, stop-the-line) and five "enablers" — trunk-based development, test automation, feature flags, backward-compatible database migration via expand/migrate/contract, and infrastructure as code — each illustrated with named organisations (Facebook web and mobile, Google's Test Automation Platform, OANDA) and worked examples, including a GitHub Actions pipeline and a column-splitting SQL migration.

The note does not claim CI/CD is universally appropriate. Fowler's own precondition — a committed, full-time team — excludes sparse-contributor open-source projects, and continuous deployment specifically is contrasted with release trains and versioned release branches for mobile app-store review queues, embedded hardware constraints, and regulated domains such as avionics (DO-178C) and medical devices (IEC 62304), where it argues the realistic pattern is continuous delivery with a human-authorised, evidenced release gate rather than full automation. It does not claim deployment frequency itself drives productivity or quality, citing Facebook's mobile study for the causally inverted reading instead. Several claims are flagged as inference or reported rather than verified: the "stop the line" phrase's Lean/Toyota origin, the IEC 62304 and DO-178C material (drawn from low-credibility sources not read in full, logged as open grounding gaps), and the Jenkins-as-pipeline Technology Radar verdict, which the note itself calls stale and advocacy-adjacent. It defers the DORA framework's own methodological limits to a sibling note rather than adjudicating them here.

Referenced sources

- [Continuous Integration](#) — Martin Fowler HIGH
- [Continuous Delivery](#) — Martin Fowler HIGH
- [Deployment Pipeline](#) — Martin Fowler HIGH
- [Continuous Delivery: Anatomy of the Deployment Pipeline — The Commit Stage](#) — Jez Humble and David Farley / InformIT HIGH
- [Anatomy of the Deployment Pipeline](#) — Jez Humble and David Farley / InformIT HIGH
- [Patterns — Continuous Delivery](#) — Jez Humble HIGH
- [Continuous Integration — Continuous Delivery](#) — Jez Humble HIGH
- [Feature Toggles \(aka Feature Flags\)](#) — Pete Hodgson / martinowler.com HIGH
- [Parallel Change](#) — Danilo Sato / martinowler.com HIGH
- [Using the expand and contract pattern](#) — Prisma Data Guide MEDIUM
- [Blue Green Deployment](#) — Martin Fowler HIGH
- [Continuous Delivery \(book page and foreword\)](#) — Jez Humble and David Farley; foreword by Martin Fowler HIGH
- [CruiseControl](#) — Wikipedia MEDIUM
- [Continuous deployment | Technology Radar](#) — Thoughtworks HIGH
- [Automated deployment pipeline | Technology Radar](#) — Thoughtworks HIGH
- [Pipelines as code | Technology Radar](#) — Thoughtworks HIGH
- [Jenkins as a deployment pipeline | Technology Radar](#) — Thoughtworks HIGH

- [Continuous Deployment of Mobile Software at Facebook](#) — Rossi, Shibley, Su, Beck, Savor & Stumm, FSE 2016 **HIGH**
- [Continuous Deployment at Facebook and OANDA](#) — Savor, Douglas, Gentili, Williams, Beck & Stumm, ICSE 2016 **HIGH**
- [Rapid release at massive scale](#) — Chuck Rossi / Engineering at Meta **HIGH**
- [The State of Continuous Integration Testing @Google](#) — John Micco / Google **HIGH**
- [Taming Google-Scale Continuous Testing](#) — Memon et al., ICSE-SEIP 2017 **HIGH**
- [Embedded DevOps: A Survey on the Application of DevOps Practices in Embedded Software and Firmware Development](#) — Parthiv Katapara and Anand Sharma, Nirma University **MEDIUM**
- [App Store Review Time for Mobile Apps in 2026](#) — Jesus Vargas / LOW/CODE **LOW**
- [Release Trains vs Continuous Deployment](#) — Hector Reyes Aleman / Medium **LOW**
- [Formal Verification and Compliance Release Control of Cloud Native Avionics OTA Pipeline Based on GitOps](#) — SPRC Open / FAPM **LOW**
- [IEC 62304 Medical Software Compliance Guide](#) — Jama Software **LOW**

2.46 V46 — DORA's five software delivery performance metrics

This note documents DORA Metrics, the empirically validated set of software delivery performance indicators maintained by Google Cloud's DevOps Research and Assessment (DORA) program, which grew out of Puppet's State of DevOps survey (2013) and was acquired by Google, per DORA's own announcement, in December 2018. As of 2024 the framework comprises five metrics split across two categories: Throughput — Lead Time for Changes, Deployment Frequency, and Failed Deployment Recovery Time (the renamed and, in 2024, reclassified former MTTR) — and Instability — Change Failure Rate and the newly introduced Deployment Rework Rate. The note's central claim, drawn from DORA's own research, is that speed and stability are not a tradeoff: top performers excel at both, and low performers lag on both. It traces the framework's evolution from four-tier performance benchmarks (2023–2024) to seven team archetypes derived from eight measures (2025), and covers a 2026 ROI report that models AI adoption's financial impact through a "J-Curve" of temporary productivity dip, a "verification tax," and an "instability tax," alongside an illustrative 39% first-year return on investment and roughly eight-month payback period.

The note is explicit about what the metrics do not capture: delivery performance says nothing about customer satisfaction, code quality, or technical debt, and the metrics can be gamed if treated as targets rather than diagnostics — including DORA's own warning against "tokenmaxxing," gamifying AI token consumption. The underlying

survey methodology is self-reported, with acknowledged biases and incomplete transparency on response rates and sampling. The 2025 shift to seven archetypes is presented as unsettled — the note records it as an open question whether the archetype labels remain stable year to year, since they emerge from annual cluster analysis that may reshuffle patterns. The 2026 ROI figures are explicitly framed by DORA itself as "high-uncertainty" illustrations meant to spark conversation rather than validated predictions, and the note cites an independent stress test showing the modeled result flips from roughly +39% to -36% depending on an unresolved assumption about how long the adoption dip lasts. The note also leaves open how well Rework Rate actually correlates with Change Failure Rate across different kinds of teams.

Referenced sources

- [DORA's software delivery performance metrics](#) — Nathen Harvey / DORA **HIGH**
- [DORA Quick Check](#)
- [A history of DORA's software delivery metrics](#) — Nathen Harvey / DORA **HIGH**
- [DORA 2025: Year in review](#) — DORA / Google Cloud **HIGH**
- [Announcing the 2025 DORA Report: State of AI-Assisted Software Development](#) — Google Cloud **HIGH**
- [The DORA 4 key metrics become 5](#) — Steve Fenton / CD Foundation **MEDIUM**
- [The Continual Evolution of DORA](#) — Abi Noda / DX Newsletter **MEDIUM**
- [Nicole Forsgren](#) — Wikipedia — Wikipedia **MEDIUM**
- [DORA Report 2024 - A Look at Throughput and Stability](#) — Rachel Stephens / Red-Monk **HIGH**
- [DORA Metrics: What Are They & How To Track Them?](#) — Cortex **MEDIUM**
- [DORA Metrics: A Full Guide to Elite Performance Engineering](#) — Multitudes **MEDIUM**
- [DORA metrics in the age of AI-driven delivery](#) — Future Processing **MEDIUM**
- [New DORA Report Claims Strong Engineering Foundations Drive AI Return on Investment](#) — Matt Saunders / InfoQ **HIGH**
- [ROI of AI-assisted Software Development report](#) — Google Cloud / DORA **HIGH**
- [Finding balance in the era of tokenmaxxing](#) — DORA / Google Cloud **HIGH**
- [Quick Check updates](#) — DORA / Google Cloud **HIGH**
- [How to measure the business value of generative AI](#) — Google Cloud **HIGH**
- [RDEL #141: How can engineering leaders calculate the return on their AI investments?](#) — Lizzie Matusov / Research-Driven Engineering Leadership **MEDIUM**
- [DORA ROI of AI: How to Stress-Test It Before Your CFO Sees It](#) — Faros AI **MEDIUM**
- [DORA Joins Google Cloud](#) — The DORA Team **HIGH**

- [Too Many DevEx Metrics, Too Little Guidance: DevEx Metrics Compass](#) — André N. Meyer, Patrick Meyer, Gail C. Murphy, Thomas Fritz / ACM Queue **HIGH**
- [Datadog](#)
- [Axify](#)
- [Swarmia](#)
- [Apache DevLake](#)

2.47 V47 — Deployment Frequency as a DORA metric

The note defines Deployment Frequency as one of DORA's core metrics: how often an organization ships code to production, historically classified into four tiers (Elite, High, Medium, Low) whose thresholds DORA re-derives each year from that year's survey cluster analysis rather than from fixed cutoffs — meaning a team with unchanged practices can shift tiers purely from demographic changes in respondents. It documents that the 2024 report found the Medium cluster with a lower change-failure rate than the High cluster, an ordering anomaly that, along with the growing use of AI coding tools, motivated DORA to retire the four-tier model in 2025 in favor of seven qualitative team archetypes across eight measures. The note's largest section traces how DORA's view of AI's effect on this metric changed: the 2024 report found AI adoption correlated with a throughput decline and a stability decline; the 2025 report reversed the throughput finding (positive relationship) while the negative relationship with stability persisted in both years. It also cites independent 2026 telemetry from Faros AI covering 22,000 developers, which measured deployments per week down 11.7% and code churn up 861% under high AI adoption — a result the note treats as coexisting with, rather than contradicting, DORA's self-reported throughput gains, since the two instruments measure different things (developer self-report versus system records).

The note is explicit about what remains unsettled. It flags that DORA never published a mapping between its 2024 and 2025 measurement instruments, so the magnitude of the throughput reversal cannot be verified against the 2024 units. It marks the batch-size mechanism behind AI-driven instability as DORA's own hypothesis rather than a measured finding, and it lists open questions the evidence does not yet answer: whether the modeled "J-Curve" adoption dip actually closes over time, and whether higher deployment frequency causally drives rework or both instead share a common cause such as growing batch size. The note consistently distinguishes verified claims, reported (secondary) claims, and its own inferences rather than presenting all of these at the same confidence level.

Referenced sources

- [DORA's software delivery performance metrics](#) — DORA / Google Cloud **HIGH**

- [A history of DORA's software delivery metrics](#) — DORA / Google Cloud **HIGH**
- [DORA Research: 2025 Overview](#) — DORA / Google Cloud **HIGH**
- [The 2024 DevOps Performance Clusters](#) — Steve Fenton / Octopus Deploy **MEDIUM**
- [Is Your AI Strategy Only as Good as Your Team Archetype?](#) — Steve Pereira / Visible Consulting **MEDIUM**
- [DORA 2025: Measuring Software Delivery After AI](#) — Rachel Stephens / RedMonk **HIGH**
- [DORA AI Capabilities Model](#) — DORA / Google Cloud **HIGH**
- [ROI of AI-assisted Software Development report](#) — DORA / Google Cloud **HIGH**
- [The Acceleration Whiplash](#) — AI Engineering Report 2026 — Faros AI **MEDIUM**
- [DORA Report 2024 - A Look at Throughput and Stability](#) — Rachel Stephens / RedMonk **HIGH**
- [Announcing the 2025 DORA Report](#) — Google Cloud / DORA **HIGH**
- [Google DORA: Software delivery caught up to AI coding tools](#) — Andrea Benito / TechTarget **MEDIUM**
- [Accelerate State of DevOps Report 2024](#) — DORA / Google Cloud **HIGH**
- [From adoption to impact: putting the DORA AI Capabilities Model to work](#) — Google Cloud / DORA **HIGH**
- [New DORA Report Claims Strong Engineering Foundations Drive AI Return on Investment](#) — Matt Saunders / InfoQ **MEDIUM**
- [DORA ROI of AI: How to Stress-Test It Before Your CFO Sees It](#) — Faros AI **MEDIUM**
- [How are developers using AI? Inside our 2025 DORA report](#) — Google **HIGH**
- [DORA Publications](#) — DORA / Google Cloud **HIGH**

2.48 V48 — The limitations of DORA metrics for measuring AI-assisted software development

This note argues that the DORA delivery metrics (Deployment Frequency, Lead Time for Changes, Change Failure Rate, Failed Deployment Recovery Time, and Deployment Rework Rate) reveal a persistent paradox when applied to AI-assisted development: AI speeds up individual tasks while organizational delivery stability degrades. Google's 2024 DORA Report found AI adoption negatively associated with both throughput and stability; the 2025 report reversed the throughput finding to positive while the negative relationship with stability persisted. The note's central, repeatedly corroborated thesis is that "AI doesn't fix a team; it amplifies what's already there" — organizations with mature DevOps practices convert AI gains into real delivery improvement, while fragmen-

ted ones accelerate technical debt and instability. This amplifier finding is independently reproduced, in similar or identical language, by Sonar's 2026 developer survey and Perforce's 2026 State of DevOps Report. The note also triangulates across independent measurement methods — DORA's self-report survey, CircleCI's telemetry from 28+ million CI workflows (showing main-branch success rates at a five-year low), and DX's organizational PR-throughput analysis of 400+ companies (a modest 7.76% median gain) — to argue that real productivity gains are real but far more modest than vendor claims, because the bottleneck has shifted from writing code to reviewing, verifying, and integrating it (the "verification tax"). It further documents measurable code-quality degradation (GitClear's 2026 study of 623 million commits) and traces the well-known METR finding that AI made experienced developers 19% slower on average, alongside a large gap between perceived and actual productivity impact.

The note is explicit about what remains unsettled rather than resolved. It flags the amplifier thesis itself as difficult to falsify — positive outcomes credit organizational foundation, negative outcomes blame it — and notes that "platform quality" is not independently defined from the outcomes it predicts. It downgrades the METR 19% slowdown figure to only partially verified after a 2026 Bayesian reanalysis showed the estimate's confidence interval spans both large speedups and slowdowns, leaving the direction more likely than the magnitude. It treats the 2024-to-2025 DORA throughput reversal as unable to serve as longitudinal evidence, both because DORA is a repeated cross-section rather than a panel study and because the two years used different effect-size units (percentage-per-adoption-increment versus standardized effects) without DORA foregrounding the change. It is agnostic on long-term (3–5 year) equilibrium effects, on whether instability is an inherent consequence of higher change velocity or specifically a function of applying AI to mature/legacy codebases (a competing, explicitly unproven conjecture from a 2026 cross-disciplinary review), and on whether the "J-curve" recovery trajectory DORA proposes actually holds as tooling generations change. The note treats several cited industry sources (Sonar, GitClear, Perforce) as commercially interested and grades them accordingly rather than treating their figures as neutral.

Referenced sources

- [Accelerate State of DevOps Report 2024](#) — Google DORA (2024) **HIGH**
- [Announcing the 2025 DORA Report](#) — Google Cloud (2025) **HIGH**
- [AI Is Amplifying Software Engineering Performance, Says the 2025 DORA Report](#) — InfoQ (2026) **HIGH**
- [DORA AI Capabilities Model](#) — Google DORA (2025) **HIGH**
- [Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity](#) — METR (2025) **HIGH**

- [How to measure AI developer productivity](#) — Lenny's Podcast with Nicole Forsgren (2025-10-19) **HIGH**
- [Software developers shift to AI code reviewers](#) — Computer Weekly (2026) **MEDIUM**
- [GenAI: Metrics of Value for Developers — Option Value and DORA](#) — IT Revolution (2025) **MEDIUM**
- [State of DevOps 2025: Review of the DORA Report](#) — Splunk (2025) **MEDIUM**
- [The Maintainability Gap: AI Code Quality in 2026](#) — GitClear (v2026.6.1, June 2026) **MEDIUM**
- [DORA 2025: AI is an amplifier, not a magic wand](#) — Riccardo Mestroni / Google Cloud Community (2025) **MEDIUM**
- [Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions](#) — Pearce, Ahmad, Tan, Dolan-Gavitt, Karri — NYU Tandon / IEEE S&P 2022 (2021) **HIGH**
- [Do Users Write More Insecure Code with AI Assistants?](#) — Perry, Srivastava, Kumar, Boneh — Stanford / ACM CCS 2023 (2022) **HIGH**
- [GitHub Copilot: Productivity Boost or DORA Metrics Disaster?](#) — InfoWorld (2025) **MEDIUM**
- [What are the seven team profiles of engineering delivery performance?](#) — Lizzie Matusov / RDEL Newsletter (2025) **MEDIUM**
- [A history of DORA's software delivery metrics](#) — Nathen Harvey / DORA (2026) **HIGH**
- [We are Changing our Developer Productivity Experiment Design](#) — METR (2026) **HIGH**
- [AI Coding Tools and Productivity: What the Controlled Evidence Shows](#) — DEV Community (2026) **LOW**
- [DORA Metrics: What the Research Actually Says](#) — ZB Mowrey (2025) **LOW**
- [The 2026 State of Software Delivery](#) — CircleCI, sponsored by Thoughtworks (2026-02-18) **HIGH**
- [Measuring the Self-Reported Impact of Early-2026 AI on Technical Worker Productivity](#) — Joel Becker / METR (2026-05-11) **HIGH**
- [The ROI of AI-assisted Software Development](#) — Google DORA (2026-04) **HIGH**
- [AI and engineering velocity: A longitudinal analysis](#) — Abi Noda & Brian Houck / DX (2026-04-28) **HIGH**
- [Sonar Data Reveals Critical "Verification Gap" in AI Coding](#) — Sonar / SonarSource (2026-01-08) **MEDIUM**
- [DORA Metrics in Practice 2026 \(Volume 0\)](#) — Stride Research / Newlight Solutions (2026-05-18) **MEDIUM**

- [Writing code versus shipping code: Productivity effects across generations of AI coding tools](#) — Demirer et al. / CEPR VoxEU (2026) **HIGH**
- [Devs doubt AI-written code, but don't always check it](#) — The Register (2026-01-09) **MEDIUM**
- [Code maintainability plummets in the AI coding era](#) — Bill Doerrfeld / LeadDev (2026-07-07) **MEDIUM**
- [Google DORA: Software delivery caught up to AI coding tools](#) — Andrea Benito / TechTarget (2025-09-23) **MEDIUM**
- [2025 Developer Survey — AI section](#) — Stack Overflow (published 2025-12-29) **HIGH**
- [Modelling variation in the METR Uplift Study](#) — LessWrong (2026-09-05) **LOW**
- [Vibe Coding: Practice, Performance, Productivity, and Risk — A State-of-the-Art Review](#) — Michels, Abu Ghazaleh, Lazzari, Kassem & Klein — arXiv:2608.20446 (2026-08-20) **HIGH**
- [Beyond Code Generation: Reliability, Verification, and Cost Economics in the Agentic Software Development Lifecycle](#) — Happy Bhati — arXiv:2609.04681 (2026-09-04) **MEDIUM**
- [Perforce 2026 State of DevOps Report Indicates Mature DevOps Practices Lead to AI Success](#) — Perforce Software (2026-02-24) **MEDIUM**
- [Balancing AI tensions: Moving from AI adoption to effective SDLC use](#) — Jessica Baolin & Nathen Harvey / DORA (2026-03-10) **HIGH**

2.49 V49 — The small batches principle in software delivery

The note is a reading-note summary of Thomas A. Limoncelli's 2016 ACM Queue article "The Small Batches Principle," which argues that software delivery teams should ship work in small, frequent increments rather than accumulating changes into large releases. It traces the principle's origin to lean manufacturing and the Toyota Production System developed by Taiichi Ohno and Eiji Toyoda, where small-batch production emerged as an economic necessity in postwar Japan, and to Shigeo Shingo's SMED methodology, which made small-batch changeovers economically viable by cutting setup cost. Limoncelli's central illustration is a carpenter who inspects work after each cut rather than after finishing every board in a batch, so that a slipped blade is caught at the second piece rather than the tenth — the note treats this as the clearest statement of why smaller units of work bound the cost of any single defect. The note reports three benefits the article claims follow from small batches: reduced risk (easier testing, review, and rollback), reduced waste (shorter feedback loops prevent building features nobody wants), and higher customer satisfaction (fixes and improvements arrive as continuous flow). It also links the principle forward to the Minimum Viable Product

and to DORA's empirical research program, including a 2024 finding — attributed to DORA team lead Nathen Harvey — that AI-assisted coding correlates with reduced delivery throughput and stability because it lets developers generate larger batches of code, undermining the discipline Limoncelli described.

The note is explicit that Limoncelli's article itself supplies no primary empirical data: it argues from analogy and from reference to the lean manufacturing literature, and the empirical corroboration it now enjoys (DORA's surveys, the *Accelerate* book) arrived years after publication. It flags the carpenter analogy as assuming that defects are detectable at each step, which may not hold for software with complex interdependencies or failures that surface only under load. The note also records what the article does not address: how to decompose a large feature into releasable small batches, contexts where large batches may be structurally required (regulatory release windows, hardware coordination), and the possibility that many small changes create their own review and deployment toil. It flags the 2024 DORA finding on AI and batch inflation as a single-year result whose causal mechanism is plausible but unreplicated, with confounders — team growth, generated code volume, review bottlenecks — not ruled out. The note takes no further position on the article's arguments and does not claim the principle applies equally to every delivery context, noting that regulated or safety-critical software may be a nuanced exception.

Referenced sources

- The Small Batches Principle, Thomas A. Limoncelli, ACM Queue — <https://queue.acm.org/detail.cfm?id=2945077>
- Toyota Production System, Toyota — <https://global.toyota/en/company/vision-and-philosophy/production-system/>
- Toyota Production System (lexicon term), Lean Enterprise Institute — <https://www.lean.org/lexicon-terms/toyota-production-system/>
- Thomas Limoncelli speaker profile, USENIX LISA18 — <https://www.usenix.org/conference/lisa18/speaker-or-organizer/thomas-limoncelli-stack-overflow-inc>
- Working in Small Batches, DORA — <https://dora.dev/capabilities/working-in-small-batches/>
- The Small Batches Principle (publication record), ResearchGate — https://www.researchgate.net/publication/304529532_The_small_batches_principle
- Accelerate excerpt, IT Revolution — https://itrevolution.com/wp-content/uploads/2022/06/ACC_excerpt.pdf
- What Is an MVP?, Lean Startup Co. — <https://leanstartup.co/resources/articles/what-is-an-mvp/>

- DORA 2024: AI and Platform Engineering Fall Short, The New Stack — <https://the-newstack.io/dora-2024-ai-and-platform-engineering-fall-short/>
- The Phoenix Project: 10 Years of Transformation, IT Revolution — <https://itrevolution.com/articles/the-phoenix-project-10-years-of-transformation/>

2.50 V50 — Copy-on-write as a cross-scale sharing mechanism

This note's claim is that copy-on-write (CoW) is a single three-step protocol — share a physical unit between two or more logical owners, protect it so writes are interceptable, and copy only the unit a write actually touches — that recurs, unchanged in structure, across `fork()`'s anonymous pages, shared-library/DLL mappings, filesystem snapshots, database MVCC row versions, and container image layers, with only the unit of sharing and the arbiter that enforces protection varying between instances. The evidence is a mix of teaching-lab implementation material (Duke's and KU Leuven's copy-on-write fork exercises) for the fault-handler mechanics, vendor and kernel reference documentation (Linux `/proc`, `mmap(2)`, Windows memory-protection constants, ReFS, overlayfs) for how each instance is parameterised on each platform, and production engineering accounts (Redis, Instagram/Meta's CPython work, Docker) for the mechanism's real-world cost. Built on top of that unification, the note argues that the same protocol has a predictable failure mode it calls the "write storm": because the trap fires per unit on first write, a workload that only looks read-heavy — a GC mark phase, a refcount increment, a metadata `chmod`, a snapshot under sustained writes — silently becomes a write-heavy one, and the resulting cost (faults, TLB shutdowns felt by other cores, and copy volume inflated by whatever granularity the platform imposes, such as transparent huge pages) is invisible to any metric that counts bytes written rather than the spread of writes across units. It also treats Windows and Linux as substantively different on process creation (Windows has no supported user-mode `fork()`, so it gets neither the savings nor the storm from that path) while finding genuine parity between the two platforms on library sharing and filesystem snapshots.

The note is explicit that several claims are inference rather than sourced fact, and flags them under a dedicated "Grounding gap" section rather than citing a near-miss source: the arithmetic that a 512 MiB, stride-1 fork should produce on the order of 131,072 minor faults is presented as something to measure, not a verified figure, because transparent huge pages could invalidate it; the claim that Windows has no supported user-mode `fork()` is inferred from the absence of such an API in the Windows documentation surveyed and from CPython's Windows `multiprocessing` behaviour, not from a source stating the negative directly; the claim that no per-process Windows counter exists for TLB-shutdown activity analogous to Linux's `/proc/interrupts` is called explicitly unverified, since an absence found by searching is weak evidence; and the gener-

alisation that tracing garbage collectors on other runtimes (not just CPython) dirty pages by writing mark bits during collection is stated as unconfirmed against a primary JVM or V8 source. The note does not evaluate whether CoW is a good design choice in the abstract — it presents the trade-off (free reads, expensive divergence) as workload-dependent and leaves the reader to judge fit.

Referenced sources

- [Lab COW: Copy-on-write fork](#), Duke University, CompSci 310 (2025) **HIGH**
- [Exercise: Copy-on-Write Fork](#), DistriNet, KU Leuven — OS labs (n.d.) **HIGH**
- [The /proc Filesystem — The Linux Kernel documentation](#), The Linux Kernel Organization (2026) **HIGH**
- [Mysteries of Windows Memory Management Revealed](#), Russinovich & Solomon, Microsoft — conference session deck, re-hosted (n.d.) **MEDIUM**
- [What does "Private Data" define in VMMap?](#), Stack Overflow (2015) **MEDIUM**
- [Copy-on-Write, Explained Through fork\(\) and Snapshots](#), Pickuma (2026-06-10) **MEDIUM**
- [Diagnosing latency issues](#), Redis (2026) **HIGH**
- [Memory Protection Constants \(WinNT.h\)](#), Microsoft Learn (2026) **HIGH**
- [On-demand-fork: A Microsecond Fork for Memory-Intensive and Latency-Sensitive Applications](#), Zhao, Gong & Fonseca, EuroSys (2021) **HIGH**
- [Redis fork/COW memory storm: why persistence doubles memory](#), Netdata (n.d.) **MEDIUM**
- [mmap\(2\) — Linux manual page](#), man7.org / Linux man-pages project (2026) **HIGH**
- [Position Independent Code \(PIC\) in shared libraries](#), Eli Bendersky (2011) **HIGH**
- [How important is it nowadays to ensure that all my DLLs have non-conflicting base addresses?](#), Raymond Chen, Microsoft — The Old New Thing (2017) **HIGH**
- [A fine detail on how DLLs are relocated as the result of a base address collision, and consequences](#), Raymond Chen, Microsoft — The Old New Thing (2017) **HIGH**
- [The Basic Concepts of Snapper](#), SUSE, SUSE Linux Enterprise Server 16.0 documentation (2026) **HIGH**
- [LVM volume snapshots](#), SUSE, Storage Administration Guide, SLES 15 SP6 (2026) **HIGH**
- [Well-known Databases Use Different Approaches for MVCC](#), Amit Kapila / EnterpriseDB (n.d.) **MEDIUM**
- [Block cloning on ReFS](#), Microsoft Learn (2026) **HIGH**
- [Block Cloning — Win32 apps](#), Microsoft Learn (2026) **HIGH**

- [Volume Shadow Copy Service](#), MicrosoftDocs/windowsserverdocs repository (2026) **HIGH**
- [Latency Implications of Virtual Memory](#), Erik Rigtorp (n.d.) **HIGH**
- [Optimizing the TLB Shutdown Algorithm with Page Access Tracking](#), Nadav Amit, USENIX ATC (2017) **HIGH**
- [Overlay Filesystem — The Linux Kernel documentation](#), The Linux Kernel Organization (n.d.) **HIGH**
- [InternalDocs/garbage_collector.md — CPython](#), Python Software Foundation / CPython contributors (2026) **HIGH**
- [Issue 31558: gc.freeze\(\) — an API to mark objects as uncollectable](#), Python bug tracker / Łukasz Langa et al. (2017) **HIGH**
- [bpo-31558: Add gc.freeze\(\) — pull request discussion](#), CPython repository (2017) **HIGH**
- [PEP 683 — Immortal Objects, Using a Fixed Refcount](#), Python Software Foundation (2022) **HIGH**
- [Introducing Immortal Objects for Python](#), Eddie Elizondo, Engineering at Meta (2023-08-15) **HIGH**
- [Fixing Copy on Writes from reference counting and immortal objects](#), CPython issue tracker (2020) **HIGH**
- [mmap — Memory-mapped file support](#), Python Software Foundation (2026) **HIGH**
- [OverlayFS storage driver](#), Docker Inc. (2026) **HIGH**
- [CCoW: Optimizing Copy-on-Write Considering the Spatial Locality in Workloads](#), Electronics, MDPI (2022) **MEDIUM**
- [Copy-on-write](#), Wikipedia (2026) **MEDIUM**

2.51 V51 — Code review as a social, comprehension-bound process built on top of Git

This note establishes that code review — the practice of developers other than a change's author inspecting it before it merges — is a social convention invented on top of Git's data model, not a feature of Git itself, and that the "pull request" is only the most recent of four historical answers to the question of what unit gets reviewed. It traces the reviewable unit through four generations: the emailed patch series reviewed on the Linux Kernel Mailing List since 1991; Gerrit's one-commit-per-change model, keyed by a Change-Id, which gives dependent changes a native "Relation Chain"; Phabricator's stacked-diff workflow, where each commit lands independently onto master; and GitHub/GitLab's pull request, whose unit is an entire branch of arbitrary size. The note

grounds its central claims in three empirical studies: Bacchelli and Bird's mixed-method study at Microsoft, which found that only 14% of review comments concern defects while knowledge transfer and design improvement dominate; a Google case study of roughly nine million changes showing review at huge scale staying fast (median one reviewer, under four hours' median latency) through strict size and culture norms; and the Cisco/SmartBear study of 2,500 reviews, which measured hard ceilings on review effectiveness — no defects found past 90 minutes or past roughly 400 lines. Its synthesis is that review is a bounded-capacity comprehension task whose dominant output is shared understanding rather than defect-catching, and that review latency behaves as a queue, not a fixed delay, so it compounds as reviewer utilization rises.

The note is explicit that it is reference-grounding for a future course topic deck, not the deck itself, and deliberately does not re-derive material owned by two companion notes: Git's object model and the mechanics of branching, merging and branch protection. It does not claim that pull requests are an inferior design, only that they trade granularity for accessibility, and that practices like stacked PRs, CODEOWNERS and WIP-limited review gates exist to buy back what the branch-sized unit gives up. Several of its more far-reaching claims — the queueing argument for review latency, the comparative-advantage table of review units — are explicitly marked as the author's own inference drawn from cited premises, not direct findings of the cited sources. It also flags open, unsettled territory: whether code review's knowledge-transfer function survives as AI writes a growing share of the code is treated as an emerging concern, documented through a single study and a practitioner's stated expectation, rather than a settled conclusion.

Referenced sources

- [Expectations, Outcomes, and Challenges of Modern Code Review](#) — Alberto Bacchelli and Christian Bird **HIGH**
- [Modern Code Review: A Case Study at Google](#) — Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko and Alberto Bacchelli **HIGH**
- [About Gerrit Code Review](#) — Gerrit Code Review project **HIGH**
- [Gerrit Code Review — Changes](#) — Gerrit Code Review project documentation **HIGH**
- [Best Kept Secrets of Peer Code Review](#) — Jason Cohen, Steven Teleki and Eric Brown; SmartBear Software **HIGH** **MEDIUM**
- [A Brief History of the Pull Request](#) — rdnlsmith **MEDIUM**
- [git-format-patch Documentation](#) — Git project **HIGH**
- [Phabricator](#) — Wikipedia **MEDIUM**

- [GitHub invokes spirit of Phabricator with preview of Stacked PRs](#) — Tim Anderson, The Register **MEDIUM**
- [Stacked Diffs Versus Pull Requests](#) — Jackson Gabbard **MEDIUM**
- [About code owners](#) — GitHub Docs **HIGH**
- [Code Owners](#) — GitLab Docs **HIGH**
- [Stacked Diffs \(and why you should know about them\)](#) — Gergely Orosz, The Pragmatic Engineer **MEDIUM**
- [In Praise of Stacked PRs](#) — Ben Congdon **MEDIUM**
- [Stacked PRs: A Better Way to Review Code](#) — Travis DePrato, Aviator **MEDIUM**
- [Stacked Pull Requests](#) — Matt Rickard **LOW**
- [The Standard of Code Review](#) — Google engineering practices documentation **HIGH**
- [What to Look For In a Code Review](#) — Google engineering practices documentation **HIGH**
- [Small CLs](#) — Google engineering practices documentation **HIGH**
- [Conventional Comments](#) — Conventional Comments **MEDIUM**
- [Speed of Code Reviews](#) — Google engineering practices documentation **HIGH**
- [From Async Code Reviews to Co-Creation Patterns](#) — Dragan Stepanović, InfoQ **MEDIUM**
- [Trunk Based Development: Stop PR Reviews!](#) — Daniel Truemper **LOW**
- [Mob Programming — key issues it can solve, benefits, and learnings](#) — Futurice **MEDIUM**
- [On Pair Programming](#) — Birgitta Böckeler and Nina Siessegger, martinowler.com **MEDIUM**
- [Code reviews? What about pair programming, and trunk-based development?](#) — Curtis Einsmann, Medium **LOW**

2.52 V52 — Code review's measured function, and the pull request's place in its history

This note's central claim is that code review's stated purpose and its measured output diverge: practitioners say review exists to find defects, but in the best-sourced study of the practice — Bacchelli and Bird's mixed-method study at Microsoft, which hand-classified 570 review comments from 200 threads and surveyed 165 managers and 873 programmers — defect comments were only the fourth most common category, at 14%, while what review actually produced in volume was code improvements, knowledge

transfer, team awareness and alternative solutions. A companion finding from the same study treats review as a comprehension task whose bottleneck is understanding the change, not diligence in spotting bugs, and a third strand, from a ten-month, 2,500-review Cisco study, shows defect yield collapsing sharply past roughly 200–400 lines or 60–90 minutes of review. On top of that evidentiary base, the note traces the pull request itself as the fourth of four historical answers to "what is the unit of review" — after the emailed patch series (Linux/Git), the single Gerrit commit keyed by a Change-Id, and the Phabricator/stacked-diff model — and argues the PR wins on accessibility while losing on granularity, which is why practices like stacked pull requests, CODEOWNERS routing, and comment-severity labelling exist: they recover discipline the commit-based systems had by default.

The note is explicit about what it does not claim. It does not re-derive Git's object model or the mechanics of branching and merge strategy, which it assigns to two companion notes; it does not treat code review as a reliable defect gate, calling that framing a category error given the evidence; and it flags GitHub's native Stacked PRs feature, announced in private preview in April 2026, as "arriving rather than arrived," not yet generally available. It also marks its own extrapolations — for instance, the queueing-theory consequence that neither adding reviewers nor shrinking changes helps once reviewer capacity is saturated — as inference layered on cited premises, distinct from the sourced claims themselves, and treats review's erosion as a knowledge-sharing channel under AI-assisted coding as an open, still-unsettled question rather than a settled conclusion.

Referenced sources

- [Expectations, Outcomes, and Challenges of Modern Code Review](#) — Alberto Bacchelli and Christian Bird, ICSE 2013. **HIGH**
- [Modern Code Review: A Case Study at Google](#) — Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko and Alberto Bacchelli, ICSE-SEIP 2018. **HIGH**
- [About Gerrit Code Review](#) — Gerrit Code Review project. **HIGH**
- [Gerrit Code Review — Changes](#) — Gerrit Code Review project documentation. **HIGH**
- [Best Kept Secrets of Peer Code Review](#) — Jason Cohen, Steven Teleki and Eric Brown, SmartBear Software. **HIGH** for the case-study data, **MEDIUM** as a vendor publication
- [A Brief History of the Pull Request](#) — rdnlsmith (2023-11-19). **MEDIUM**
- [git-format-patch Documentation](#) — Git project. **HIGH**
- [Phabricator](#) — Wikipedia. **MEDIUM**

- [GitHub invokes spirit of Phabricator with preview of Stacked PRs](#) — Tim Anderson, The Register (2026-04-14). **MEDIUM**
- [Stacked Diffs Versus Pull Requests](#) — Jackson Gabbard (2018-09-29). **MEDIUM**
- [About code owners](#) — GitHub Docs. **HIGH**
- [Code Owners](#) — GitLab Docs. **HIGH**
- [Stacked Diffs \(and why you should know about them\)](#) — Gergely Orosz, The Pragmatic Engineer (2023-10-17). **MEDIUM**
- [In Praise of Stacked PRs](#) — Ben Congdon (2022-07-17). **MEDIUM**
- [Stacked PRs: A Better Way to Review Code](#) — Travis DePrato, Aviator (2025-03-30). **MEDIUM**
- [Stacked Pull Requests](#) — Matt Rickard (2022-07-18). **LOW**
- [The Standard of Code Review](#) — Google engineering practices documentation. **HIGH**
- [What to Look For In a Code Review](#) — Google engineering practices documentation. **HIGH**
- [Small CLs](#) — Google engineering practices documentation. **HIGH**
- [Conventional Comments](#) — Conventional Comments. **MEDIUM**
- [Speed of Code Reviews](#) — Google engineering practices documentation. **HIGH**
- [From Async Code Reviews to Co-Creation Patterns](#) — Dragan Stepanović, InfoQ (2022-11-08). **MEDIUM**
- [Trunk Based Development: Stop PR Reviews!](#) — Daniel Truemper (2023-08-22). **LOW**
- [Mob Programming — key issues it can solve, benefits, and learnings](#) — Futurice. **MEDIUM**
- [On Pair Programming](#) — Birgitta Böckeler and Nina Siessegger, martinowler.com. **MEDIUM**
- [Code reviews? What about pair programming, and trunk-based development?](#) — Curtis Einsmann, Medium (2022-09-12). **LOW**

2.53 V60 — Code review and the pull request as a review mechanism

This note argues that code review and the pull request are not Git features but social conventions layered on top of it, and that the pull request is only one historical answer — the fourth of four — to the question of what unit a team reviews. It traces that lineage from the emailed patch series used by Linux since 1991, through Gerrit's one-commit-per-change model (2008) and Phabricator's stacked diffs (2007), to GitHub's branch-

sized pull request (2008), arguing the PR wins on accessibility and loses on review granularity. The core empirical claim rests on three independently sourced findings: Bacchelli and Bird's study of code review at Microsoft found that defect-finding, though the top-stated motivation, produced only 14% of review comments, while the dominant measured output was knowledge transfer, team awareness and alternative solutions; the same study found comprehension, not diligence, is the real bottleneck; and the Cisco/SmartBear study of 2,500 reviews found defect detection collapses after roughly 400 lines or 90 minutes of review. From these the note derives practical mechanisms — CODEOWNERS routing, stacked pull requests as a workaround for the PR's branch-sized unit, severity-labelled comments, and treating review latency as a queueing problem governed by utilisation rather than a fixed delay — and compares asynchronous PR review against pair and mob programming as a synchronous alternative that removes review wait entirely.

The note is explicit about what it does not claim. It does not argue code review is a reliable defect gate — it argues the opposite, that treating it as one is a category error, and that automating review down to defect-catching discards the knowledge-transfer function that is most of its measured value. It does not re-derive the Git object model or branch/merge mechanics, which it assigns to two companion notes, and it does not extend its AI-authorship observations beyond noting that review's knowledge-transfer role appears to be eroding where AI writes most of the code — a point it marks as the author's own inference rather than a settled finding. Where sources disagree, such as competing size thresholds from the same Cisco study, the note treats them as a paired recommendation-and-ceiling rather than a contradiction, and it flags its own inferential steps explicitly rather than presenting them as established results.

Referenced sources

- [Expectations, Outcomes, and Challenges of Modern Code Review](#) — Alberto Bacchelli and Christian Bird, ICSE 2013 **HIGH**
- [Modern Code Review: A Case Study at Google](#) — Sadowski, Söderberg, Church, Sipko and Bacchelli, ICSE-SEIP 2018 **HIGH**
- [About Gerrit Code Review](#) — Gerrit Code Review project **HIGH**
- [Gerrit Code Review — Changes](#) — Gerrit Code Review project documentation **HIGH**
- [Best Kept Secrets of Peer Code Review](#) — Jason Cohen, Steven Teleki and Eric Brown, SmartBear Software **HIGH** for the case-study data, **MEDIUM** as a vendor publication
- [A Brief History of the Pull Request](#) — rdnlsmith (2023-11-19) **MEDIUM**
- [git-format-patch Documentation](#) — Git project **HIGH**
- [Phabricator](#) — Wikipedia **MEDIUM**

- [GitHub invokes spirit of Phabricator with preview of Stacked PRs](#) — Tim Anderson, The Register (2026-04-14) **MEDIUM**
- [Stacked Diffs Versus Pull Requests](#) — Jackson Gabbard (2018-09-29) **MEDIUM**
- [About code owners](#) — GitHub Docs **HIGH**
- [Code Owners](#) — GitLab Docs **HIGH**
- [Stacked Diffs \(and why you should know about them\)](#) — Gergely Orosz, The Pragmatic Engineer (2023-10-17) **MEDIUM**
- [In Praise of Stacked PRs](#) — Ben Congdon (2022-07-17) **MEDIUM**
- [Stacked PRs: A Better Way to Review Code](#) — Travis DePrato, Aviator (2025-03-30) **MEDIUM**
- [Stacked Pull Requests](#) — Matt Rickard (2022-07-18) **LOW**
- [The Standard of Code Review](#) — Google engineering practices documentation **HIGH**
- [What to Look For In a Code Review](#) — Google engineering practices documentation **HIGH**
- [Small CLs](#) — Google engineering practices documentation **HIGH**
- [Conventional Comments](#) — Conventional Comments **MEDIUM**
- [Speed of Code Reviews](#) — Google engineering practices documentation **HIGH**
- [From Async Code Reviews to Co-Creation Patterns](#) — Dragan Stepanović, InfoQ (2022-11-08) **MEDIUM**
- [Trunk Based Development: Stop PR Reviews!](#) — Daniel Truemper (2023-08-22) **LOW**
- [Mob Programming — key issues it can solve, benefits, and learnings](#) — Futurice **MEDIUM**
- [On Pair Programming](#) — Birgitta Böckeler and Nina Siessegger, martinowler.com **MEDIUM**
- [Code reviews? What about pair programming, and trunk-based development?](#) — Curtis Einsmann, Medium (2022-09-12) **LOW**

2.54 V61 — Code Review and the Pull Request as a Review Mechanism

This note argues that code review — manual inspection of a change by someone other than its author — is a social convention built atop Git rather than a Git feature, and that its modern (informal, asynchronous, change-focused) form rests on three measured findings. A Microsoft study of 570 hand-classified review comments found that "defects" made up only 14% of comments against "code improvements" at 29%, meaning review

chiefly delivers knowledge transfer, team awareness and alternative solutions rather than bug-catching. The same study found comprehension, not diligence, to be the bottleneck: 91% of programmers surveyed said reviewing unfamiliar files takes longer. A ten-month, 2,500-review case study at Cisco found defect detection collapses with size and time — no defect was found after 90 minutes of review in any of three styles studied, supporting a 200-400 line ceiling. The note traces the "reviewable unit" through four historical forms — the emailed patch series, Gerrit's commit-keyed change, Phabricator's single-commit diff, and the branch-sized GitHub/GitLab pull request — arguing the pull request wins on accessibility but loses granularity, which is why stacked PRs, CODEOWNERS, severity-labeled comments and queue limits exist: they buy back what commit-keyed systems provide for free.

The note explicitly rejects treating review as a defect gate, calling that framing "a category error" since only a small share of comments find bugs, mostly simple ones. It takes no position on which reviewable-unit model is objectively best, presenting the comparison as trade-offs between onboarding cost and granularity rather than a verdict. Claims that go beyond its cited sources are marked explicitly, as "[inference]" or "Confident," rather than stated as established fact — for example, that a WIP limit is the most robust control on review-queue wait time. It treats GitHub's native stacked-PR support as unsettled, a private preview rather than a shipped feature, and treats review's knowledge-transfer value under AI-assisted coding as currently eroding rather than resolved. Scoped as reference grounding for a planned teaching deck, it deliberately excludes AI-authored PR volume and branch-lifetime arguments, routing those to companion notes.

Referenced sources

- [Expectations, Outcomes, and Challenges of Modern Code Review](#) — Alberto Bacchelli and Christian Bird HIGH
- [Modern Code Review: A Case Study at Google](#) — Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko and Alberto Bacchelli HIGH
- [About Gerrit Code Review](#) — Gerrit Code Review project HIGH
- [Gerrit Code Review — Changes](#) — Gerrit Code Review project documentation HIGH
- [Best Kept Secrets of Peer Code Review](#) — Jason Cohen, Steven Teleki and Eric Brown; SmartBear Software HIGH MEDIUM
- [A Brief History of the Pull Request](#) — rdnlsmith MEDIUM
- [git-format-patch Documentation](#) — Git project HIGH
- [Phabricator](#) — Wikipedia MEDIUM

- [GitHub invokes spirit of Phabricator with preview of Stacked PRs](#) — Tim Anderson, The Register **MEDIUM**
- [Stacked Diffs Versus Pull Requests](#) — Jackson Gabbard **MEDIUM**
- [About code owners](#) — GitHub Docs **HIGH**
- [Code Owners](#) — GitLab Docs **HIGH**
- [Stacked Diffs \(and why you should know about them\)](#) — Gergely Orosz, The Pragmatic Engineer **MEDIUM**
- [In Praise of Stacked PRs](#) — Ben Congdon **MEDIUM**
- [Stacked PRs: A Better Way to Review Code](#) — Travis DePrato, Aviator **MEDIUM**
- [Stacked Pull Requests](#) — Matt Rickard **LOW**
- [The Standard of Code Review](#) — Google engineering practices documentation **HIGH**
- [What to Look For In a Code Review](#) — Google engineering practices documentation **HIGH**
- [Small CLs](#) — Google engineering practices documentation **HIGH**
- [Conventional Comments](#) — Conventional Comments **MEDIUM**
- [Speed of Code Reviews](#) — Google engineering practices documentation **HIGH**
- [From Async Code Reviews to Co-Creation Patterns](#) — Dragan Stepanović, InfoQ **MEDIUM**
- [Trunk Based Development: Stop PR Reviews!](#) — Daniel Truemper **LOW**
- [Mob Programming — key issues it can solve, benefits, and learnings](#) — Futurice **MEDIUM**
- [On Pair Programming](#) — Birgitta Böckeler and Nina Siessegger, martinowler.com **MEDIUM**
- [Code reviews? What about pair programming, and trunk-based development?](#) — Curtis Einsmann, Medium **LOW**

2.55 V62 — Git's content-addressable object model and design trade-offs

This note's central claim is that Git is not fundamentally a version-control system but a content-addressable filesystem with a version-control interface built on top — and that this interface is a leaky abstraction, best taught from the data model up rather than from the commands down. It grounds the claim in a small, closed set of primitives: four object types (blob, tree, commit, tag), each named by the hash of its own content and stored under `.git/objects`, linked into a Merkle DAG so a single commit hash transitively authenticates the entire tree and history behind it; and a separate, mutable

layer of refs — branches and `HEAD` — that are the only things Git ever edits in place, with every command either adding objects or moving refs. The evidence comes mainly from Git's own official documentation (Pro Git's internals chapters), corroborated by MIT's Missing Semester notes and Mary Rose Cook's essay "Git from the Inside Out," and demonstrated with a plumbing-command sequence (`hash-object`, `write-tree`, `commit-tree`, `update-ref`) that reconstructs `add / commit / branch` from first principles. It also traces the design to Torvalds writing Git in days in April 2005 after losing free BitKeeper access, explaining why the tool optimises for speed, local operation, and cryptographic integrity over ergonomics — trade-offs now inherited by the roughly 95% of developers who use Git as their default.

The note is explicit about what it does not claim. It does not claim Git tracks renames as fact: only a similarity heuristic applied at diff time, a choice Torvalds defended in a 2005 mailing-list message quoted in the note. It does not conflate snapshots with delta storage, separating the conceptual model (whole-tree snapshots) from the packfile layer (delta compression) — calling that conflation the most common misconception among former Subversion users. On the pending SHA-1-to-SHA-256 transition, it treats timing as genuinely unsettled: Git's own governing document states no release date is planned for Git 3.0, while secondary write-ups found during research claim SHA-256 is already default and Git 3.0 targets late 2026 — a direct conflict the note flags rather than resolves, preferring the primary source and treating the disagreement as a worked example of source grading. It also records an open backfill item: two secondary articles behind that contradiction have not yet been read individually to attribute the misstatement to a named author.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / git-scm.com HIGH
- [Git — BreakingChanges documentation](#) — The Git project (fetched 2026-09-23) HIGH
- [Git — Wikipedia](#) — Wikipedia contributors (fetched 2026-09-23) MEDIUM

- [Re: Handling renames](#) — Linus Torvalds, [git mailing list](#), 14 April 2005 — Linus Torvalds / [public-inbox.org git archive](#) (2005) **HIGH**
- [Version Control \(Git\)](#) — MIT Missing Semester — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL **HIGH**
- [Git from the Inside Out](#) — Mary Rose Cook (also published at Code Words, Recurse Center) **HIGH**
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / Onward! 2013 (ACM) **HIGH**
- [Jujutsu — Git comparison](#) — The Jujutsu project **HIGH**
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project **HIGH**
- [Tracking renamed files in Git](#) — Ole Begemann (2025) **MEDIUM**
- [Git — partial-clone documentation](#) — The Git project **HIGH**
- [git-lfs/docs/spec.md](#) — The Git LFS project **HIGH**
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / The GitHub Blog **HIGH**
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / Protocol Labs (2014) **MEDIUM**
- [Content-Addressing File System Objects](#) — Nix Reference Manual — The Nix project **HIGH**
- [pcottle/learnGitBranching](#) — Peter Cottle **HIGH**
- [Learning resources: Version control with Git](#) — Princeton Research Computing **MEDIUM**
- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**
- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance](#) — GitLab Docs — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**
- [Migrate to Git 3.0, SHA-256 and reftables](#) — SitePoint (cited in the note's Grounding gap as an unread backfill source; the note grades no credibility for it)
- [LWN.net Articles/1094575](#) — LWN.net (cited in the note's Grounding gap as an unread backfill source; the note grades no credibility for it)

2.56 V63 — Git's Content-Addressable Object Model and Design Trade-offs

This note argues that Git is best understood not as a version-control tool but as a content-addressable filesystem with a version-control interface layered on top, and that the interface is a leaky abstraction best taught bottom-up from the underlying data model rather than top-down from the command line. It lays out that model as four object types — blobs, trees, commits, and tags — each named by the hash of its own content, linked into a Merkle DAG so that a single commit hash transitively authenticates an entire project's history, with mutable references (branches, HEAD) as the only layer that is ever edited in place; every other Git object is immutable and every command either adds objects or moves refs. It traces the tool's origin to Linus Torvalds writing it in days in April 2005 after Linux kernel development lost free access to BitKeeper, and connects that origin to design choices still visible today: Git stores whole-tree snapshots rather than diffs (deduplicating unchanged blobs and trees automatically), optimizes hard for local, offline operation, and treats branches as single-hash files that are essentially free to create. The note also surveys where this model strains — large binaries, very large monorepos, and no native tracking of file renames — and the mitigations layered on top, such as partial clone, sparse checkout, the commit-graph file, and Git LFS, describing each as a bolt-on rather than a change to the core model.

The note is explicit about what remains unsettled or outside its scope. It does not claim Git's command-line interface is well designed — it cites a peer-reviewed conceptual-design analysis identifying specific violations, such as staging and working-tree operations not being orthogonal — and treats the CLI's difficulty as a documented finding rather than a matter of taste. On the pending SHA-1-to-SHA-256 and refable transition, it explicitly flags a conflict between Git's own primary breaking-changes documentation, which states there is no planned release date for Git 3.0, and secondary web sources claiming SHA-256 is already the default and that Git 3.0 targets a late-2026 release; it sides with the primary source and marks the timeline as an open question rather than resolving it. It similarly declines to adjudicate whether Git's rename-tracking gap or its lack of forward references from parent to child commits are real defects versus inherent trade-offs of the content-addressed model, presenting both sides (including a named critic's polemical objection) without resolving which framing is correct. The note is reference grounding for a planned course topic, not the finished teaching material itself.

Referenced sources

- [Pro Git — Git Internals: Git Objects](#) — Scott Chacon & Ben Straub / [git-scm.com](#)

HIGH

- [Pro Git — A Short History of Git](#) — Scott Chacon & Ben Straub / [git-scm.com](#) **HIGH**
- [Pro Git — What is Git?](#) — Scott Chacon & Ben Straub / [git-scm.com](#) **HIGH**
- [Pro Git — Git Internals: Git References](#) — Scott Chacon & Ben Straub / [git-scm.com](#) **HIGH**
- [Pro Git — Git Internals: Packfiles](#) — Scott Chacon & Ben Straub / [git-scm.com](#) **HIGH**
- [Pro Git — Git Internals: Plumbing and Porcelain](#) — Scott Chacon & Ben Straub / [git-scm.com](#) **HIGH**
- [Git — BreakingChanges documentation](#) — The Git project (fetched 2026-09-23) **HIGH**
- [Git — Wikipedia](#) — Wikipedia contributors (fetched 2026-09-23) **MEDIUM**
- [Re: Handling renames](#) — Linus Torvalds, [git mailing list](#), 14 April 2005 — Linus Torvalds / [public-inbox.org](#) [git archive](#) (2005) **HIGH**
- [Version Control \(Git\) — MIT Missing Semester](#) — Anish Athalye, Jon Gjengset & Jose Javier Gonzalez Ortiz / MIT CSAIL **HIGH**
- [Git from the Inside Out](#) — Mary Rose Cook **HIGH**
- [What's Wrong with Git? A Conceptual Design Analysis](#) — Santiago Perez de Rosso & Daniel Jackson / [Onward! 2013 \(ACM\)](#) **HIGH**
- [Jujutsu — Git comparison](#) — The Jujutsu project **HIGH**
- [jj-vcs/jj — Jujutsu version control system](#) — The Jujutsu project **HIGH**
- [Tracking renamed files in Git](#) — Ole Begemann (2025) **MEDIUM**
- [Git — partial-clone documentation](#) — The Git project **HIGH**
- [git-lfs/docs/spec.md](#) — The Git LFS project **HIGH**
- [Get up to speed with partial clone and shallow clone](#) — Derrick Stolee / [The GitHub Blog](#) **HIGH**
- [IPFS — Content Addressed, Versioned, P2P File System](#) — Juan Benet / [Protocol Labs](#) (2014) **MEDIUM**
- [Content-Addressing File System Objects](#) — [Nix Reference Manual](#) — The Nix project **HIGH**
- [pcottle/learnGitBranching](#) — Peter Cottle **HIGH**
- [Learning resources: Version control with Git](#) — Princeton Research Computing **MEDIUM**
- [git concepts simplified](#) — Sitaram Chamarty **MEDIUM**
- [How I teach Git](#) — Thomas Broyer (2023) **MEDIUM**
- [Git Is Not Fine](#) — Bill Jings **LOW**
- [Improving monorepo performance](#) — [GitLab Docs](#) — GitLab **HIGH**
- [Think Like \(a\) Git](#) — Sam Livingston-Gray **MEDIUM**

- <https://www.sitepoint.com/migrate-to-git-3-0-sha-256-and-reftables/> — cited in the note's Grounding gap as an unread secondary source to backfill; ungraded
- <https://lwn.net/Articles/1094575/> — cited in the note's Grounding gap as an unread secondary source to backfill; ungraded